

National Engineering Degree

Academic year 2025/2026



Computer Science and Information Systems

Software Engineering and Data Science

Final-Year Engineering Project Report

Discipline: Web development

Enterprise Knowledge Intelligence Platform based on Artificial Intelligence

*Plateforme d'Intelligence de Connaissance d'Entreprise basée sur
l'Intelligence Artificielle*

Prepared by	Adam Farhat
Host organization	HyprCraft Solutions
Industrial supervisor	mohamed aziz rezgui
Academic supervisor	Jacem Mhenni
Defense date	July 24, 2026
Jury chair	Haifa Mohdhi
Examiner	Hajer Maraoui

Dedication

I dedicate this work to my parents, whose unconditional love, support, and sacrifices have been the foundation of all my achievements. Their encouragement has been a constant source of strength throughout my academic journey.

To my family and friends, I express my sincere gratitude for their continuous support, patience, and understanding during both the difficult and rewarding moments of this work.

I also dedicate this project to my own efforts and perseverance, which allowed me to overcome challenges and stay committed to my goals.

Finally, I extend my deepest appreciation to my professors, whose guidance, expertise, and dedication have greatly contributed to my learning and to the successful completion of this project.

Acknowledgements

I would like to express my sincere gratitude to everyone who contributed to the success of this final-year engineering project.

My thanks go first to my industrial supervisor at HyprCraft Solutions, for providing a meaningful problem statement, regular feedback, and access to the organizational context that shaped the requirements of the Knowledge Platform.

I am equally grateful to my academic supervisor at ESPRIM, Jacem Mhenni, for methodological guidance, critical review of the technical choices, and support in aligning the deliverable with engineering-degree expectations.

I also thank the examination jury for their time and constructive evaluation, as well as the ESPRIM faculty who prepared us for this capstone experience.

Finally, I acknowledge my colleagues and peers whose discussions, testing, and code reviews improved the robustness of the implemented solution.

Abstract

Organizations produce large volumes of internal documentation—policies, technical guides, reports, and procedures—yet employees often struggle to find reliable information quickly. Knowledge is scattered across storage systems, search remains keyword-oriented, and access rules are difficult to enforce consistently. These constraints slow decision-making and increase dependence on informal channels.

This report presents the **Knowledge Platform**, an enterprise web application that centralises document management and provides intelligent search and question-answering grounded in organisational content. The solution combines a modern full-stack web architecture with hybrid retrieval techniques and department-scoped access control, so users receive answers supported by traceable sources rather than unsupported generated text.

The main contribution is a complete knowledge lifecycle: secure ingestion of heterogeneous documents, semantic indexing, hybrid retrieval combining meaning-based and keyword matching, streaming answers with citations, conversation history, administrative governance, and user feedback to monitor answer quality over time. The platform supports three organisational roles—Administrator, Manager, and Employee—with permissions aligned to department structure.

The work was conducted at HyprCraft Solutions using an Agile Scrum methodology across five development sprints. Validation combined automated integration testing, repeatable deployment packaging, manual browser scenarios, and an evaluation workflow for retrieval-augmented answers. Results indicate that the platform meets its functional and non-functional targets while remaining suitable for iterative improvement in a real organisational setting.

Keywords: Retrieval-Augmented Generation, Enterprise Search, Vector Database,

Access Control, Full-Stack Web Application, Large Language Models

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
Table of contents	viii
List of figures	xii
List of tables	xiii
List of abbreviations and acronyms	xiv
General introduction	1
1 State of the Art	3
1.1 Scientific and Technical Context	3
1.1.1 The Enterprise Knowledge Lifecycle Problem	3
1.1.2 Limitations of Traditional Document Systems	4
1.2 Information Retrieval: From Keywords to Semantics	4
1.2.1 Classical Retrieval Models	4
1.2.2 Dense Retrieval and Semantic Search	5
1.2.3 Hybrid Retrieval Systems	5
1.3 Retrieval-Augmented Generation (RAG)	5
1.3.1 Concept and Architecture	5
1.3.2 Advantages of RAG	6
1.3.3 Limitations in Enterprise Context	6
1.3.4 Evaluation of RAG Systems	6

1.4	Access Control and Knowledge Governance	6
1.4.1	Role-Based Access Control (RBAC)	7
1.4.2	Challenges in Knowledge Systems	7
1.5	Enterprise Knowledge Systems and Existing Solutions	7
1.5.1	Shared Drives and Document Management Systems	7
1.5.2	Vector Databases and RAG Frameworks	8
1.5.3	Commercial AI Copilots	8
1.5.4	Knowledge Base Platforms	9
1.5.5	Enterprise AI Search Systems (Glean)	9
1.6	Comparative Analysis	9
1.7	Project Positioning	10
1.7.1	Key Contribution	11
1.8	Chapter Conclusion	11
2	Requirements and Methodology	12
2.1	Analysis of the Existing Situation and Usage Context	12
2.1.1	Host Organisation Context	12
2.1.2	System Actors	13
2.1.3	Global Use-Case Model	14
2.2	Functional and Non-Functional Requirements	15
2.2.1	Functional Requirements	15
2.2.2	Non-Functional Requirements	18
2.2.3	Access Control Model	18
2.3	Specifications and Performance Indicators	19
2.4	Selected Methodology	20
2.4.1	Agile Scrum Framework	20
2.4.2	Sprint Plan	22
2.4.3	Development and Modelling Tools	30
2.5	Chapter Conclusion	30
3	Design and Architecture	31
3.1	Overall Solution Architecture	31
3.2	Logical Subsystem Decomposition	33

3.3	Domain Data Model	34
3.4	Technology Choice Justification	34
3.5	Dynamic Interaction Design by Sprint	36
3.5.1	Sprint 1: Authentication and access control	37
3.5.2	Sprint 2: Document library and ingest	50
3.5.3	Sprint 3: Search, RAG and conversations	60
3.5.4	Sprint 4: Governance and notifications	68
3.5.5	Sprint 5: Operations, testing and deployment	84
3.6	Chapter Conclusion	87
4	Implementation	88
4.1	Hardware and Software Environment	88
4.2	Implementation Steps by Sprint	89
4.2.1	Sprint 1: Authentication and access control	90
4.2.2	Sprint 2: Document library and ingest pipeline	93
4.2.3	Sprint 3: Search, RAG and conversations	101
4.2.4	Sprint 4: Governance and notifications	103
4.2.5	Sprint 5: Operations, testing and deployment	108
4.3	Difficulties Encountered and Solutions	110
4.4	Produced Technical Deliverables	110
4.5	Chapter Conclusion	111
5	Validation	112
5.1	Validation plan	112
5.2	Protocols and experimental conditions	113
5.3	Presentation of Results	113
5.3.1	KPI validation results	113
5.3.2	Integration test results by sprint domain	114
5.3.3	RAG evaluation results	116
5.3.4	Manual UX validation	117
5.4	Critical discussion	121
5.4.1	Strengths	121
5.4.2	Deviations from targets	121

5.4.3	Limitations	122
5.4.4	Threats to validity and confidence level	122
5.4.5	Perspectives	122
5.5	Chapter Conclusion	123
General conclusion and perspectives		124
References		126
A Validation Summary		128
B Scientific Integrity and Reproducibility		132
B.1	Declaration of AI Tool Usage	132
B.2	Reproducibility Details	132
B.3	Integrity Checklist	133

List of Figures

2.1	Global use-case diagram of the Knowledge Platform	14
2.2	Scrum sprint timeline — iterative delivery across five sprints	23
3.1	Overall platform architecture	32
3.2	Global domain class diagram (persisted entities and main relationships) .	34
3.3	Sprint 1 use-case diagram — Authentication and access control	38
3.4	Sprint 1 domain class diagram — Authentication and access control . . .	39
3.5	User sign-in	40
3.6	Silent session renewal	42
3.7	User sign-out	43
3.8	Forgotten password recovery	44
3.9	Changing password while signed in	45
3.10	Updating the user profile	46
3.11	Blocking access to a restricted feature	47
3.12	Loading the signed-in user’s context	48
3.13	Registration information page	49
3.14	Restricted feature explanation page	49
3.15	Sprint 2 use-case diagram — Document library and ingest	50
3.16	Sprint 2 domain class diagram — Document library and ingest	51
3.17	Uploading a document	52
3.18	Background document processing	53
3.19	Reprocessing an existing version	54
3.20	Downloading a document file	55
3.21	Browsing the document library	56
3.22	Opening a document detail view	57
3.23	Favourites and recently viewed documents	58
3.24	Updating document metadata	59

3.25	Publishing a new document version	60
3.26	Sprint 3 use-case diagram — Search, RAG and conversations	61
3.27	Sprint 3 domain class diagram — Search, RAG and conversations	62
3.28	Semantic search	63
3.29	Asking a question with AI assistance	64
3.30	Creating conversations and adding messages	65
3.31	Rating an answer	66
3.32	Managing conversation threads	67
3.33	Administrator feedback overview	68
3.34	Sprint 4 use-case diagram — Governance and notifications	69
3.35	Sprint 4 domain class diagram — Governance and notifications	70
3.36	Automatic notification when a document is added	71
3.37	Sending a manual announcement	72
3.38	Reading and managing notifications	73
3.39	Manager department overview	74
3.40	Creating a user account	75
3.41	Merging two departments	76
3.42	Replacing a user’s department access in bulk	76
3.43	Creating a department	77
3.44	Granting or revoking single department access	78
3.45	Platform-wide statistics	79
3.46	Activity log and export	80
3.47	Document change audit log	81
3.48	Exporting the document catalogue	82
3.49	Deleting multiple documents	83
3.50	Updating or locking a user account	84
3.51	Sprint 5 use-case diagram — Operations, testing and deployment	85
3.52	Sprint 5 domain class diagram — Operations, testing and deployment	85
3.53	Service health check	86
4.1	Sign-in page	90
4.2	Forgot password	91
4.3	Profile settings	91

4.4	Employee home hub	92
4.5	Manager home hub	92
4.6	Administrator home hub (light)	93
4.7	Dark theme	93
4.8	Department library browse	94
4.9	Document detail side panel	95
4.10	Document upload modal	96
4.11	Recent documents	97
4.12	Favorites view	98
4.13	Table view	99
4.14	Ingest processing status	100
4.15	In-app PDF preview modal	101
4.16	Version archive dialog	101
4.17	Ask / RAG chat with citations	102
4.18	Answer feedback controls	103
4.19	Notifications inbox	104
4.20	Send notification modal	104
4.21	Manager department overview	105
4.22	Administration hub	105
4.23	Administrator user directory	106
4.24	Department management	106
4.25	Administrator document library	106
4.26	Document audit log	107
4.27	Auth activity log	107
4.28	System KPI dashboard	108
5.1	Integration test run	116
5.2	RAG quick eval run	117
5.3	Login error state	119
5.4	Access restricted page	119
5.5	Ingest READY status	120
5.6	Ask with citations	120
5.7	Manager department view	121

List of Tables

1.1	Comparative analysis of enterprise knowledge management solutions . . .	10
2.1	System actors and their primary capabilities	13
2.2	Functional requirements with acceptance criteria	15
2.3	Non-functional requirements	18
2.4	Key performance indicators	20
2.5	Agile Scrum sprint plan	22
2.6	Sprint 1 backlog: authentication and access control	24
2.7	Sprint 2 backlog: document library and ingest	26
2.8	Sprint 3 backlog: search, RAG and conversations	27
2.9	Sprint 4 backlog: governance and notifications	28
2.10	Sprint 5 backlog: operations, testing and deployment	29
2.11	Development and modelling tools	30
3.1	Subsystem responsibilities	33
3.2	Decision-support matrix for platform architecture (scores: 1 = weak, 3 = strong)	35
3.3	Requirements-to-design traceability (excerpt)	36
4.1	Development and deployment stack (detailed)	89
4.2	Index of implementation sequence diagrams (Chapter ??, §??)	109
4.3	Implementation challenges and resolutions	110
5.1	Experimental setup for validation	113
5.2	KPI validation results	114
5.3	Validation test summary by sprint domain	115
5.4	Manual UX validation summary	118
A.1	Requirements Traceability Matrix	129

A.3	Technology Architecture Comparison Matrix	130
A.4	Functional Validation Test Summary	131
B.1	Reproducibility parameters	133

List of abbreviations and acronyms

Acronym	Definition
AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Matching 25 (lexical ranking function)
BullMQ	Redis-backed job queue library used for document ingest
CORP	Cross-Origin Resource Policy (browser security header)
CSP	Content Security Policy
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DTO	Data Transfer Object
FR	Functional Requirement
HNSW	Hierarchical Navigable Small World (approximate nearest-neighbor index)
HTTP	Hypertext Transfer Protocol
HyDE	Hypothetical Document Embeddings (retrieval augmentation technique)
IDE	Integrated Development Environment
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
MIME	Multipurpose Internet Mail Extensions (file type identification)
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping (Prisma)
PFE	Final-Year Project (Projet de Fin d'Études)
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
SSE	Server-Sent Events
TTL	Time To Live (cache expiration)
UML	Unified Modeling Language

General introduction

In recent years, organizations have undergone an accelerated digital transformation, leading to an exponential increase in the production of digital documents such as technical reports, internal procedures, contracts, and operational guidelines. While this information is essential for daily decision-making, it is often dispersed across multiple systems, shared drives, and communication channels, making it difficult to access efficiently.

This fragmentation of knowledge creates a significant operational challenge. Employees frequently struggle to locate relevant information, rely on informal communication channels, or recreate existing knowledge, which results in time loss, inconsistency, and reduced productivity. This issue becomes even more critical in engineering environments where accurate and timely access to information is essential.

Beyond simple document storage, organizations now require intelligent systems capable of organizing knowledge, controlling access according to organizational structure, and enabling efficient retrieval of relevant information. At the same time, the increasing use of artificial intelligence introduces new opportunities for improving how users interact with internal knowledge bases, particularly through natural language interfaces.

This final-year project was conducted during an internship at HyprCraft Solutions. It aims to explore and address the challenges of enterprise knowledge management by designing and implementing a platform that centralizes organizational documents and improves information accessibility through intelligent search and interaction mechanisms.

The main problem addressed in this work can be stated as follows: *How can an engineering organization structure its internal knowledge, ensure secure and role-based access to information, and enable efficient retrieval of reliable answers from its documentation?*

To address this problem, the project focuses on the following objectives:

1. design a secure platform supporting multiple user roles with controlled access to organizational documents;
2. improve information retrieval from heterogeneous document sources;
3. enable more efficient access to knowledge through intelligent assistance mechanisms;
4. ensure system reliability through testing and validation approaches;
5. provide a reproducible and deployable software solution.

The scope of this project includes the design and implementation of a full-stack web application for document management and knowledge access. It covers authentication, document organization, and intelligent retrieval functionalities, while ensuring structured access based on organizational roles and departments.

However, certain aspects remain outside the scope of this work, including mobile application development, large-scale distributed infrastructure deployment, and external enterprise authentication systems.

This report is organized as follows. Chapter ?? presents the state of the art and positions the project within existing solutions. Chapter ?? describes the requirements analysis and methodology. Chapter ?? details the system design and architecture. Chapter ?? presents the implementation process. Chapter ?? covers testing and validation. Finally, the conclusion summarizes the contributions and discusses future work.

CHAPTER 1

State of the Art and Project Positioning

This chapter surveys the scientific and industrial landscape relevant to enterprise knowledge management, information retrieval, and grounded answer generation over organisational documents. It analyses existing solutions comparatively and positions the Knowledge Platform within this context. Implementation details, requirements, and validation are deliberately out of scope here; they are addressed in subsequent chapters.

1.1 Scientific and Technical Context

1.1.1 The Enterprise Knowledge Lifecycle Problem

Modern organisations face a fundamental challenge in managing knowledge at scale. On one hand, they continuously generate large volumes of heterogeneous digital assets, including technical documentation, procedures, contracts, internal reports, spreadsheets, emails, and operational guidelines. On the other hand, these assets are distributed across multiple disconnected systems such as shared drives, email platforms, messaging tools, and departmental repositories.

This fragmentation creates a structural inefficiency that goes beyond retrieval. Enterprise knowledge is difficult not only to find, but also to:

- collect in a structured and consistent manner;

- centralize across heterogeneous tools and departments;
- organize according to business logic and ownership;
- govern through access control policies;
- retrieve in a meaningful and context-aware way.

As a result, employees often rely on informal communication or personal experience instead of documented knowledge. Studies show that knowledge workers spend a significant portion of their time searching for information rather than performing productive tasks [?].

1.1.2 Limitations of Traditional Document Systems

Traditional enterprise tools such as Google Drive, OneDrive, and Microsoft SharePoint were primarily designed for storage and collaboration rather than intelligent knowledge management.

While they provide reliable file storage and basic access control, they suffer from key limitations:

- lack of semantic understanding of content;
- keyword-based search that ignores meaning and context;
- weak knowledge structuring and duplication of information;
- limited ability to answer questions directly;
- coarse-grained permission models not aligned with organizational logic.

These systems function as passive repositories rather than active knowledge systems.

1.2 Information Retrieval: From Keywords to Semantics

1.2.1 Classical Retrieval Models

Traditional information retrieval relies on lexical matching approaches such as TF-IDF and BM25 [?]. These methods rank documents based on term frequency and inverse document frequency, making them effective for exact keyword matching.

However, they fail to capture semantic similarity. Two expressions with identical meaning but different wording may not match, which limits their effectiveness in enterprise contexts.

1.2.2 Dense Retrieval and Semantic Search

Dense retrieval methods address this limitation by encoding queries and documents into continuous vector representations [?, ?]. This enables semantic matching between conceptually similar texts even when lexical overlap is low.

Despite their advantages, dense retrieval models also have limitations:

- weaker performance on rare or technical terms;
- sensitivity to embedding model quality;
- reduced interpretability of ranking decisions.

1.2.3 Hybrid Retrieval Systems

To overcome these limitations, modern systems adopt hybrid retrieval strategies that combine lexical and semantic approaches [?, ?]. Results are typically merged using ranking fusion techniques such as Reciprocal Rank Fusion (RRF), which balances precision and recall.

This hybrid approach is now widely considered the standard for enterprise search systems.

1.3 Retrieval-Augmented Generation (RAG)

1.3.1 Concept and Architecture

Retrieval-Augmented Generation (RAG) [?] enhances traditional retrieval systems by integrating Large Language Models (LLMs). Instead of returning documents, the system retrieves relevant passages and uses them as context for answer generation.

A typical RAG pipeline consists of:

1. a retriever that selects relevant document chunks;

2. an optional reranker that refines retrieval quality;
3. a generator that produces a grounded natural language response.

1.3.2 Advantages of RAG

RAG systems provide several advantages over standalone LLMs:

- factual grounding in external documents;
- reduced hallucination through retrieval constraints;
- ability to update knowledge without retraining models;
- improved traceability through source citations.

1.3.3 Limitations in Enterprise Context

Despite their advantages, RAG systems introduce new challenges:

- strong dependency on retrieval quality;
- sensitivity to chunking and indexing strategies;
- difficulty in systematic evaluation;
- lack of native access control mechanisms;
- complexity of production deployment.

1.3.4 Evaluation of RAG Systems

Evaluating RAG systems remains a complex task, as classical IR metrics do not capture answer quality or factual grounding. Recent research proposes LLM-based evaluation frameworks that assess faithfulness, relevance, and completeness of generated answers [?]. These methods enable automated detection of hallucinations and quality regressions in production systems.

1.4 Access Control and Knowledge Governance

1.4.1 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) [?,?] is the standard model for managing permissions in enterprise systems. Users are assigned to roles, and roles are associated with permissions over resources.

This model simplifies administration and supports organizational scalability.

1.4.2 Challenges in Knowledge Systems

In knowledge platforms, access control must be enforced beyond simple document visibility. It must govern:

- which documents can be retrieved during search;
- which sources can influence generated answers;
- cross-department boundaries and hierarchical structures;
- auditability of access and usage history.

A critical requirement is that access control must be applied at the retrieval layer to prevent information leakage in generated responses.

1.5 Enterprise Knowledge Systems and Existing Solutions

1.5.1 Shared Drives and Document Management Systems

Systems such as Google Drive, OneDrive, and Microsoft SharePoint [?] provide foundational document storage and collaboration capabilities.

Strengths:

- reliable storage and synchronization;
- basic access control mechanisms;
- widespread enterprise adoption.

Limitations:

- lack of semantic search;
- no question-answering capabilities;
- weak knowledge structuring;
- folder-based organization not aligned with knowledge flows.

1.5.2 Vector Databases and RAG Frameworks

Tools such as Pinecone, Weaviate, LangChain, and LlamaIndex [?, ?, ?] provide infrastructure for building retrieval systems.

Strengths:

- scalable semantic retrieval;
- modular RAG pipeline construction;
- flexible architecture for developers.

Limitations:

- no complete end-user system;
- no document lifecycle management;
- no access control or governance layer;
- significant engineering effort required for production use.

1.5.3 Commercial AI Copilots

Solutions such as Microsoft Copilot and ChatGPT Enterprise [?] provide conversational interfaces over enterprise data.

Strengths:

- natural language interaction;
- strong ecosystem integration;
- low adoption barrier.

Limitations:

- limited transparency in retrieval mechanisms;
- restricted control over embedding and ranking strategies;
- vendor lock-in;

- limited customization of access control logic.

1.5.4 Knowledge Base Platforms

Tools such as Guru and Tettra focus on structured knowledge management.

Strengths:

- organized documentation workflows;
- curated and validated content;
- team-oriented knowledge sharing.

Limitations:

- manual content creation required;
- no automated ingestion pipelines;
- limited or no AI-based retrieval;
- not suitable for large unstructured corpora.

1.5.5 Enterprise AI Search Systems (Glean)

Enterprise solutions such as Glean represent the most advanced commercial attempt at unified knowledge search.

Strengths:

- unified search across multiple enterprise data sources;
- natural language question answering with citations;
- strong user experience and integration capabilities.

Limitations:

- proprietary and closed architecture;
- no transparency in retrieval pipeline;
- no self-hosting or full customization;
- limited support for feedback-driven optimization.

1.6 Comparative Analysis

Table 1.1 – Comparative analysis of enterprise knowledge management solutions

Solution	Strengths	Limitations	Relevance to this work
Shared Drives (Drive, Share-Point)	Reliable storage and basic access control	No semantic search, no QA, weak structuring	Baseline storage layer only
Vector DB + RAG frameworks	Strong retrieval and modular pipelines	No full system or governance layer	Core retrieval component only
Commercial Copilots	Ease of use and enterprise integration	Black-box systems, limited control	Demonstrates demand but lacks transparency
Knowledge Base Tools	Structured documentation workflows	Manual maintenance, no scalable ingestion	Not suitable for large corpora
Glean	Unified enterprise AI search with citations	Closed system, no customization or transparency	Closest industrial solution but not open
Proposed System	Hybrid RAG + RBAC + full lifecycle + feedback loop	External LLM dependency	Full enterprise knowledge lifecycle solution

1.7 Project Positioning

The analysis reveals a structural gap in existing solutions: no system simultaneously addresses document lifecycle management, centralized governance, semantic retrieval, and controlled AI-based question answering in a unified and transparent architecture.

The **Knowledge Platform** addresses this gap through:

- centralized document ingestion and organization;
- role-based and department-scoped access control;
- hybrid lexical and semantic retrieval;
- Retrieval-Augmented Generation with source grounding;
- feedback-driven improvement mechanisms;
- full production-grade system architecture.

Unlike existing tools that focus on isolated components, this work proposes a complete and integrated enterprise knowledge lifecycle system.

1.7.1 Key Contribution

This work contributes a unified approach that ensures:

- knowledge is properly collected and centralized;
- access is securely governed and auditable;
- retrieval is hybrid and semantically aware;
- answers are grounded and traceable;
- the system is fully reproducible and production-ready.

1.8 Chapter Conclusion

The state-of-the-art analysis highlighted the limits of traditional document platforms and motivated a hybrid retrieval approach combined with retrieval-augmented answer generation under enterprise governance constraints. By comparing existing solutions and defining the project positioning, this chapter justified the need for an integrated knowledge platform combining lifecycle management, controlled access, semantic search, and traceable AI assistance. Chapter ?? translates these insights into functional and non-functional requirements and defines the Agile Scrum methodology adopted for delivery.

CHAPTER 2

Requirements Analysis, Specifications and Methodology

This chapter translates the gaps identified in Chapter ?? into a verifiable specification for the Knowledge Platform. It analyses the host-organisation context, defines functional and non-functional requirements with acceptance criteria, and establishes the Agile Scrum methodology and five-sprint delivery plan that structure the design (Chapter ??), implementation (Chapter ??), and validation (Chapter ??) chapters.

2.1 Analysis of the Existing Situation and Usage Context

2.1.1 Host Organisation Context

HyprCraft Solutions is a technology company whose teams produce and consume significant volumes of internal documentation: technical specifications, onboarding guides, project reports, API references, and operational procedures. Prior to this project, knowledge was distributed across disconnected tools — shared drives, email threads, and informal messaging channels — with no unified governance model and no mechanism for intelligent retrieval.

The key pain points identified through analysis of the existing situation were:

- **Fragmented storage:** documents resided in multiple systems with no single point of access, making comprehensive search impossible.
- **No access governance:** there was no systematic control over which teams or roles could access which documents, leading to either over-sharing or information silos.
- **Keyword-only search:** retrieval was limited to filename and keyword matching, returning documents rather than answers.
- **Onboarding friction:** new employees had no reliable way to find answers to operational questions, creating repeated interruptions to experienced colleagues.
- **No feedback mechanism:** when information found through search was incorrect or outdated, there was no way to signal this or prevent the same error from recurring.

2.1.2 System Actors

Four actor groups interact with the Knowledge Platform, as summarized in Table ?? and detailed in the global use-case diagram (Subsection ??, Figure ??):

Table 2.1 – System actors and their primary capabilities

Actor	Primary capabilities
Employee	Browse and search the document library, perform semantic search and RAG question-answering, manage personal profile, maintain conversation history, and rate assistant answers.
Manager	All Employee capabilities, plus access to the manager dashboard showing managed departments and their members, and the ability to send announcements to managed departments.
Administrator	Full platform control: user lifecycle management, department CRUD and merge, per-user restrictions, bulk operations, audit log access, KPI dashboards, activity exports, and platform-wide notifications.
System	Autonomous actor: runs the background document ingestion pipeline (extract, chunk, embed), emits automatic notifications on document and role events, and executes the RAG evaluation harness.

2.1.3 Global Use-Case Model

Figure ?? presents the global use-case model of the Knowledge Platform. It identifies primary actors (**Guest**, **Employee**, **Manager**, and **Administrator**), with the three authenticated roles generalizing an abstract **Authenticated user** actor. Use cases are grouped into authentication, document management, search and AI, notifications, and governance domains. Secondary actors (**Email**, **Gemini**, and **Background worker**) model external interactions through «notify», «invoke», and «enqueue» dependencies.

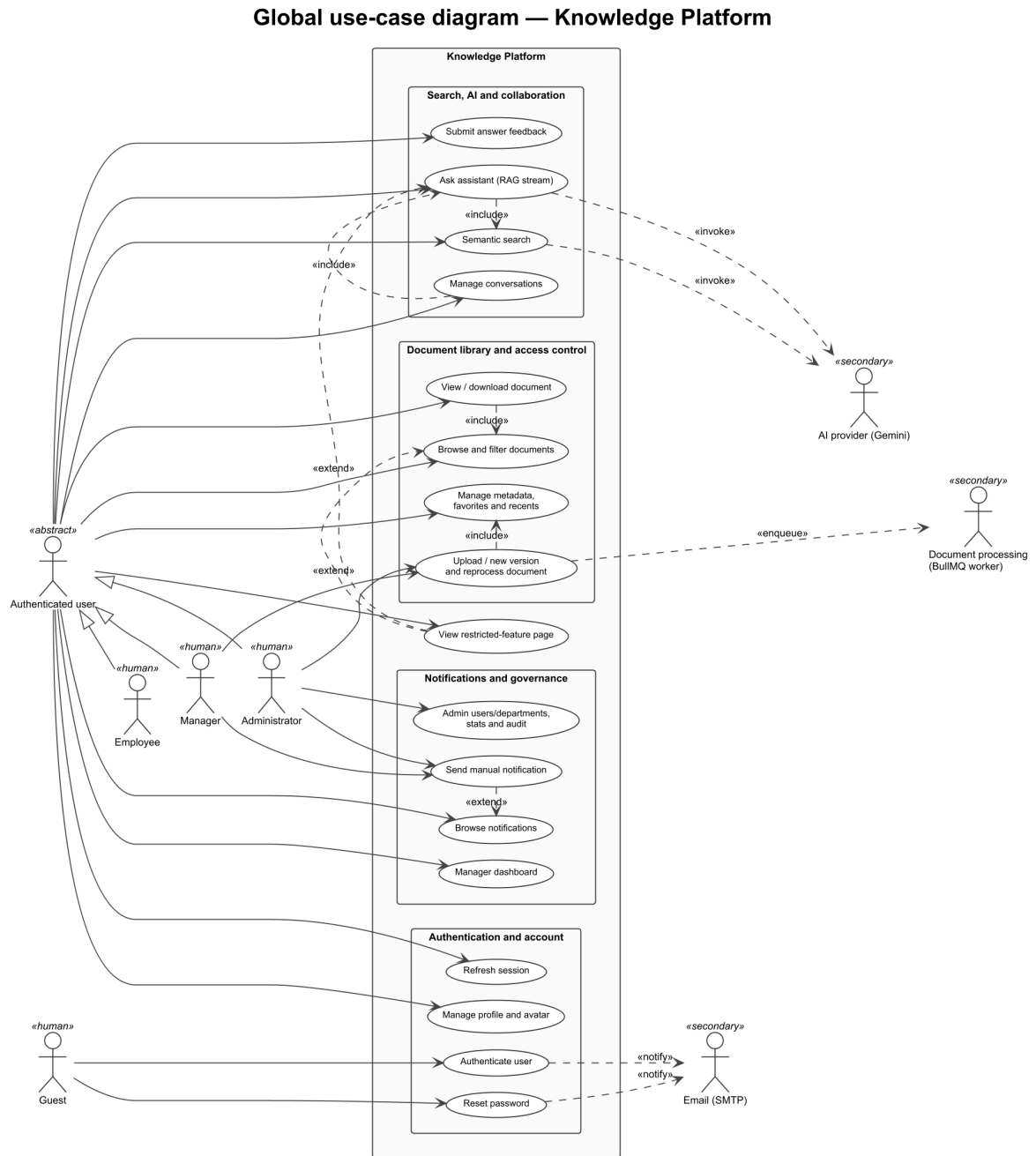


Figure 2.1 – Global use-case diagram of the Knowledge Platform

The use-case packages map directly to the functional requirement domains in Table ??: authentication (FR-01, FR-02, FR-12), document management (FR-03 to FR-05), search and AI (FR-06 to FR-08), notifications (FR-09), and governance (FR-10, FR-11).

2.2 Functional and Non-Functional Requirements

The requirements were derived from an analysis of user workflows and the intended system architecture, ensuring alignment between stakeholder needs and the design choices developed in subsequent chapters.

2.2.1 Functional Requirements

The platform defines a set of functional requirements organized around **authentication** (FR-01, FR-02), **document management** (FR-03 to FR-05), **search and AI capabilities** (FR-06 to FR-08), **notifications** (FR-09), **administration** (FR-10, FR-11), and **system-level feature controls** (FR-12). Access to all functionalities is governed by a hybrid authorization model combining Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), and per-user restriction flags (see Subsection ??).

Table 2.2 – Functional requirements with acceptance criteria

ID	Requirement	Acceptance criterion	Primary component
FR-01	JWT authentication with HttpOnly re-fresh cookie rotation	Successful login returns a short-lived access token; session refresh rotates credentials atomically; replay of a rotated token revokes the entire session family	Authentication service

ID	Requirement	Acceptance criterion	Primary component
FR-02	User profile management including avatar upload	Profile updates persist; uploaded avatars are served with cross-origin headers enabling display in the web client	Profile management service
FR-03	Document upload with multi-format extraction and async ingest	Uploaded file is stored; processing is queued; document version reaches ready state when ingestion completes	Document ingestion pipeline
FR-04	Document visibility control (ALL / DEPARTMENT / PRIVATE)	Unauthorised access returns 404 (not 403) to prevent document enumeration	Access control layer
FR-05	Document metadata: tags, favorites, recents, archive	Tag autocomplete, favourite toggle, recent views, and archive operations are all persisted	Document library service
FR-06	Semantic vector search over department-scoped document corpus	Vector similarity results ranked correctly; access-filtered; respects restrictions	Search service
FR-07	Hybrid RAG Ask with SSE streaming and source citations	Answer stream delivers source citations first, then tokens, then completion; citations reference real document passages	RAG completion service
FR-08	Conversation history with per-message feedback	Threads persisted; thumbs up/down stored; feedback statistics available to administrators	Conversation service

ID	Requirement	Acceptance criterion	Primary component
FR-09	Automatic and manual notification system with attachments	Department-scoped events emitted on document create/update/delete; manual send available to managers and administrators	Notification service
FR-10	Full administrative control: user/department CRUD, bulk operations, audit	Last-administrator safety enforced; bulk imports sanitised; audit log exportable	Administration module
FR-11	Manager dashboard scoped to manageable departments	Only departments within the manager's scope are returned; member lists are correct	Manager module
FR-12	Per-user feature restriction flags	Flags block login, document access, and AI queries; restricted users see a dedicated notice page	Authorization middleware

Chapter ?? extends this summary with design elements and validation test references (Table ??).

2.2.2 Non-Functional Requirements

Table 2.3 – Non-functional requirements

ID	Requirement	Acceptance criterion
NFR-01	Health monitoring endpoint	Health check returns success when the database and cache are reachable; otherwise returns service-unavailable with per-component detail
NFR-02	Security headers and rate limiting	Security headers enforced on all responses; login limited to 15 requests per 15 minutes per address; Ask limited to 12 requests per minute per address
NFR-03	Single-flight refresh token (web client)	Parallel requests from the same browser tab share a single in-flight refresh operation rather than issuing duplicate refresh calls
NFR-04	Docker Compose reproducibility	Container orchestration starts all four application services in a healthy state; database migrations run automatically
NFR-05	Integration test coverage	Full integration test suite passes on a live database and cache instance with seeded data
NFR-06	RAG evaluation harness	RAG evaluation workflow completes without error and produces a structured quality report

2.2.3 Access Control Model

Building on the RBAC concepts introduced in Chapter ??, the platform enforces a **hybrid authorization model** that operates at three complementary layers. This subsection specifies how the theoretical foundations are applied to the requirements defined above.

- **Role-Based Access Control (RBAC)**: every user holds one platform role
- ADMIN, MANAGER, or EMPLOYEE — that determines coarse-grained capabilities.

Administrators manage users, departments, and platform configuration (FR-10); managers access department-scoped dashboards and announcements (FR-11); employees use the document library, search, and AI assistant within their permitted scope.

- **Attribute-Based Access Control (ABAC)**: permissions are further constrained by attributes such as department membership and document visibility (**ALL**, **DEPARTMENT**, **PRIVATE**). Retrieval and RAG pipelines apply these filters before returning results, ensuring that semantic search and generated answers never expose unauthorised content (FR-04, FR-06, FR-07).
- **Per-user feature restriction flags**: administrators can disable specific capabilities for individual users — for example, blocking document library access or AI queries — via middleware that intercepts requests before business logic executes (FR-12).

This layered model ensures fine-grained control over system capabilities while maintaining scalability and security across authentication, document management, search, and AI workflows.

2.3 Specifications and Performance Indicators

The following Key Performance Indicators (KPIs) link requirements to the validation chapter (Chapter ??):

Table 2.4 – Key performance indicators

KPI	Measurement method	Target
Ingest reliability	Percentage of uploaded documents reaching ready state without failure	$\geq 95\%$
Integration test pass rate	Integration test suite pass ratio (passed / total)	74 / 74 (100%)
RAG topic classification (no API key)	Quick evaluation optimizer pass rate	$\geq 15/34$ (degraded mode)
RAG judge scores (with API key)	Faithfulness and relevance from automated judge	Documented in eval report
Security: re-fresh replay	Reuse of rotated token triggers family revocation	Verified by auth test suite
Security: avatar CORP	Cross-origin resource policy header present on avatar responses	Verified by avatars test

2.4 Selected Methodology

2.4.1 Agile Scrum Framework

The project was managed using the **Agile Scrum** methodology, chosen for its iterative structure, its emphasis on working deliverables at the end of each sprint, and its adaptability to evolving requirements — all well-suited to a complex full-stack product developed by a single engineer over a bounded internship period.

2.4.1.1 Scrum Roles

- **Product Owner:** mohamed aziz rezgui (HyprCraft Solutions), responsible for validating feature priorities and acceptance criteria.

- **Scrum Master / Developer:** Adam Farhat, responsible for sprint planning, execution, daily self-review, and retrospective.
- **Academic Advisor:** Jacem Mhenni (ESPRIM), providing methodological and scientific guidance at sprint boundaries.

2.4.1.2 Scrum Artefacts

- **Product Backlog:** the complete list of functional and non-functional requirements (FR-01 to FR-12, NFR-01 to NFR-06) prioritised by business value and technical dependency.
- **Sprint Backlog:** the subset of backlog items committed to in each sprint, decomposed into implementation tasks.
- **Increment:** a working, tested vertical slice of the platform delivered at the end of each sprint.
- **Definition of Done:** a requirement is considered done when (a) the feature is implemented, (b) at least one integration test covers the happy path and one error path, and (c) the corresponding capability is manually validated against its acceptance criteria.

2.4.1.3 Sprint Structure

Each sprint followed a fixed rhythm:

1. **Sprint Planning:** review backlog, select items, decompose into tasks, estimate effort.
2. **Development:** implement features, write integration tests, perform code review against security and style guidelines.
3. **Sprint Review:** demonstrate the working increment; validate against acceptance criteria.
4. **Sprint Retrospective:** identify what worked, what blocked, and what to improve in the next sprint.

2.4.2 Sprint Plan

The project was structured across **five sprints**, each delivering a coherent and independently testable platform capability. Detailed interaction design for each sprint appears in Section ?? of Chapter ??.

Table 2.5 – Agile Scrum sprint plan

Spr.	Theme	Deliverables	Req.	Des.
S1	Auth., users, RBAC	JWT login/logout/refresh (HttpOnly cookies), refresh-token family revocation, password policy, session cap, per-user restrictions, profile/avatar, RBAC middleware	FR-01, FR-02, FR-12 NFR-02, NFR-03	??
S2	Document library & ingest	Multi-format upload (PDF, Office, HTML, XML, ZIP), BullMQ worker (extract→chunk→embed→persist), versioning, tags, favorites, recents, archive, visibility, audit log	FR-03, FR-04, FR-05	??
S3	Hybrid RAG & Ask	Query optimiser, hybrid retrieval (pgvector + BM25 + RRF), LLM reranking, neighbour expansion, SSE answers with citations, confidence scoring, critique, claim verification, follow-ups, feedback loop, conversations	FR-06, FR-07, FR-08 NFR-06	??
S4	Admin, manager, notifications	Admin CRUD (users, departments, merge, bulk ops, CSV), dept-access management, KPI dashboard, activity/audit exports, manager view, auto/manual notifications	FR-09, FR-10, FR-11	??
S5	Tests, eval & deploy	74 Vitest/Supertest integration tests, RAG eval CLI (automated judge), Docker Compose prod config, migration pipeline, 54 UML diagrams, README	NFR-01, NFR-04, NFR-05 NFR-06	??

The **Des.** column references the sprint interaction-design subsections of Section ?? (Chapter ??).

2.4.2.1 Sprint Timeline

Figure ?? illustrates the sprint sequence defined in Table ?? and the iterative delivery of working increments across the project duration. Each sprint builds on the previous one: authentication and access control (S1), document corpus (S2), hybrid RAG (S3), governance (S4), then operational hardening and validation (S5).

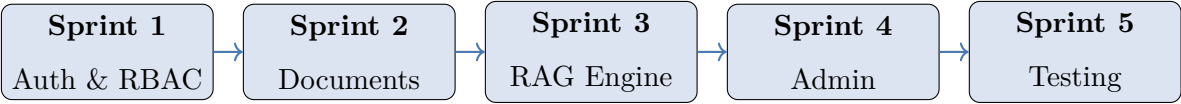


Figure 2.2 – Scrum sprint timeline — iterative delivery across five sprints

2.4.2.2 Sprint Backlogs

Tables ??–?? detail each sprint backlog at user-story level. The stories refine the scope presented in Table ?? and provide planning-level traceability between implementation tasks, acceptance criteria, and effort estimation. Interaction flows for every story are documented in Section ??.

Sprint 1 — Authentication and access control.**Table 2.6** – Sprint 1 backlog: authentication and access control

ID	User Story	Acceptance Criteria	SP	Pri.
US-S1-01	As a guest, I want to sign in securely so that I can access authorised platform features.	Valid credentials issue an access token and secure refresh cookie; invalid credentials are rejected; login rate limiting is enforced.	5	High
US-S1-02	As an authenticated user, I want seamless session continuity so that refresh operations are secure and race-condition free.	Session refresh rotates the token family atomically; replay of a rotated token revokes the family; parallel tab requests share a single refresh operation.	8	High
US-S1-03	As an authenticated user, I want to sign out so that active refresh sessions are revoked.	Logout revokes the current refresh session; subsequent refresh attempts fail.	2	Med.
US-S1-04	As a user, I want to view and update my profile, including avatar, so that my account information remains accurate.	Profile updates persist; avatar upload succeeds; avatars are served with cross-origin headers for display in the web client.	5	High
US-S1-05	As an administrator, I want role-based and per-user access restrictions so that capabilities can be controlled precisely.	Role boundaries enforce administrator, manager, and employee capabilities; restriction flags block protected features and redirect to a restricted notice page.	8	High
US-S1-06	As a security owner, I want baseline HTTP protections so that authentication endpoints resist abuse.	Security headers are active globally; login and Ask endpoints are rate-limited according to configured thresholds.	3	Med.

Sprint 2 — Document library and ingest.**Table 2.7** – Sprint 2 backlog: document library and ingest

ID	User Story	Acceptance Criteria	SP	Pri.
US-S2-01	As an employee, I want to upload documents in common formats so that they enter the shared library.	Upload validates file type and size, stores the file, creates a version record, and queues ingestion.	8	High
US-S2-02	As the system, I want to ingest uploaded files asynchronously so that text is searchable and embeddable.	Background worker extracts text, chunks content, computes embeddings, and persists passages; version reaches ready state on completion.	13	High
US-S2-03	As a security owner, I want visibility rules enforced so that unauthorised users cannot discover restricted documents.	Organisation-wide, department, and private visibility levels are enforced; unauthorised access returns 404 rather than revealing existence.	8	High
US-S2-04	As an employee, I want to browse, search, and open documents so that I can find relevant files quickly.	Document listing supports filters and pagination; detail view returns metadata and active version; download delivers file content.	5	High
US-S2-05	As an employee, I want favorites and recents so that I can return to frequently used documents.	Favourite toggle and recent-view tracking persist per user; library home surfaces both lists.	3	Med.
US-S2-06	As a document owner, I want versioning and re-ingestion so that updates and embedding upgrades are supported.	New version upload creates a version record and queues ingestion; reprocess triggers re-ingestion of an existing version.	8	Med.
US-S2-07	As a manager, I want to update metadata and retain an audit	26 Title, tags, and visibility updates persist within scope;	5	Med.

Sprint 3 — Search, RAG and conversations.**Table 2.8** – Sprint 3 backlog: search, RAG and conversations

ID	User Story	Acceptance Criteria	SP	Pri.
US-S3-01	As an employee, I want semantic search over the corpus so that I can find relevant passages by meaning.	Query is embedded and ranked against department-scoped passages; results include snippet previews.	8	High
US-S3-02	As an employee, I want cited answers streamed in real time so that I receive grounded responses to natural-language questions.	Hybrid retrieval and reranking produce an answer stream with source citations delivered before answer tokens and a completion signal.	13	High
US-S3-03	As an employee, I want persistent conversation threads so that I can review prior questions and answers.	Conversations support create, list, rename, and delete; messages store sources, confidence, and citations.	8	High
US-S3-04	As an employee, I want to rate assistant answers so that poor responses can be identified and improved.	Thumbs up/down feedback is stored; administrator statistics aggregate sentiment and volume.	5	Med.
US-S3-05	As a data scientist, I want a reproducible RAG evaluation harness so that retrieval quality can be measured over time.	Quick and full evaluation modes produce structured quality reports without disrupting continuous integration.	8	Med.

Sprint 4 — Governance and notifications.**Table 2.9** – Sprint 4 backlog: governance and notifications

ID	User Story	Acceptance Criteria	SP	Pri.
US-S4-01	As an employee, I want automatic notifications on document events so that I stay informed of library changes.	Document create, update, and delete events generate department-scoped inbox entries.	5	High
US-S4-02	As a manager or admin, I want to send manual announcements so that I can communicate with targeted teams.	Manual send validates audience scope; optional attachments are downloadable by recipients.	5	High
US-S4-03	As an employee, I want a notification inbox so that I can read and dismiss messages.	Inbox supports listing, unread count, mark-read, and mark-all-read; web client displays an unread badge.	5	Med.
US-S4-04	As a manager, I want a department dashboard so that I can oversee teams and documents I manage.	Dashboard returns only departments within the manager’s scope; member lists and department documents are correctly filtered.	8	High
US-S4-05	As an administrator, I want full user and department administration so that the platform reflects organisational structure.	User and department lifecycle operations are supported; last-administrator safety is enforced; bulk import sanitises spreadsheet formulas.	13	High
US-S4-06	As an administrator, I want bulk access management and operational exports so that governance tasks scale efficiently.	Bulk department-access updates, KPI statistics, activity logs, document audit trails, and exports are available to administrators.	8	Med.

Sprint 5 — Operations, testing and deployment.**Table 2.10** – Sprint 5 backlog: operations, testing and deployment

ID	User Story	Acceptance Criteria	SP	Pri.
US-S5-01	As an operator, I want a health endpoint so that I can verify database and queue connectivity.	Health check returns success when database and cache are reachable; otherwise returns service-unavailable with per-component detail.	3	High
US-S5-02	As a developer, I want automated integration tests so that regressions are caught before release.	Integration suite runs 74 cases across ten domains on live database and cache; target is 100% pass rate.	13	High
US-S5-03	As an operator, I want reproducible containerised deployment so that the platform can be started consistently.	Container orchestration starts database, cache, API, and web services in a healthy state; migrations run automatically.	8	High
US-S5-04	As a data scientist, I want automated RAG judge evaluation so that answer quality can be tracked with an API key.	Full evaluation produces faithfulness and relevance scores; quick mode runs without an external API key.	8	Med.
US-S5-05	As a maintainer, I want complete design documentation so that the system is defensible and maintainable.	Fifty-four UML diagrams are exported; architecture guide is updated; sequence flows are catalogued in Chapter ??.	5	Med.

2.4.3 Development and Modelling Tools

Table 2.11 – Development and modelling tools

Tool	Usage in project
Prisma Schema DSL	Domain modelling and 29 sequential migrations
PlantUML	54 UML diagrams (3 structural, 5 sprint use-case, 5 sprint class, 41 sequence)
Vitest + Supertest	Integration testing across all API domains
Docker Compose	Reproducible dev and production environment
GitHub Actions	CI pipeline (lint, audit, typecheck, integration tests)
Zod	Runtime request validation for all API inputs

2.5 Chapter Conclusion

This chapter established a verifiable specification for the Knowledge Platform by defining functional and non-functional requirements, measurable acceptance criteria, and key performance indicators. It also formalised the delivery approach through a five-sprint Scrum plan with detailed sprint backlogs, linking each increment to design, implementation, and validation objectives in the chapters that follow. Chapter ?? presents the resulting system design, including sprint-aligned interaction diagrams in Section ??.

CHAPTER 3

Design and Architecture

This chapter translates the requirements and sprint plan of Chapter ?? into a concrete system design. The presentation follows two complementary views:

- **Static design** (Sections ??–??) — platform architecture, subsystem responsibilities, persisted domain model, and technology justification;
- **Dynamic design** (Section ??) — sprint-scoped use-case and class models, followed by interaction sequence diagrams for every major API and UX flow.

Implementation details and validation evidence are developed in Chapters ?? and ??.

3.1 Overall Solution Architecture

The proposed Knowledge Platform adopts a three-tier logical architecture composed of a presentation layer, an application layer, and a data and artificial intelligence layer. This architectural approach promotes separation of concerns by isolating the user interface, business logic, and persistence mechanisms into independent yet collaborative components. Such an organization improves maintainability, scalability, security, and facilitates the future evolution of the platform. Figure ?? presents the overall architecture and the interactions between the main system components.

At the presentation layer, users interact with the platform through a responsive web application that provides access to document management, semantic search, conversational AI, notifications, and administrative functionalities. Authentication relies on a secure session management strategy based on short-lived access tokens stored in memory

and refresh tokens maintained as HttpOnly cookies, reducing the exposure of sensitive authentication credentials.

The application layer is implemented as a RESTful API responsible for coordinating all business operations. It exposes dedicated services for authentication, document management, semantic search, conversational AI, notifications, administration, and manager-specific operations. Beyond processing client requests, this layer enforces authorization policies, validates user permissions, orchestrates document processing workflows, and coordinates interactions with the underlying storage and artificial intelligence services.

The data and AI layer manages document persistence, metadata, vector embeddings, user information, notifications, and conversational history. Following each document upload or version update, an asynchronous ingestion pipeline extracts textual content, partitions it into semantic chunks, generates vector embeddings, and indexes the resulting representations to support efficient retrieval. This background processing is performed independently of user requests, thereby preserving application responsiveness while ensuring that newly uploaded documents become searchable once processing is complete.

Together, these three layers establish a modular and extensible architecture capable of supporting enterprise-scale document management and Retrieval-Augmented Generation (RAG), while maintaining a clear separation between presentation, application logic, and data processing responsibilities.

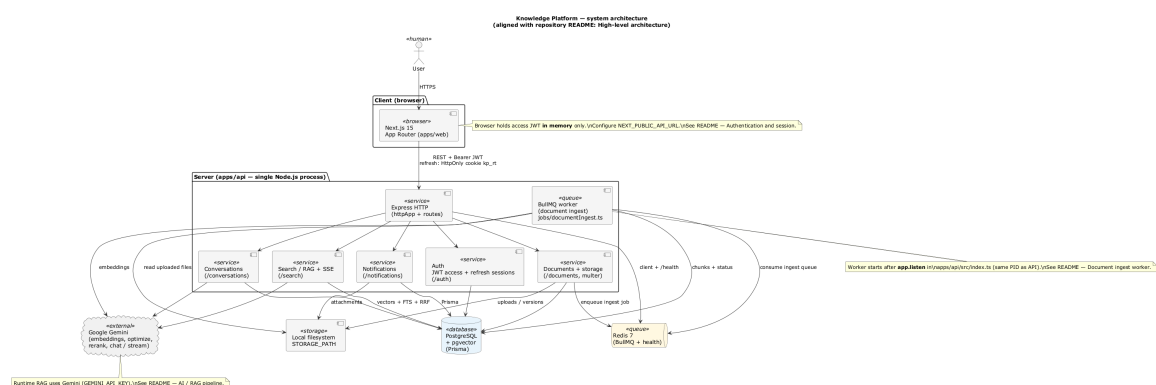


Figure 3.1 – Overall architecture of the Knowledge Platform (PlantUML source: `platform-architecture.puml`)

3.2 Logical Subsystem Decomposition

Table 3.1 – Subsystem responsibilities

Subsystem	Responsibility	Key components
Authentication	Login, refresh families, restrictions, password reset	Authentication API Web session client
Documents	CRUD, versions, visibility, audit, download	Document management service File storage layer
Ingest worker	Extract, chunk, embed, persist chunks	Background ingest worker Indexing pipeline
Search / RAG	Hybrid retrieval, SSE answers, cache	Hybrid search service AI answer streaming
Conversations	Threads, messages, feedback	Conversation management service
Notifications	Auto events and manual announcements	Notification service
Admin / Manager	Governance dashboards	Administration API Manager workspace API

3.3 Domain Data Model

The persisted domain model centres on users and organisational departments, documents and their versioned content, searchable text chunks with 768-dimensional vector embeddings, conversational threads with answer feedback, and notification records. Users may hold explicit access to several departments, which supports both role-based governance and department-scoped document visibility. Figure ?? presents the principal persisted entities and their relationships across all sprints; sprint-scoped excerpts appear in Section ??.

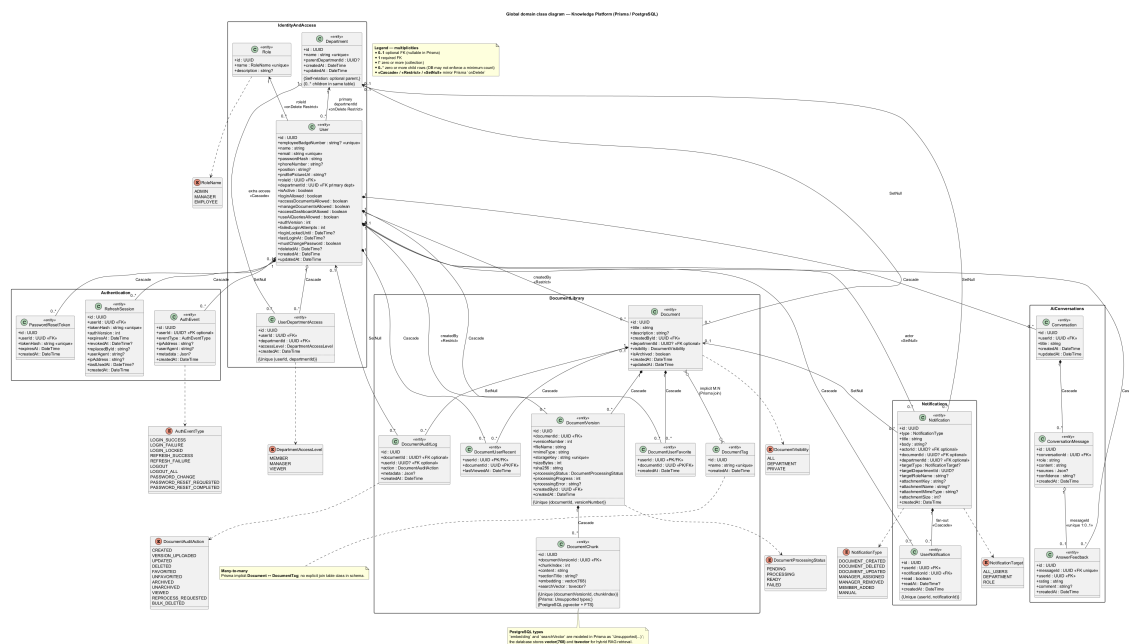


Figure 3.2 – Global domain class diagram (persisted entities and main relationships)

3.4 Technology Choice Justification

Table ?? scores architectural options (1 = weak, 3 = strong). The selected stack favours a TypeScript monorepo with Express and Next.js for streaming RAG, operational simplicity, and alignment with project skills.

Table 3.2 – Decision-support matrix for platform architecture (scores: 1 = weak, 3 = strong)

Criterion	Monorepo	Django	Microservices	Rationale
Time to market	3	2	1	Shared TypeScript types; one repository
SSE streaming fit	3	2	2	Native Express streaming for RAG
Team skill alignment	3	2	1	Matches React + Node stack
Operational complexity	3	3	1	Single API process + worker

PostgreSQL with pgvector vs. dedicated vector database: storing relational metadata and vector embeddings in the same database simplifies hybrid retrieval, transactional consistency, and department-scoped filtering within a single query layer [?].

Managed Gemini APIs vs. self-hosted open-weight models: hosted embedding and chat services reduce operational overhead while delivering strong multilingual representations at the dimensionality required by this deployment [?].

In-process background worker vs. separate worker service: running the ingest consumer alongside the API reduces deployment complexity for a prototype-scale platform, at the cost of tighter coupling when scaling out horizontally.

Table ?? links representative requirements to the chosen design elements and validation artefacts (Chapter ??).

Table 3.3 – Requirements-to-design traceability (excerpt)

Req.	Design choice	Implemented element	Validation
FR-01	Short-lived tokens with re-fresh rotation	Authentication API and web session client	Integration tests for sign-in and session renewal
FR-02	Profile images with CORP isolation	Public avatar delivery endpoint	Integration tests for avatar access
FR-03	Asynchronous document ingest	Background worker with Redis-backed job queue	Integration tests for upload and processing
FR-07	Hybrid retrieval with streamed answers	Search and AI completion services	Integration tests for semantic search and Ask
FR-10	Role-based administration	Administration service layer	Integration tests for governance endpoints
NFR-02	HTTP hardening and rate limiting	API security middleware stack	Manual review and integration tests

3.5 Dynamic Interaction Design by Sprint

Sections ??–?? describe **what** the platform is made of. This section describes **how** people, services, and background processes interact during real use. For each development sprint in Table ?? of Chapter ??, the design is presented in three steps:

1. **Sprint scope** — the capabilities delivered in that iteration;
2. **Scoped structural views** — the sprint use-case model and domain class model;
3. **Interaction sequence diagrams** — step-by-step flows between the user interface, application services, background workers, and external systems.

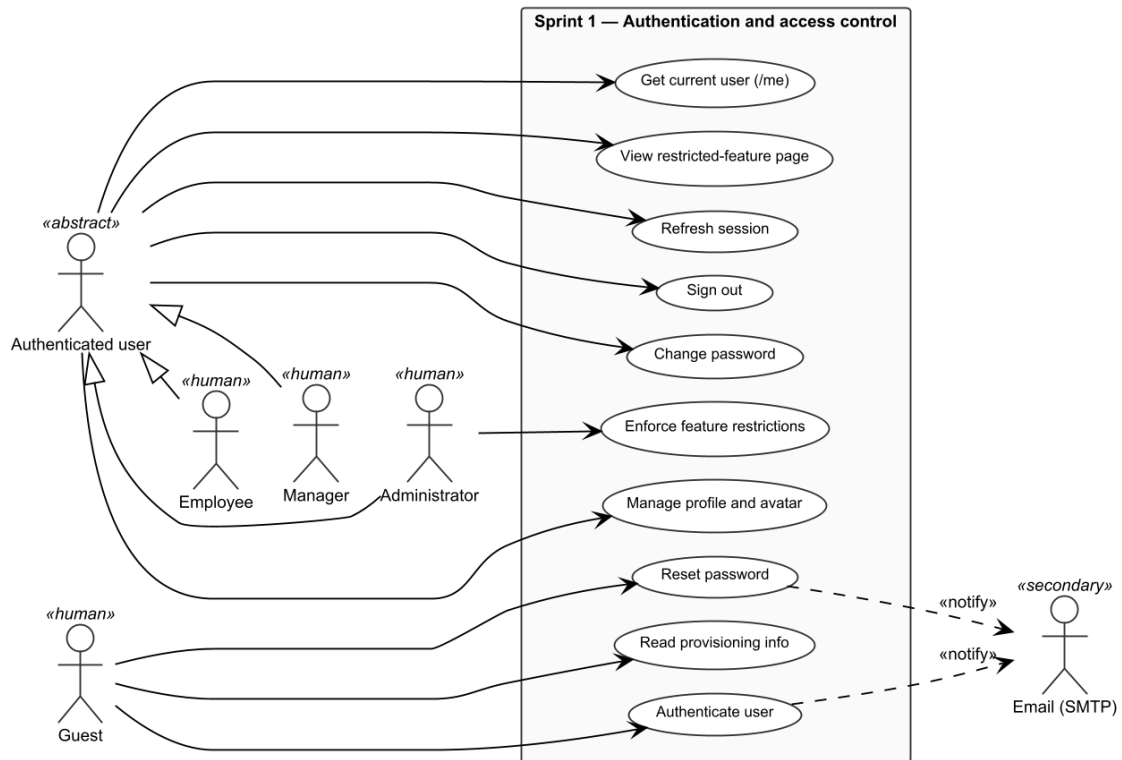
The full diagram catalog is presented in Section ??; implementation details appear in Chapter ?? and validation in Chapter ??.

3.5.1 Sprint 1: Authentication and access control

Sprint 1 defines how people enter and move through the platform. It covers sign-in and sign-out, password recovery, profile management, and the rules that limit access when an account or feature is restricted.

3.5.1.1 Scoped structural views

Use-case model.

Sprint 1 use-case diagram — Authentication and access control**Figure 3.3** – Sprint 1 use-case diagram — Authentication and access control

Domain class model.

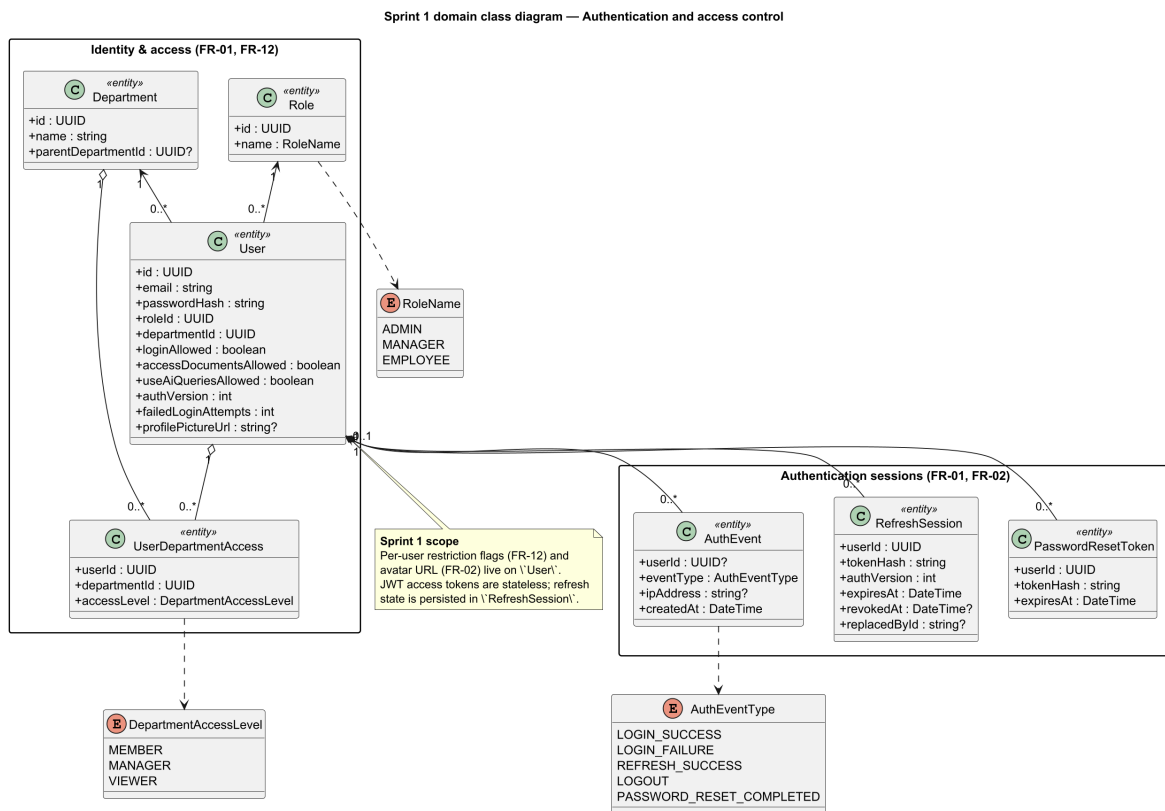


Figure 3.4 – Sprint 1 domain class diagram — Authentication and access control

3.5.1.2 Interaction sequence diagrams

Signing in and managing the session

User sign-in.

A user enters email and password; the platform verifies the credentials, applies account lockout after repeated failures, and opens a secure session with a short-lived access token and a protected refresh cookie.

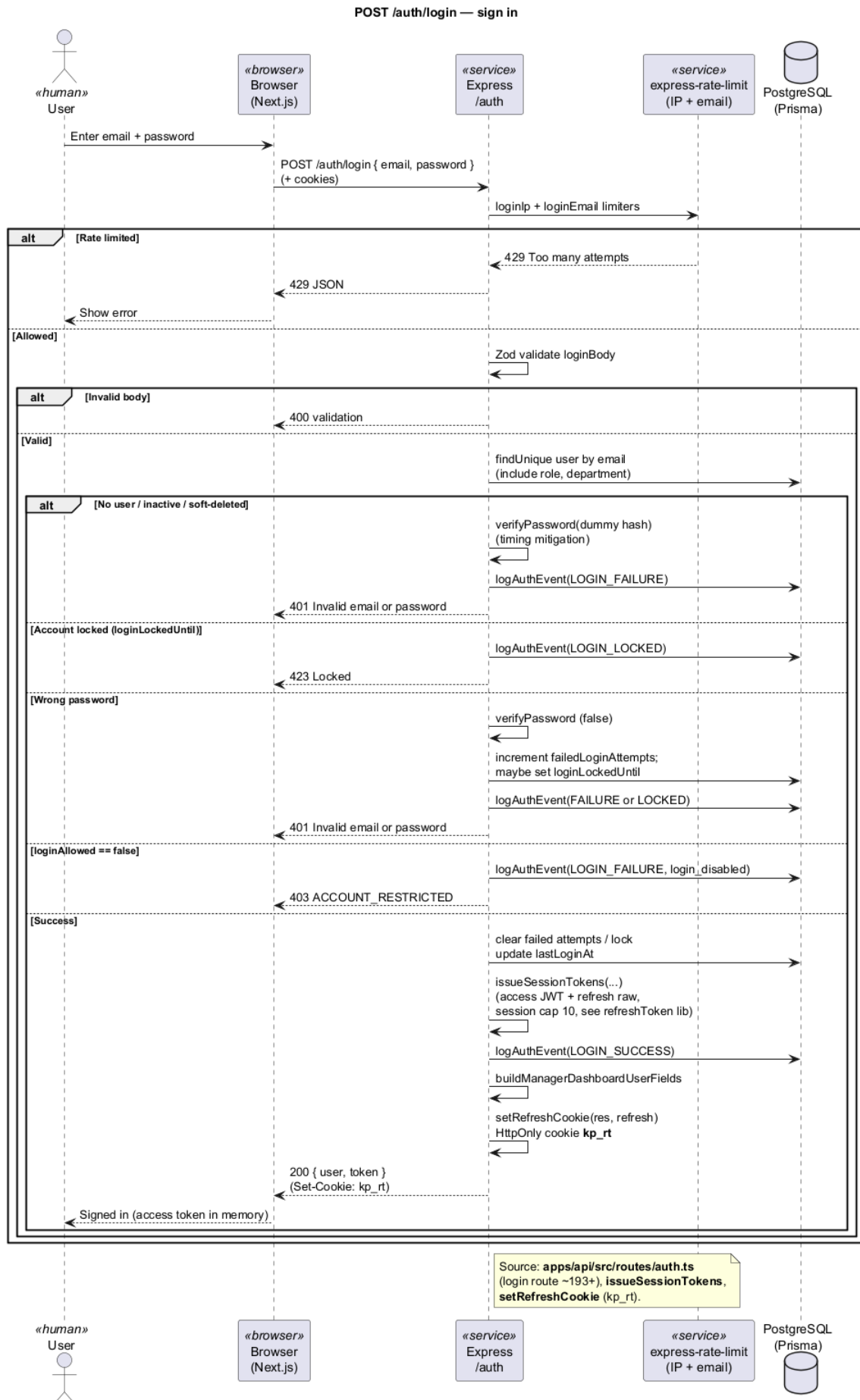
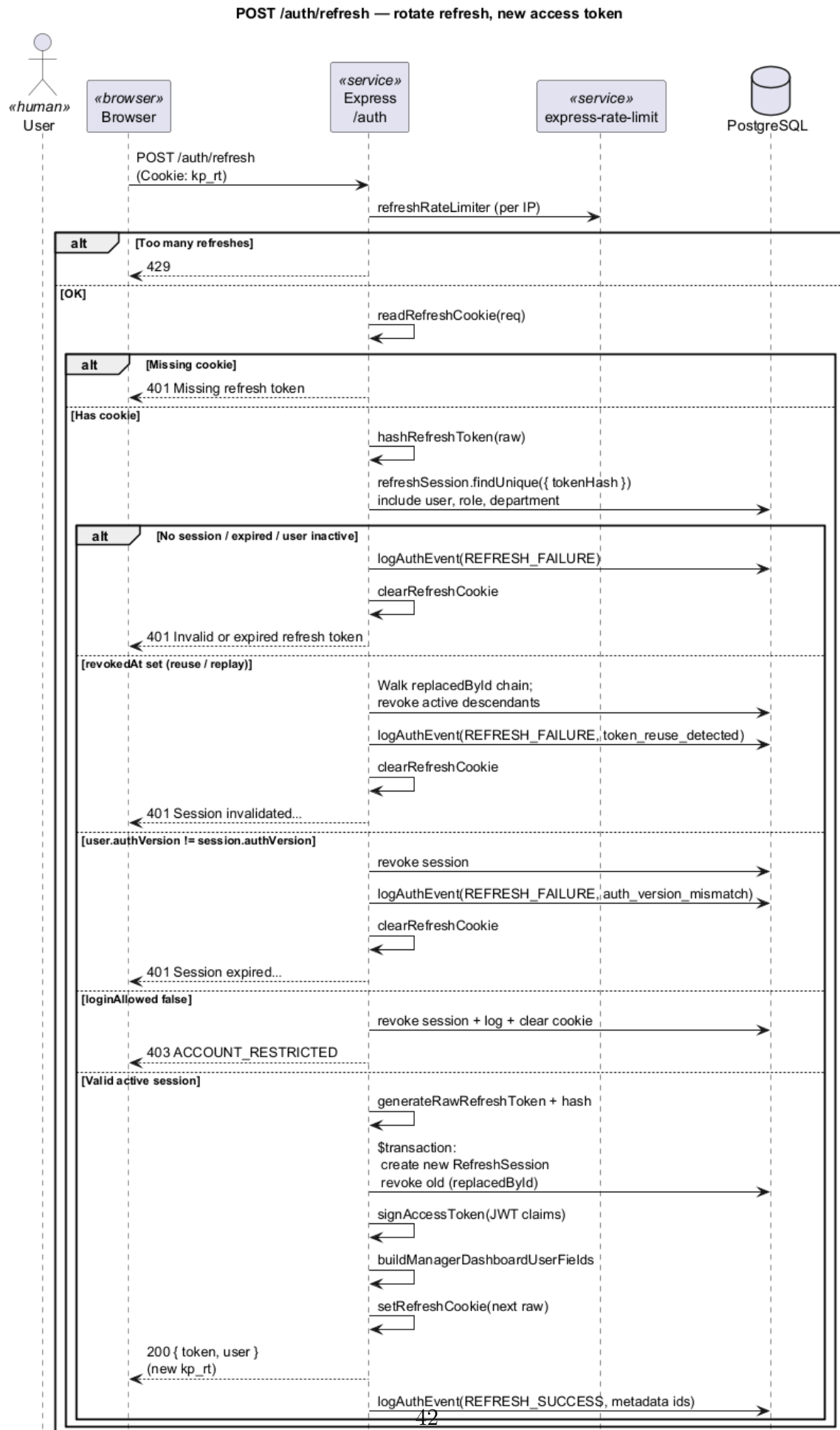


Figure 3.5 – User sign-in (sign-in flow)

Silent session renewal.

When the access token expires, the web application renews the session in the background so the user can continue working without signing in again.



User sign-out.

Signing out ends the active session on the server and removes the refresh cookie from the browser, so the account cannot be reused from that device without a new sign-in.

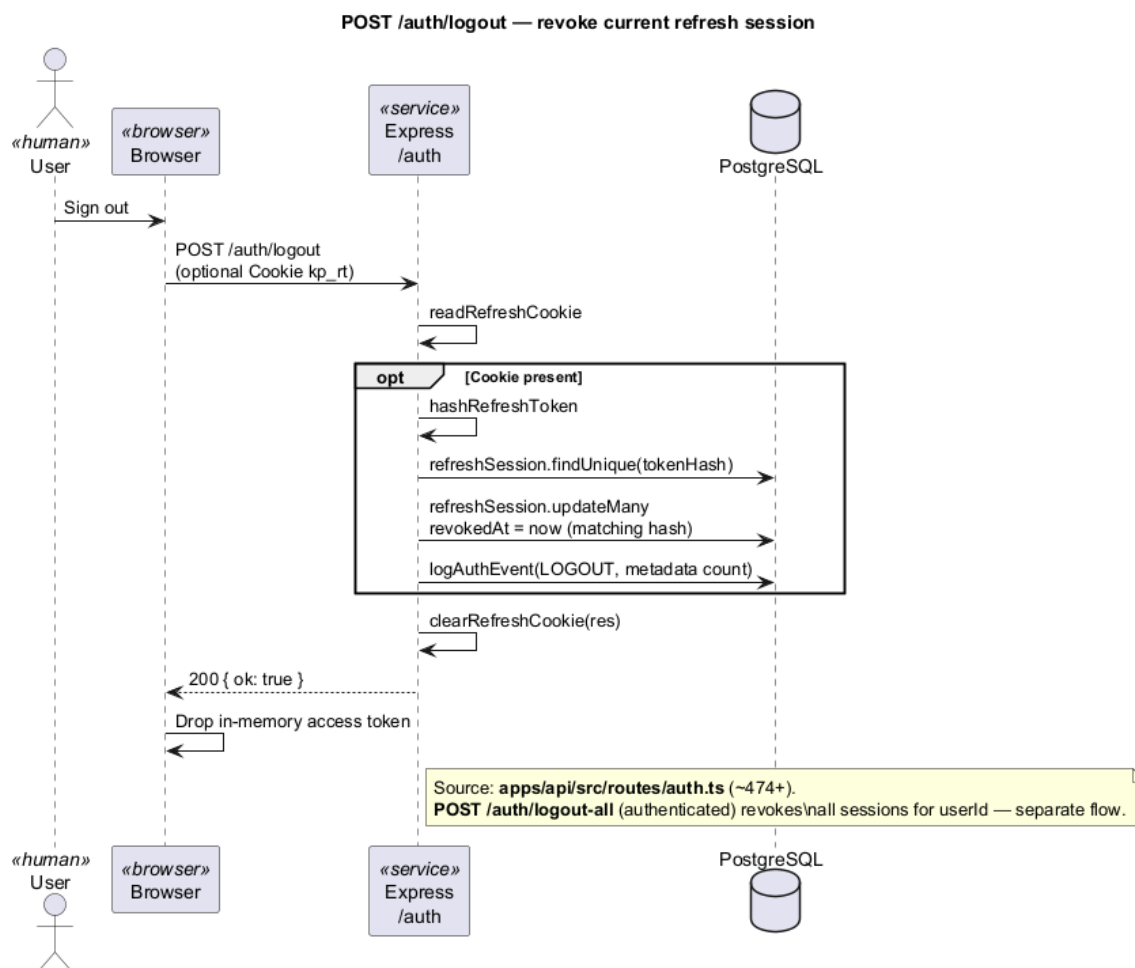


Figure 3.7 – User sign-out (sign-out flow)

Forgotten password recovery.

A user who forgot their password requests a reset link by email, then sets a new password through a time-limited link. Existing sessions are closed once the password changes.

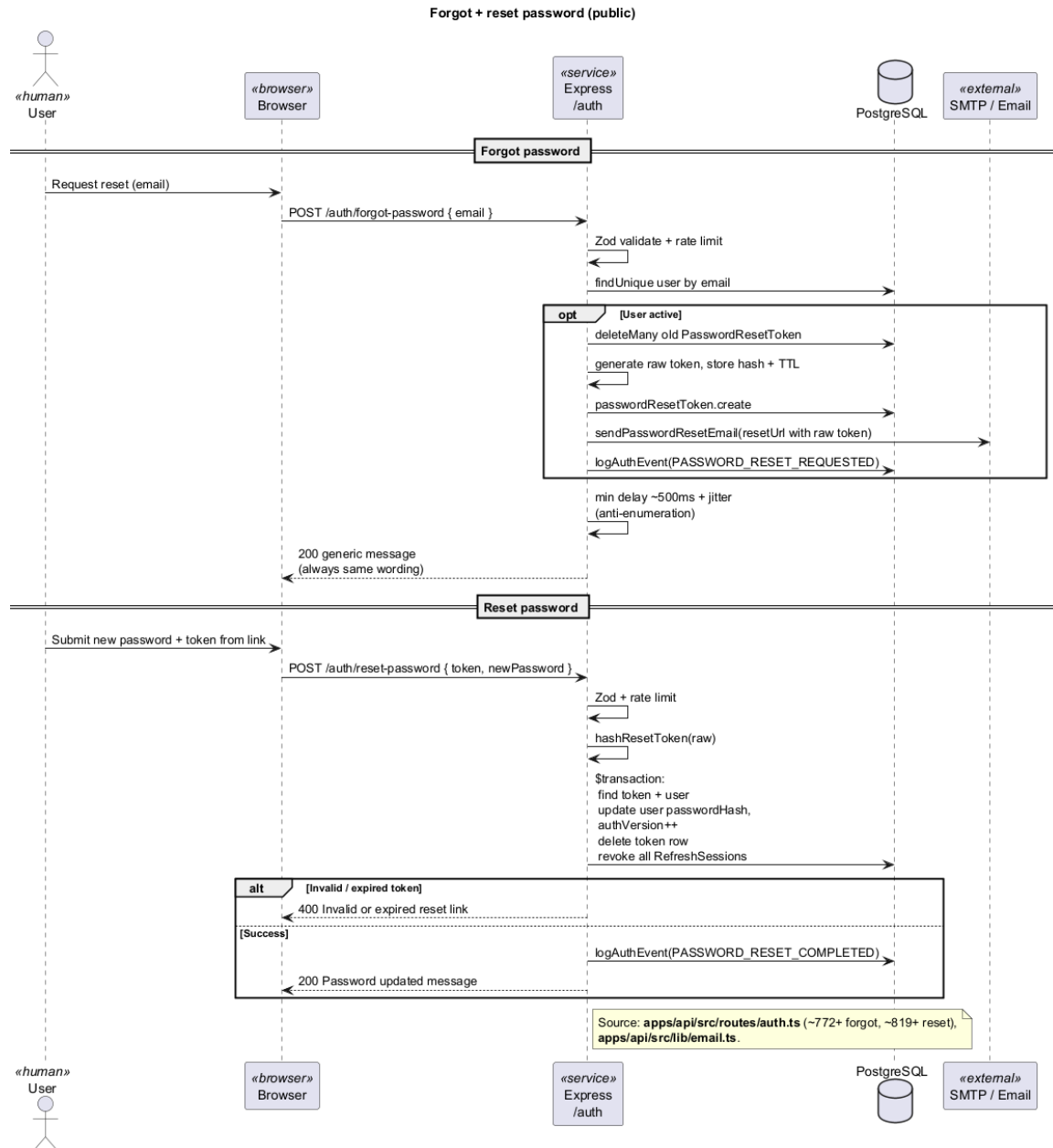


Figure 3.8 – Forgotten password recovery (password reset flow)

Changing password while signed in.

A signed-in user can change their password after confirming the current one. Successful change closes other active sessions for safety.

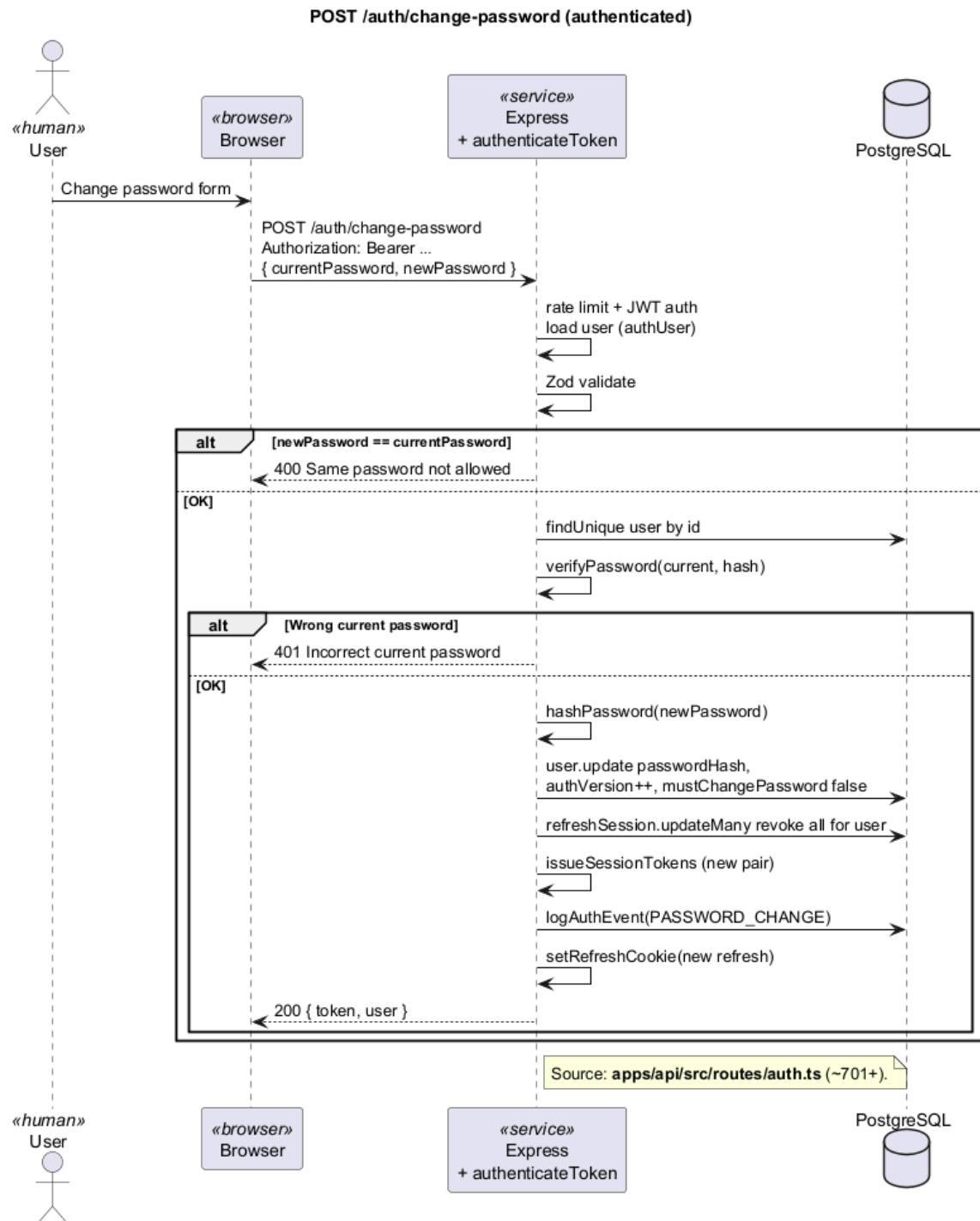


Figure 3.9 – Changing password while signed in (change-password flow)

Profile, permissions and session context

Updating the user profile.

Users can update personal details such as display name and contact information. Some changes, such as email updates, require the session to be renewed for security.

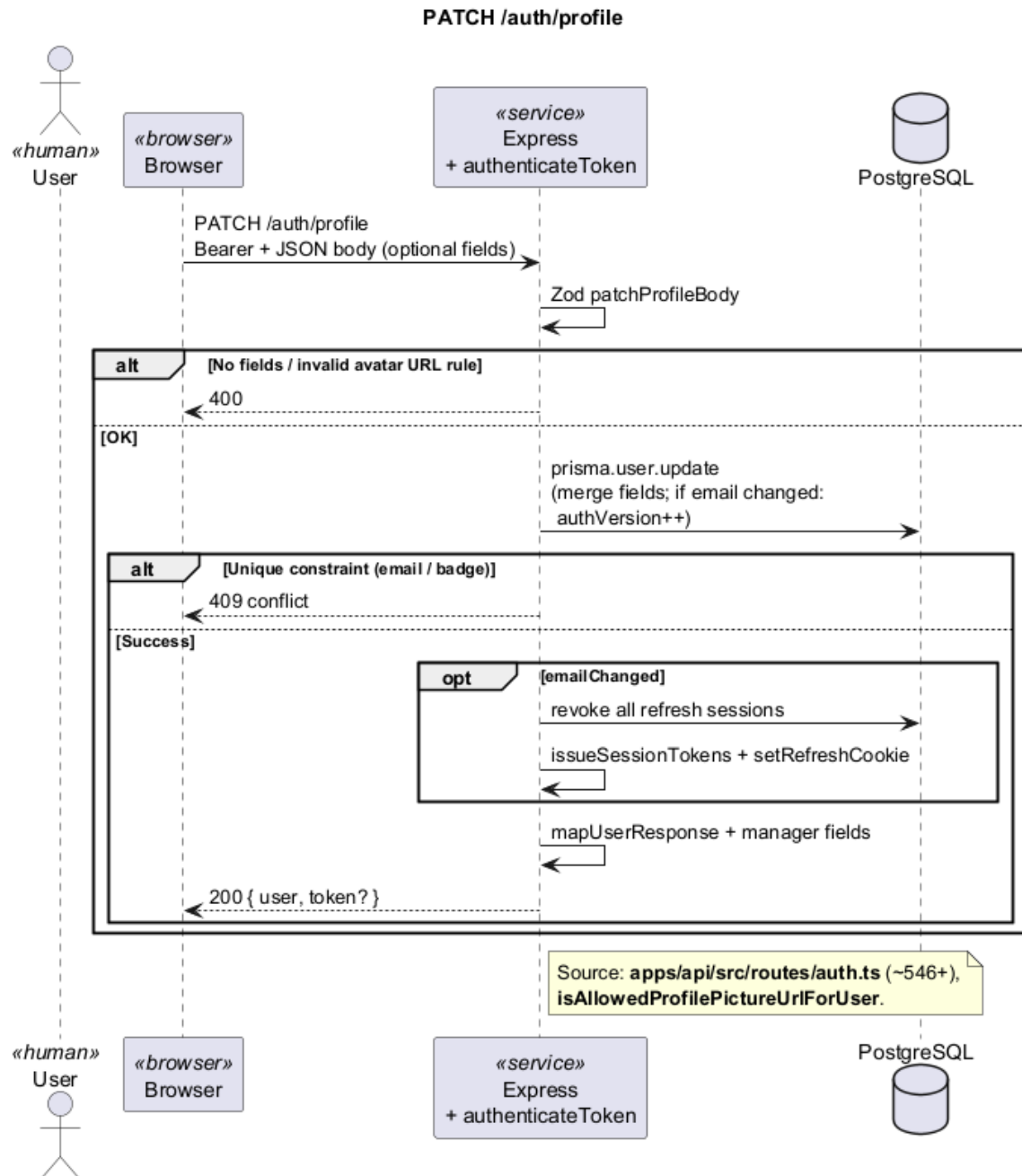


Figure 3.10 – Updating the user profile (profile update flow)

Blocking access to a restricted feature.

Before sensitive actions run, the server checks whether the user is allowed to use the document library or AI-powered search. If not, the request is rejected with a clear permission error (see Figure ?? for the page shown to the user).

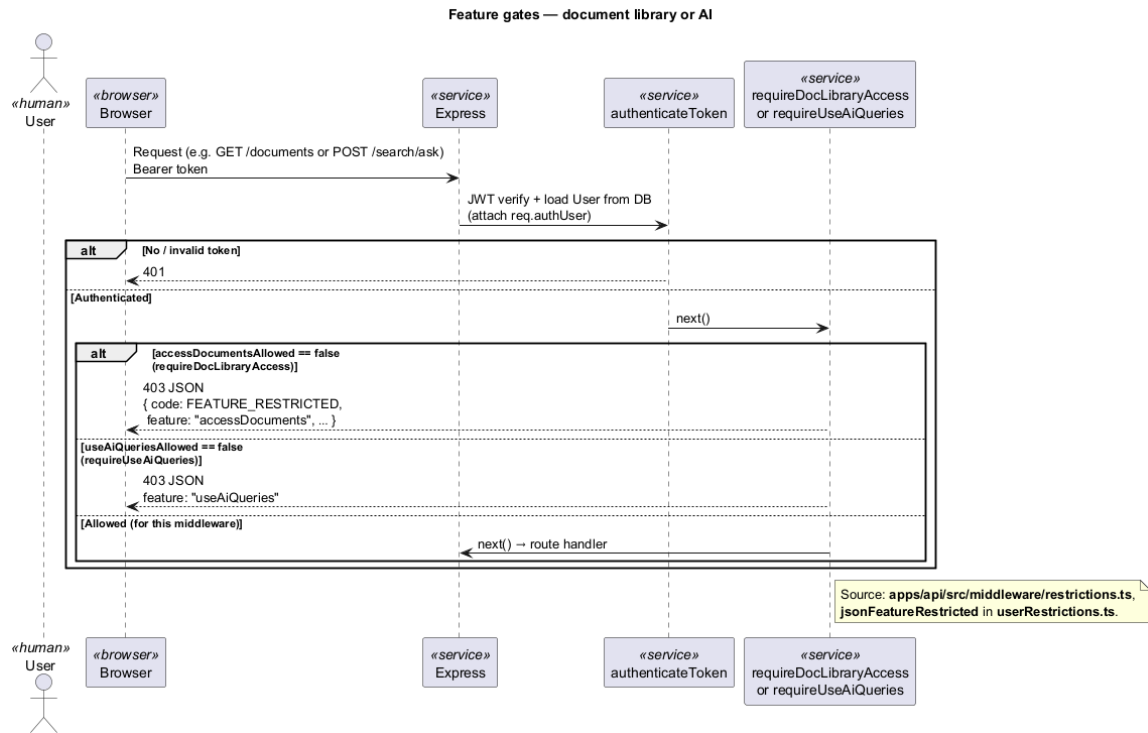


Figure 3.11 – Blocking access to a restricted feature (feature gate check)

Loading the signed-in user's context.

After authentication, the application loads the user's role, department memberships, and active restrictions so menus and pages can be adapted immediately.

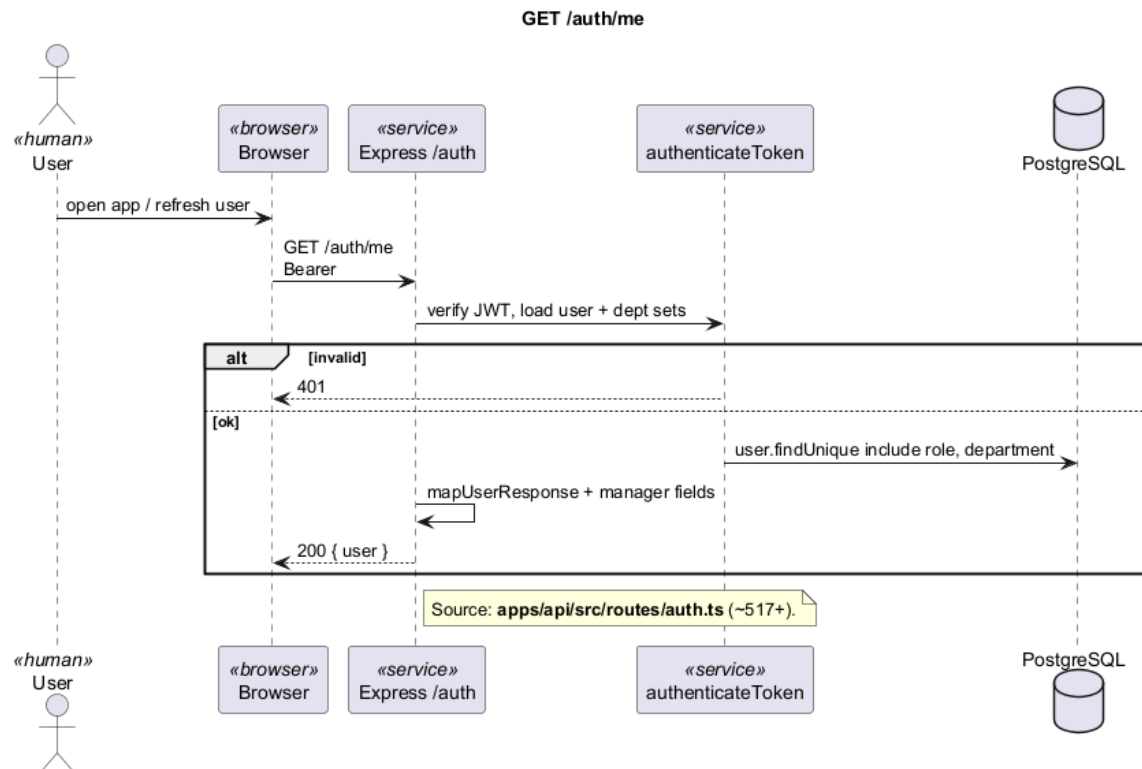


Figure 3.12 – Loading the signed-in user’s context (session context load)

Pages shown before or outside normal access

Registration information page.

Self-service account creation is not offered. The registration page explains that administrators create accounts, which keeps user provisioning under organisational control.

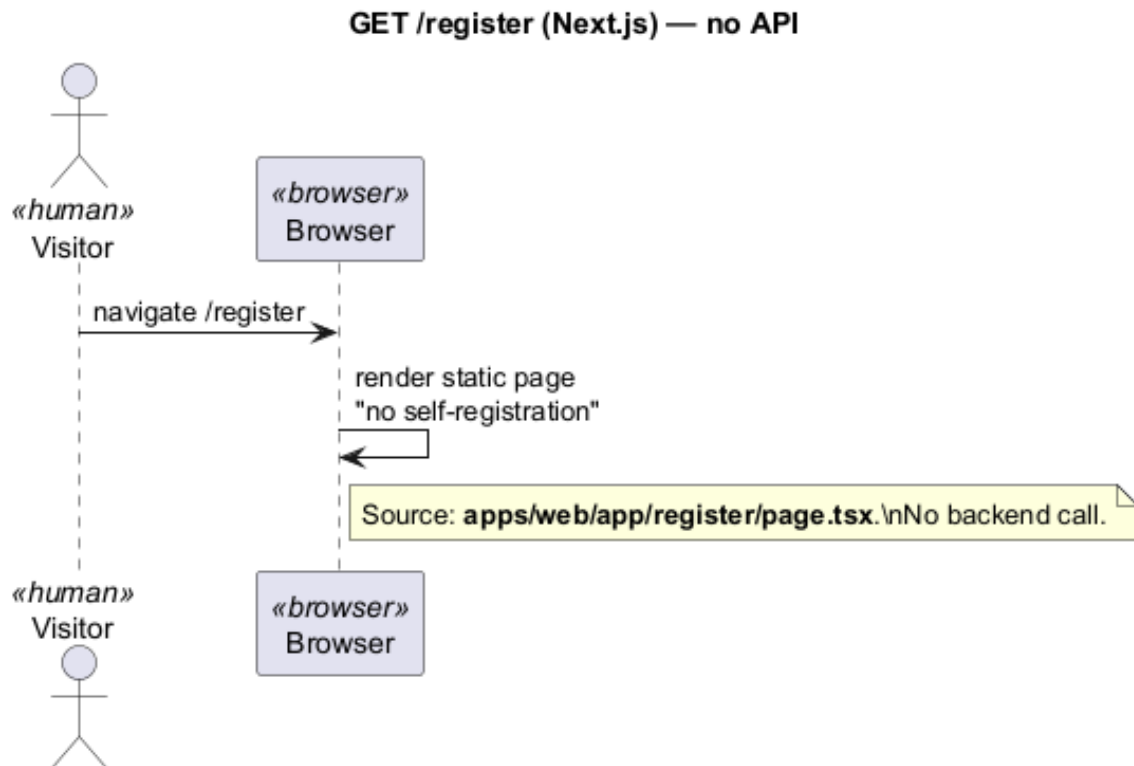


Figure 3.13 – Registration information page (static information page)

Restricted feature explanation page.

When a user reaches a feature they are not allowed to use, the application shows a dedicated explanation page with guidance instead of a generic error message. This page is shown in the browser; the permission check itself is illustrated in Figure ??.

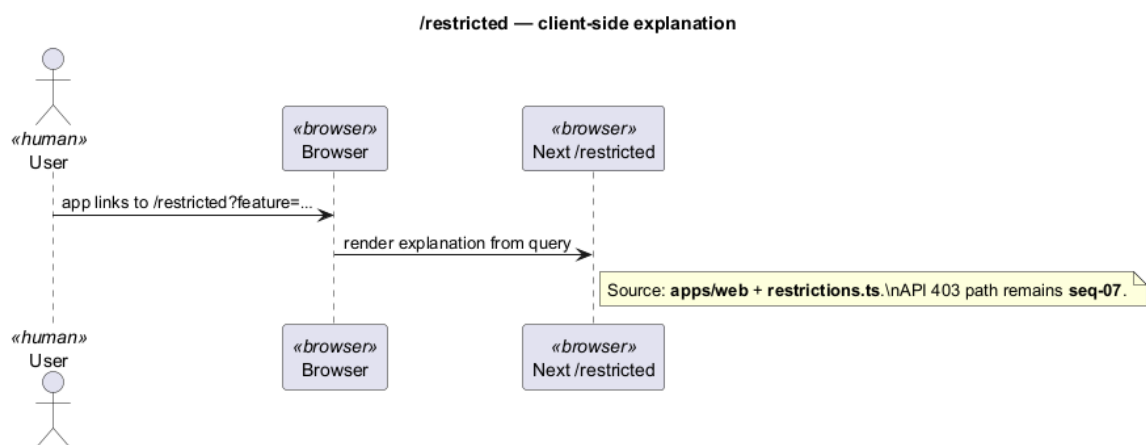


Figure 3.14 – Restricted feature explanation page (client explanation page)

3.5.2 Sprint 2: Document library and ingest

Sprint 2 introduces the document library: uploading files, preparing them for search, browsing and downloading content, and managing versions and metadata within department visibility rules.

3.5.2.1 Scoped structural views

Use-case model.

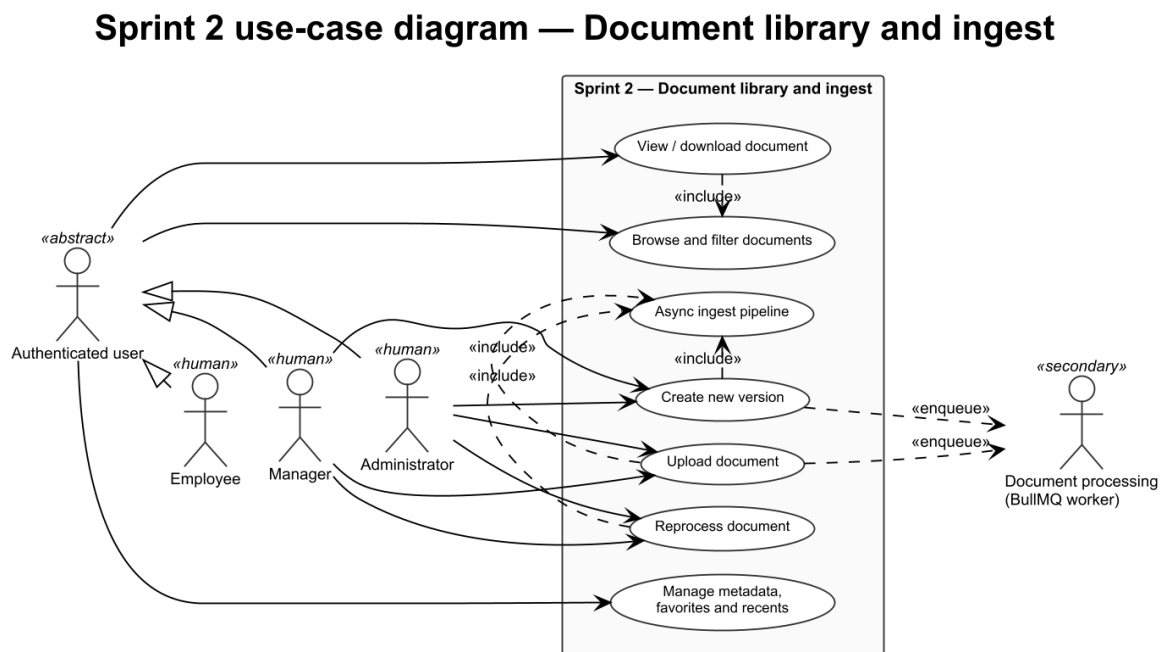


Figure 3.15 – Sprint 2 use-case diagram — Document library and ingest

Domain class model.

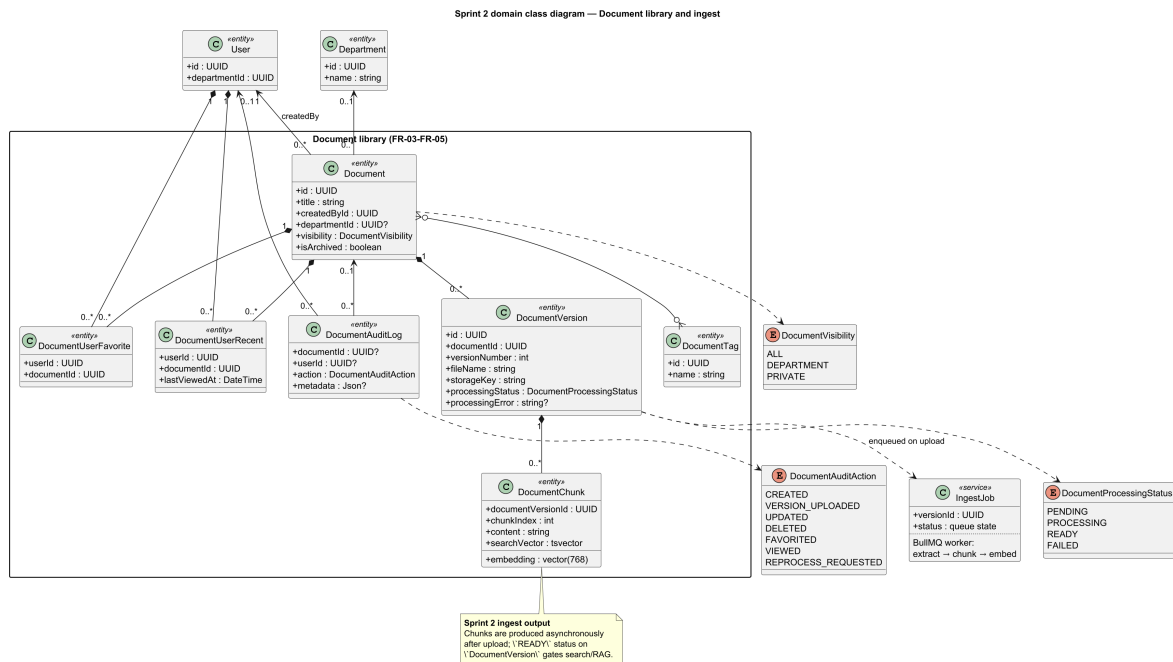


Figure 3.16 – Sprint 2 domain class diagram — Document library and ingest

3.5.2.2 Interaction sequence diagrams

Uploading and preparing documents

Uploading a document.

A user uploads a file with basic metadata. The platform stores the file, records the document in the library, and queues background processing so the user does not have to wait for indexing to finish.

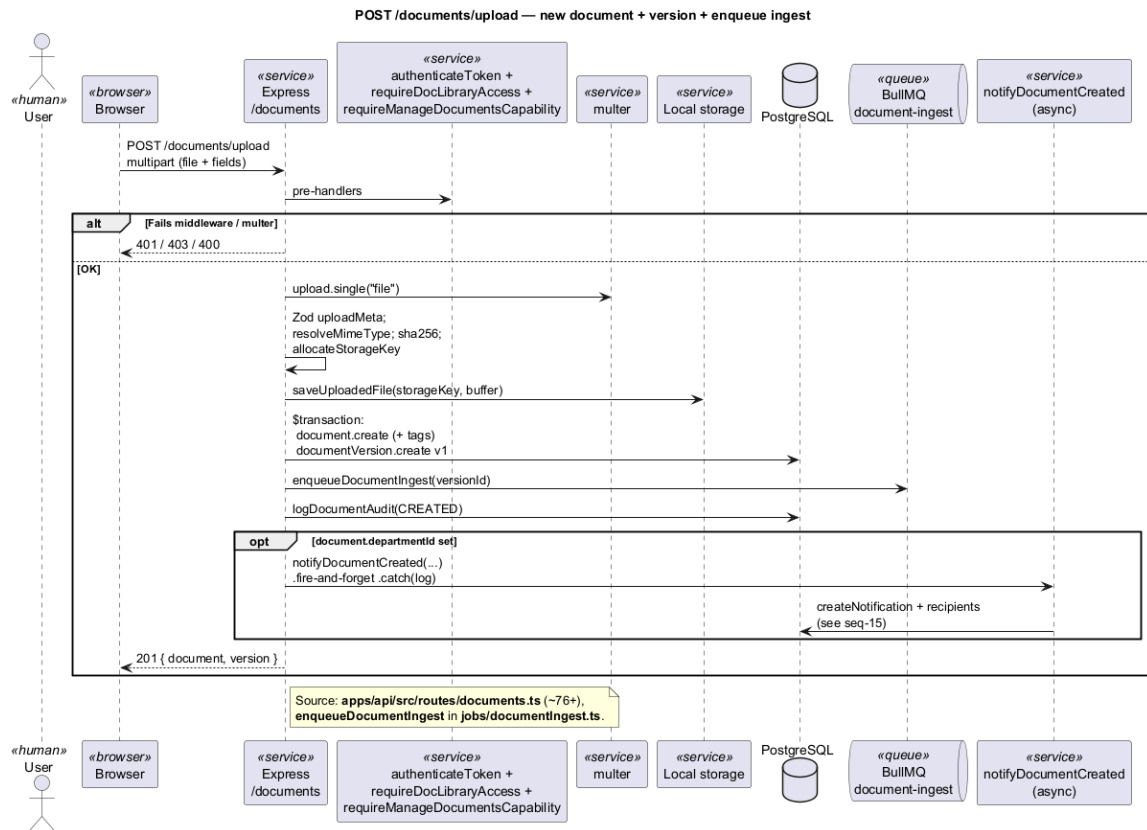


Figure 3.17 – Uploading a document (upload flow)

Background document processing.

After upload, a background worker extracts text, splits it into meaningful passages, creates searchable embeddings, and stores them so the document can later appear in semantic search and AI answers.

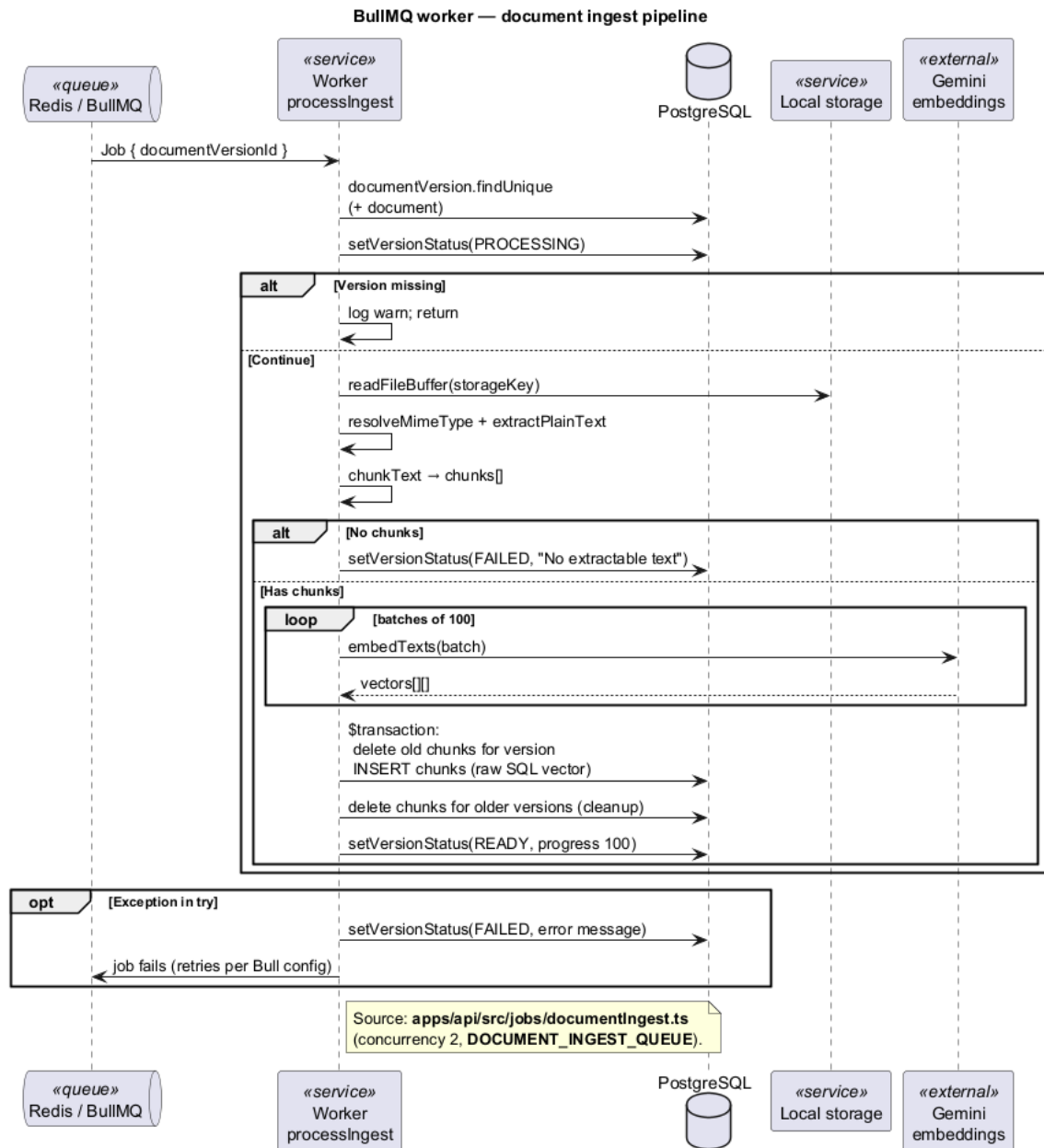


Figure 3.18 – Background document processing (ingest pipeline)

Reprocessing an existing version.

An administrator can trigger reprocessing of a document version—for example after improving the indexing pipeline—without uploading the file again.

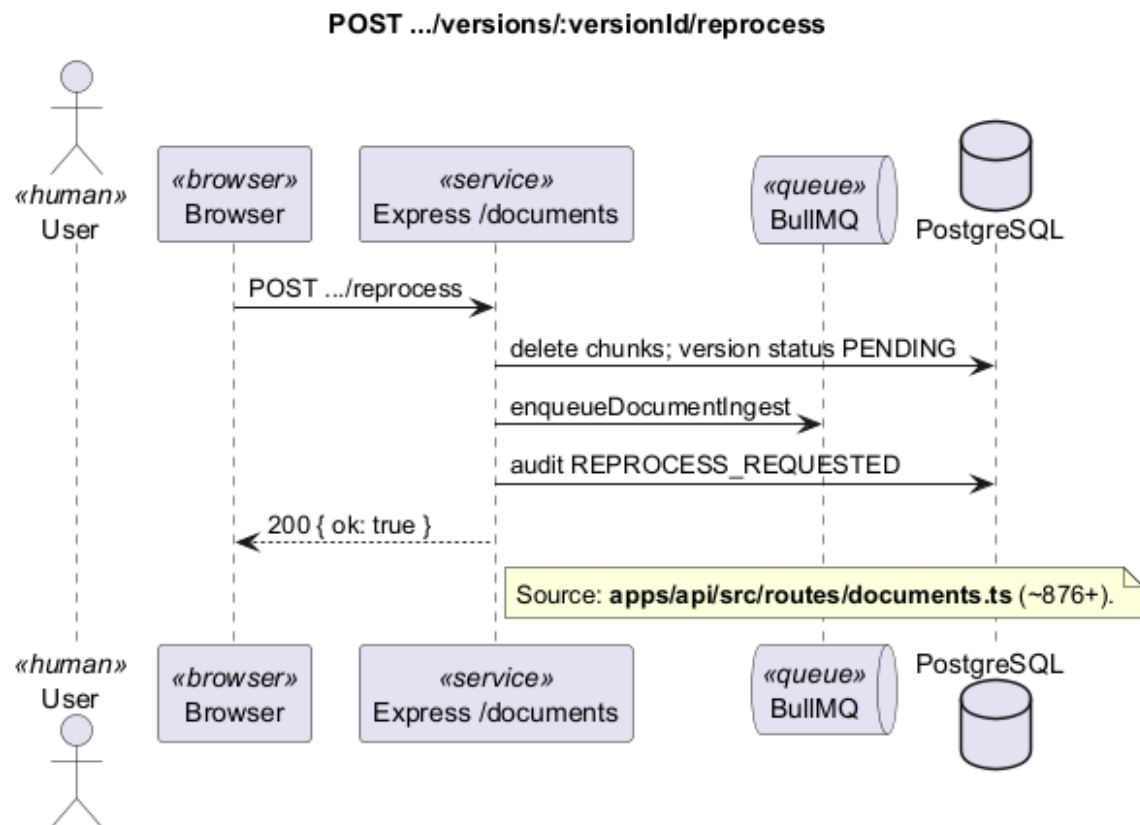


Figure 3.19 – Reprocessing an existing version (reprocess flow)

Browsing and downloading

Downloading a document file.

An authorised user downloads the original file. The platform verifies department access before sending the file.

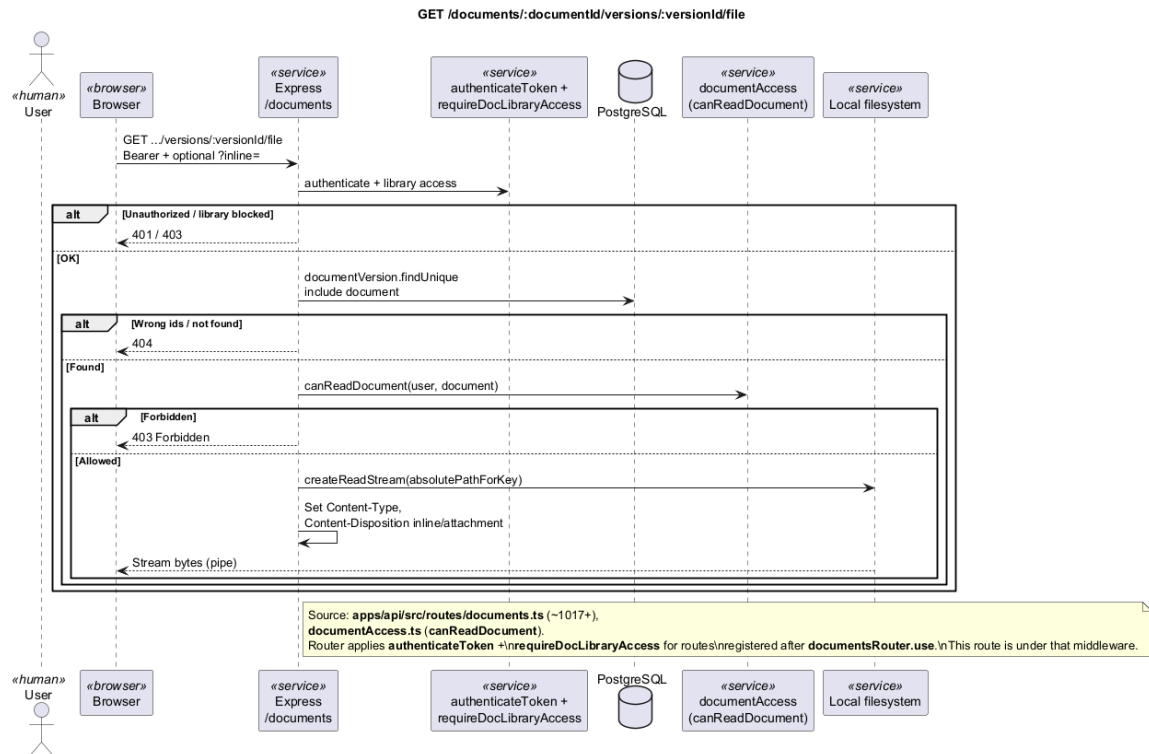


Figure 3.20 – Downloading a document file (download flow)

Browsing the document library.

Users browse the library with filters and pagination. Results show summary information suitable for the catalogue view without loading full text content.

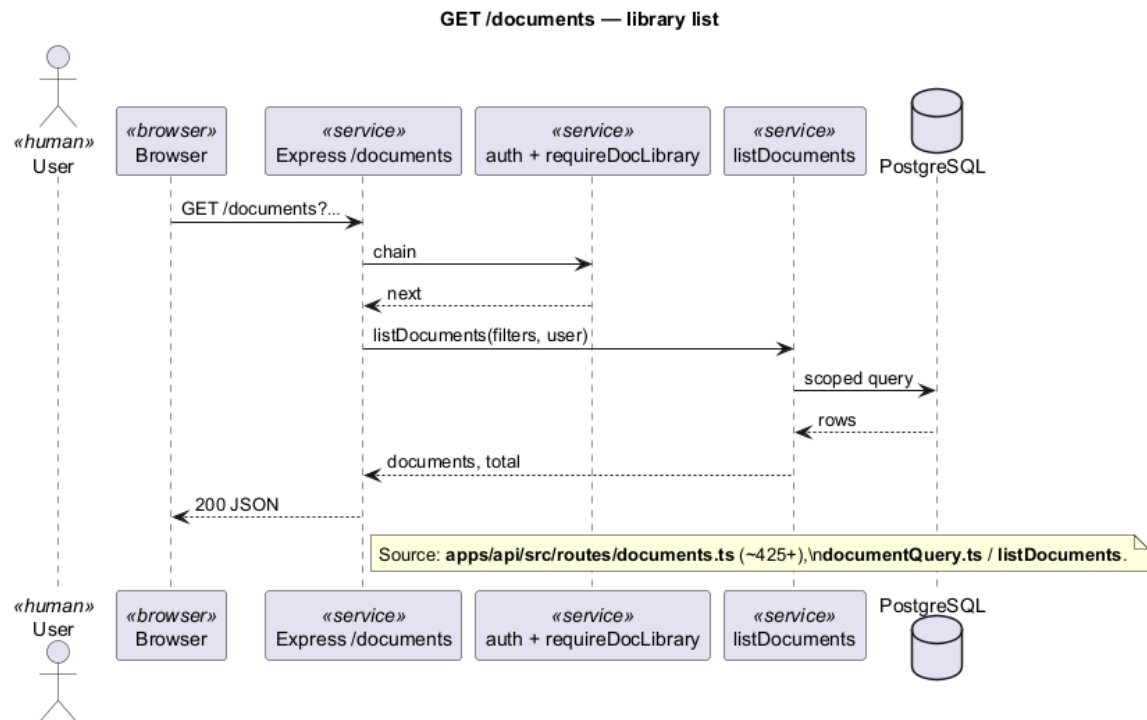


Figure 3.21 – Browsing the document library (library list flow)

Opening a document detail view.

Opening a document shows its metadata, current version, tags, processing status, and the information needed for preview or further actions.

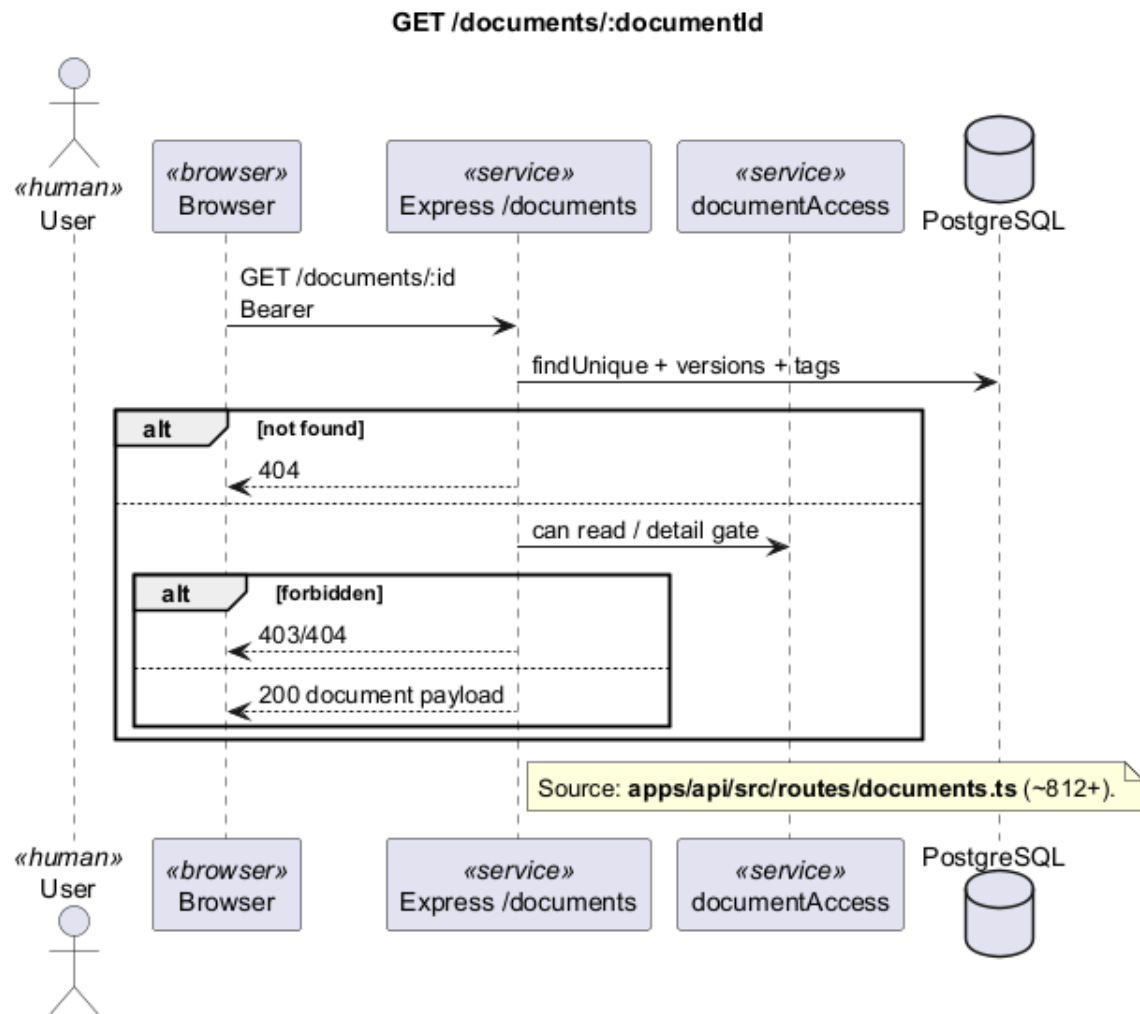


Figure 3.22 – Opening a document detail view (document detail flow)

Favourites and recently viewed documents.

Users can mark documents as favourites and revisit recently opened items from the library home view.

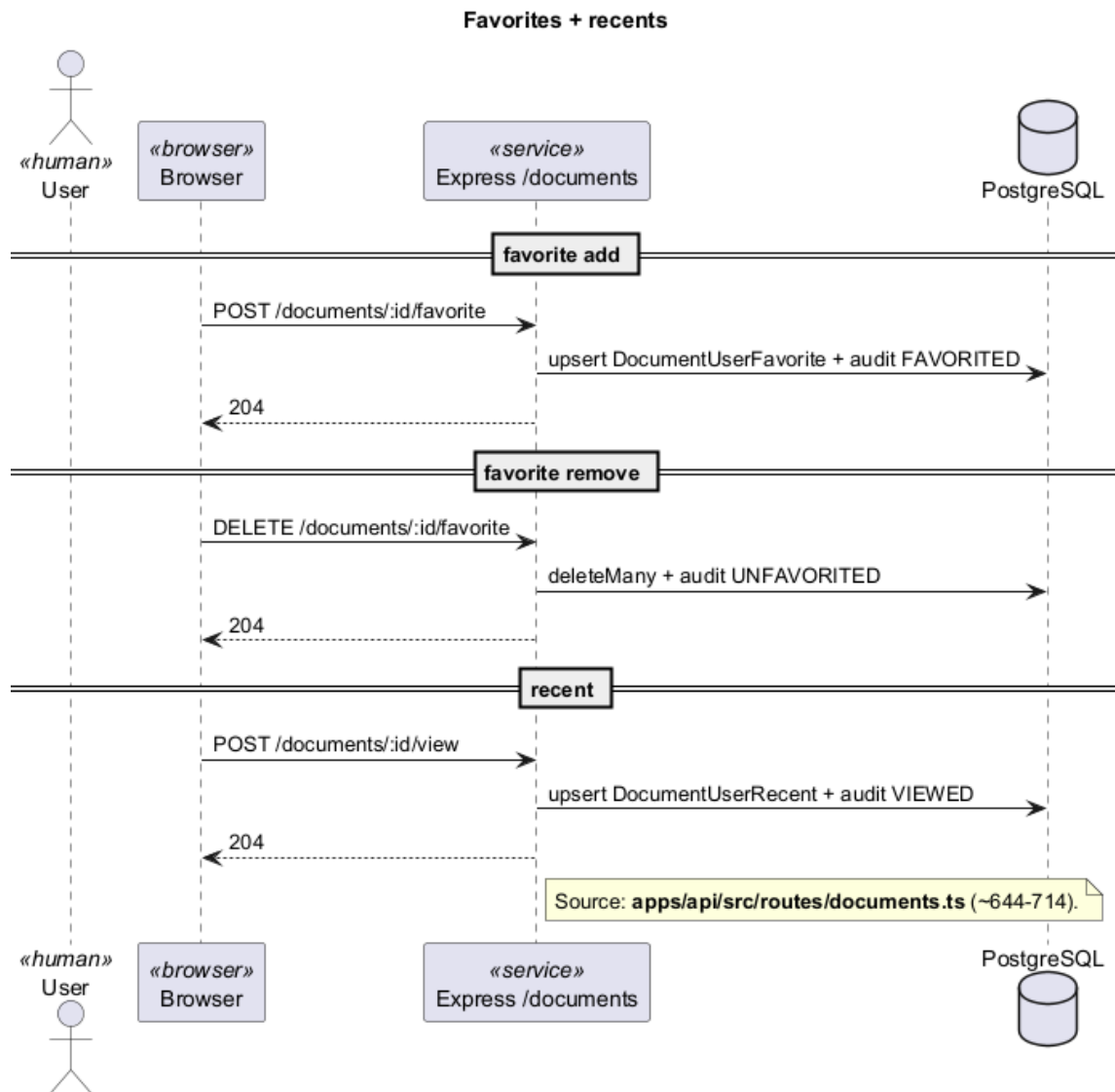


Figure 3.23 – Favourites and recently viewed documents (favourites and recents flow)

Editing metadata and versions

Updating document metadata.

Authorised users can edit title, description, tags, and visibility. Changes are recorded for traceability.

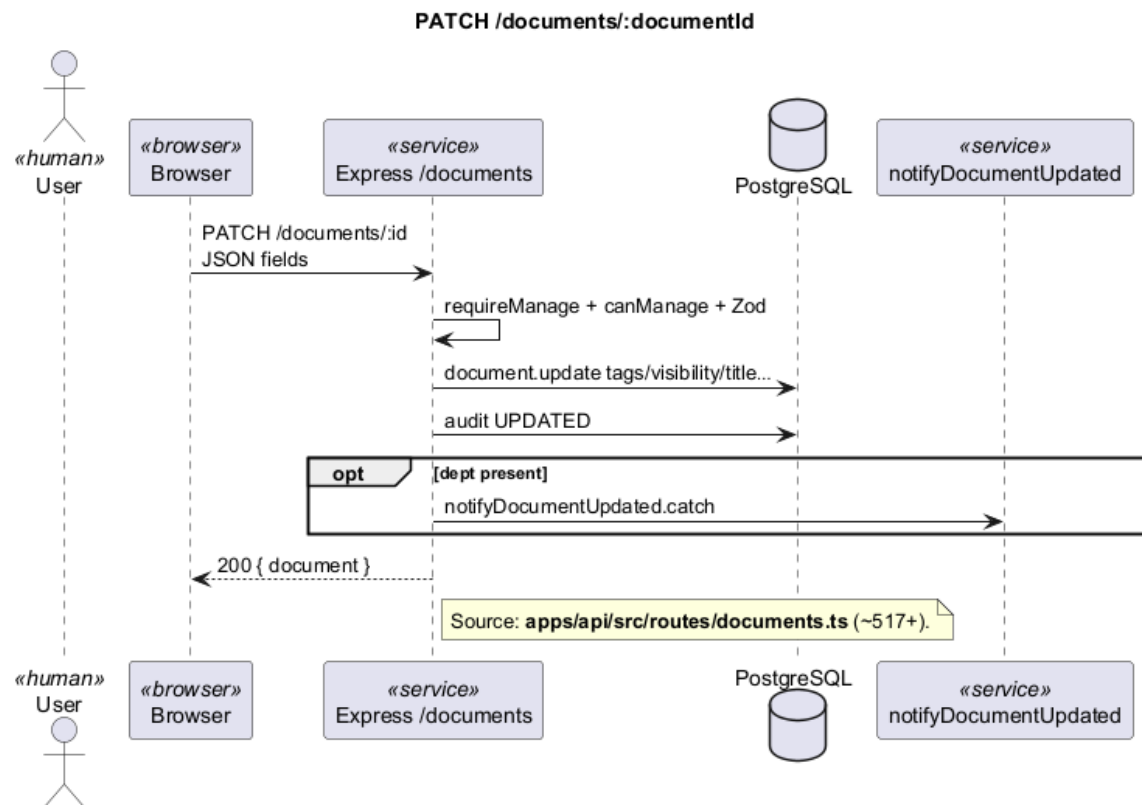


Figure 3.24 – Updating document metadata (metadata update flow)

Publishing a new document version.

Uploading a replacement file creates a new version, starts fresh indexing, and keeps earlier versions available for audit and rollback.

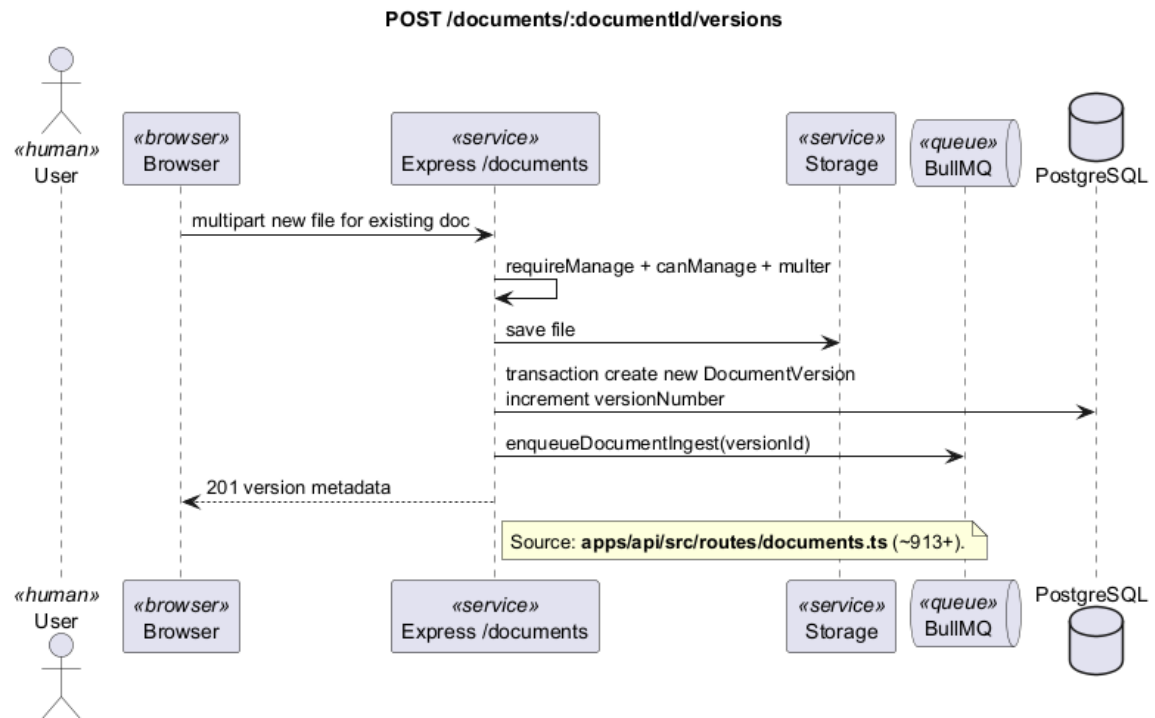


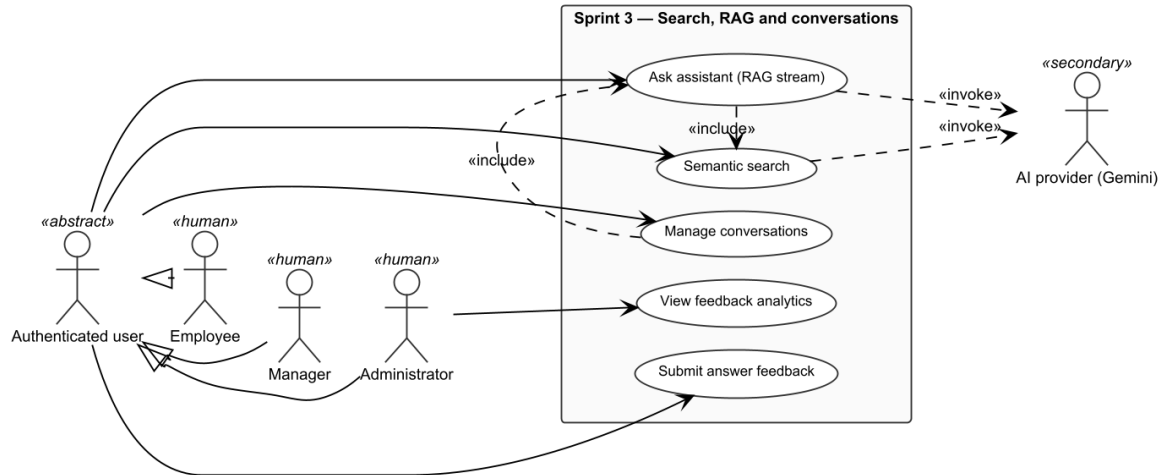
Figure 3.25 – Publishing a new document version (new version flow)

3.5.3 Sprint 3: Search, RAG and conversations

Sprint 3 adds intelligent search over the document base and a conversational workspace where users can ask questions and receive answers grounded in organisational content.

3.5.3.1 Scoped structural views

Use-case model.

Sprint 3 use-case diagram — Search, RAG and conversations**Figure 3.26** – Sprint 3 use-case diagram — Search, RAG and conversations

Domain class model.

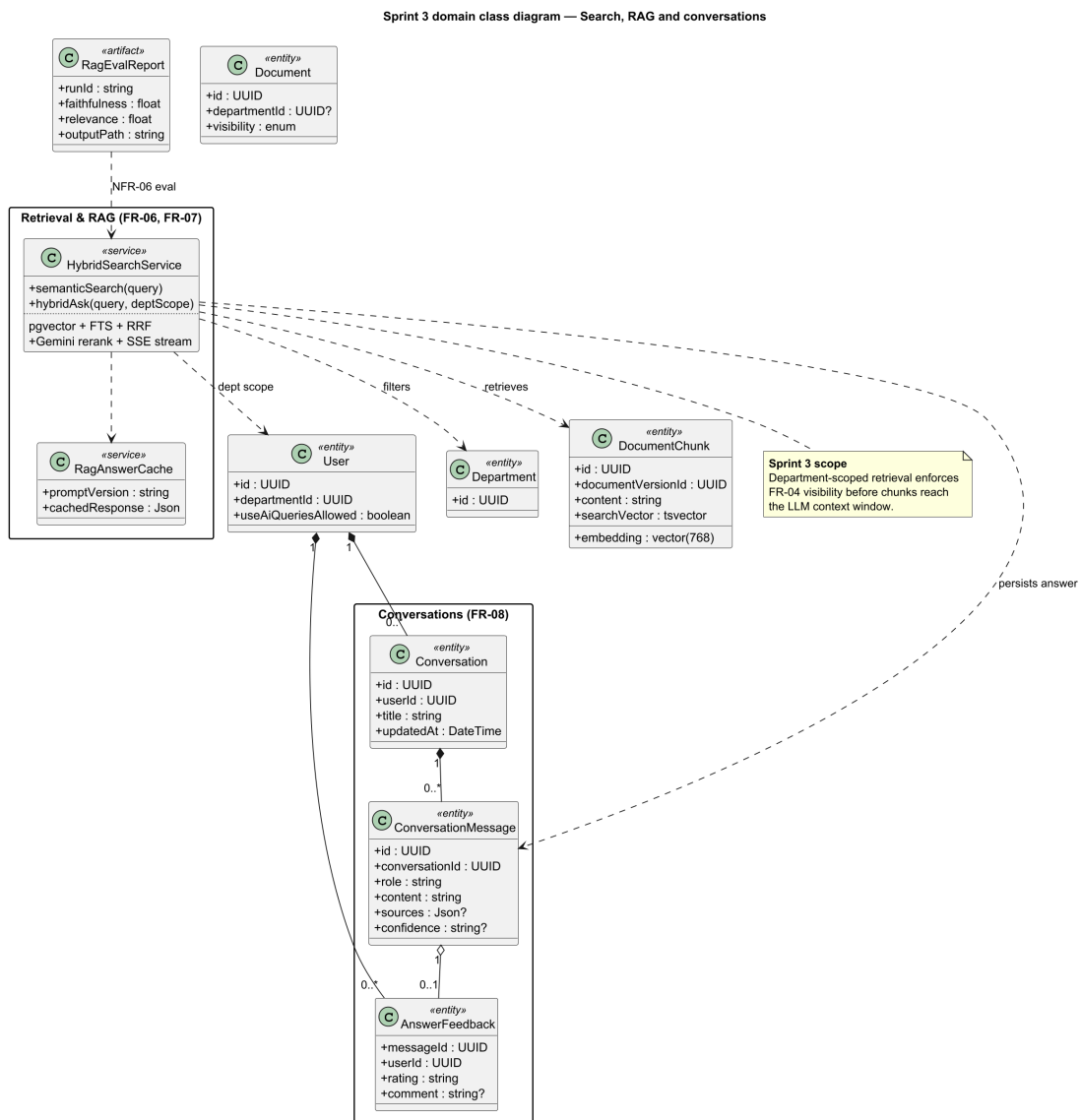


Figure 3.27 – Sprint 3 domain class diagram — Search, RAG and conversations

3.5.3.2 Interaction sequence diagrams

Search and AI-assisted answers

Semantic search.

A user enters a natural-language query; the platform finds the most relevant document passages and returns ranked excerpts for direct exploration.

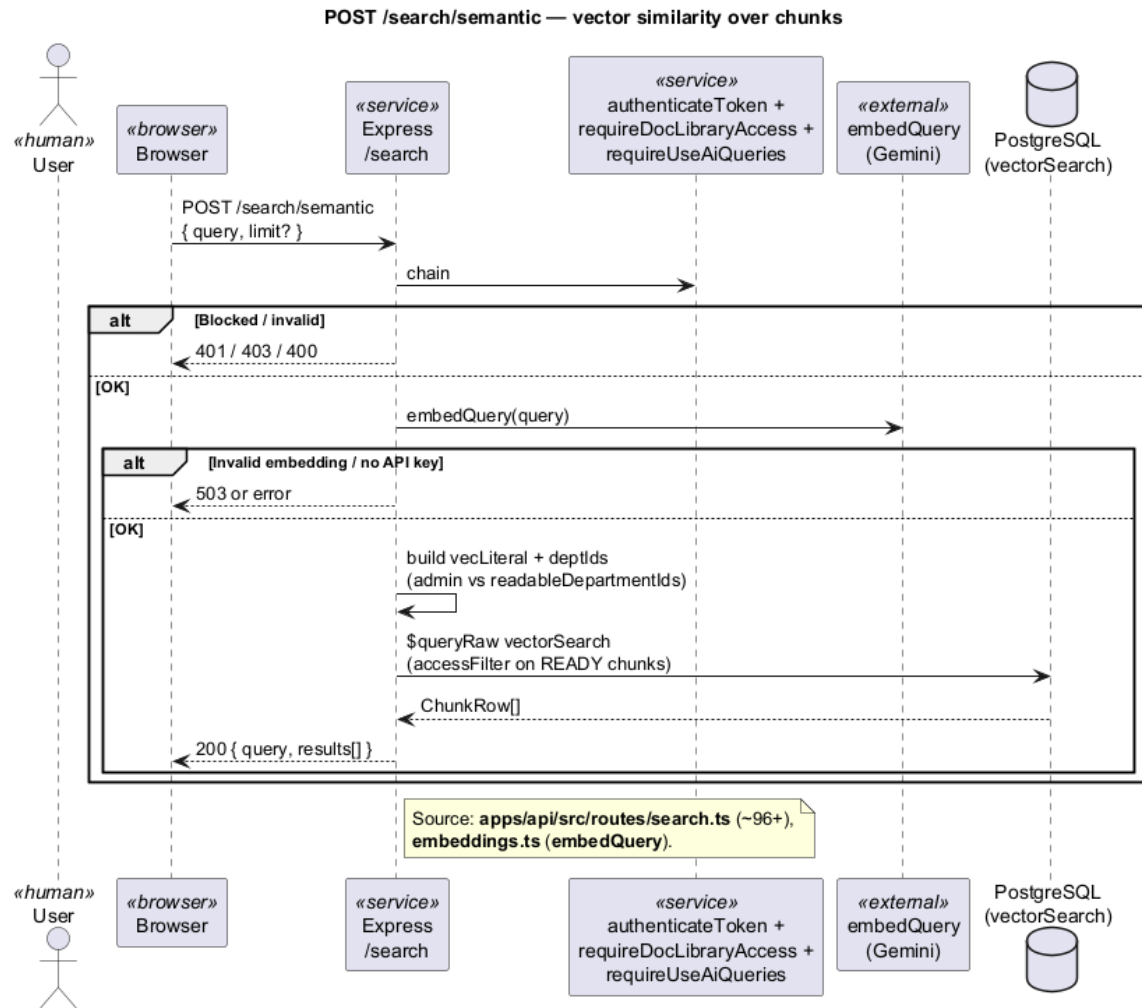


Figure 3.28 – Semantic search (semantic search flow)

Asking a question with AI assistance.

In the Ask workspace, the platform retrieves relevant passages, combines keyword and semantic matching, and streams a structured answer with source references to the user in real time.

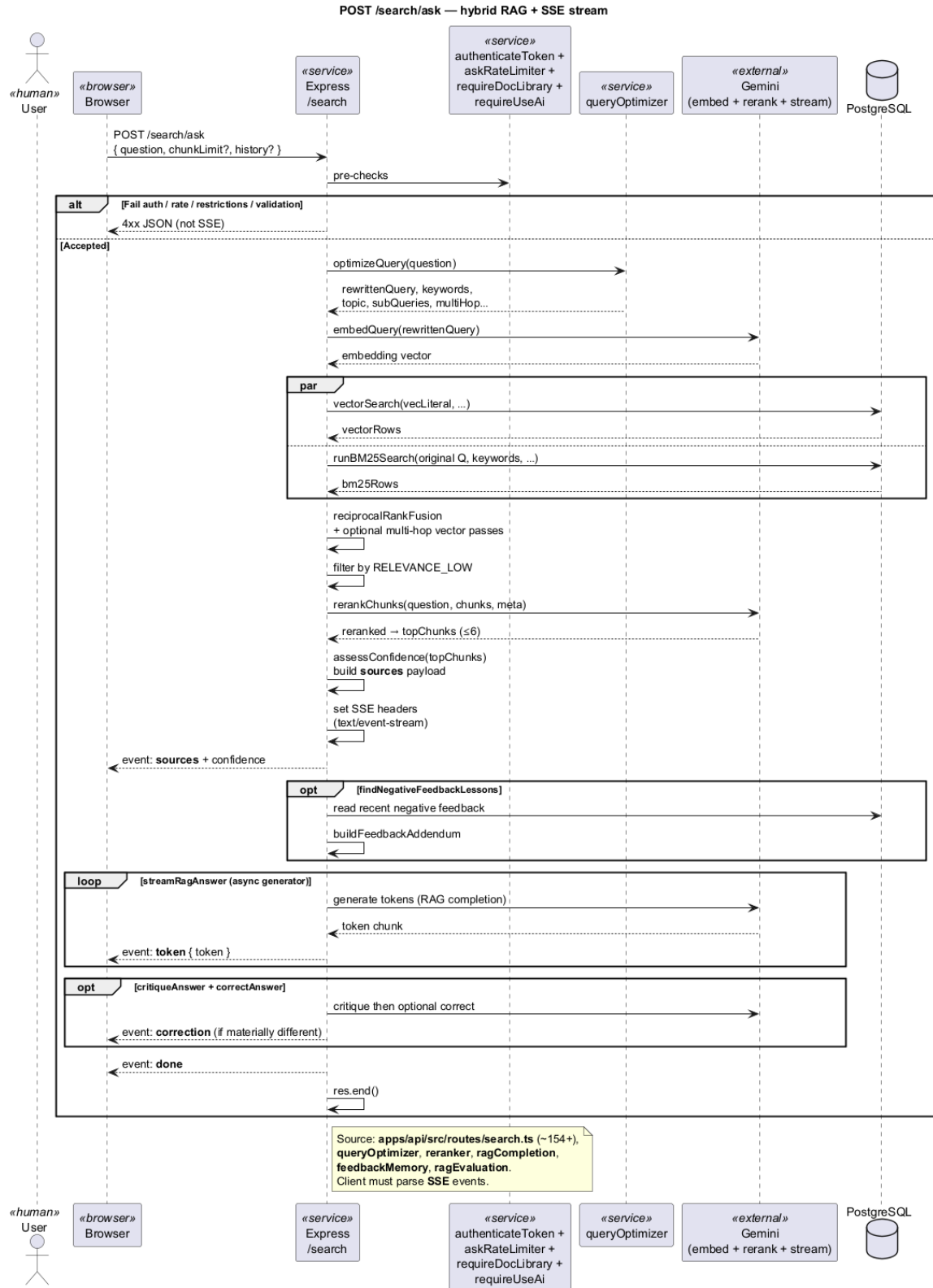


Figure 3.29 – Asking a question with AI assistance (RAG answer flow)

Conversations and answer quality

Creating conversations and adding messages.

Users open a new conversation, ask questions, receive answers, and can automatically generate a short title from the first question. Opening an existing thread loads its full history (see Figure ??).

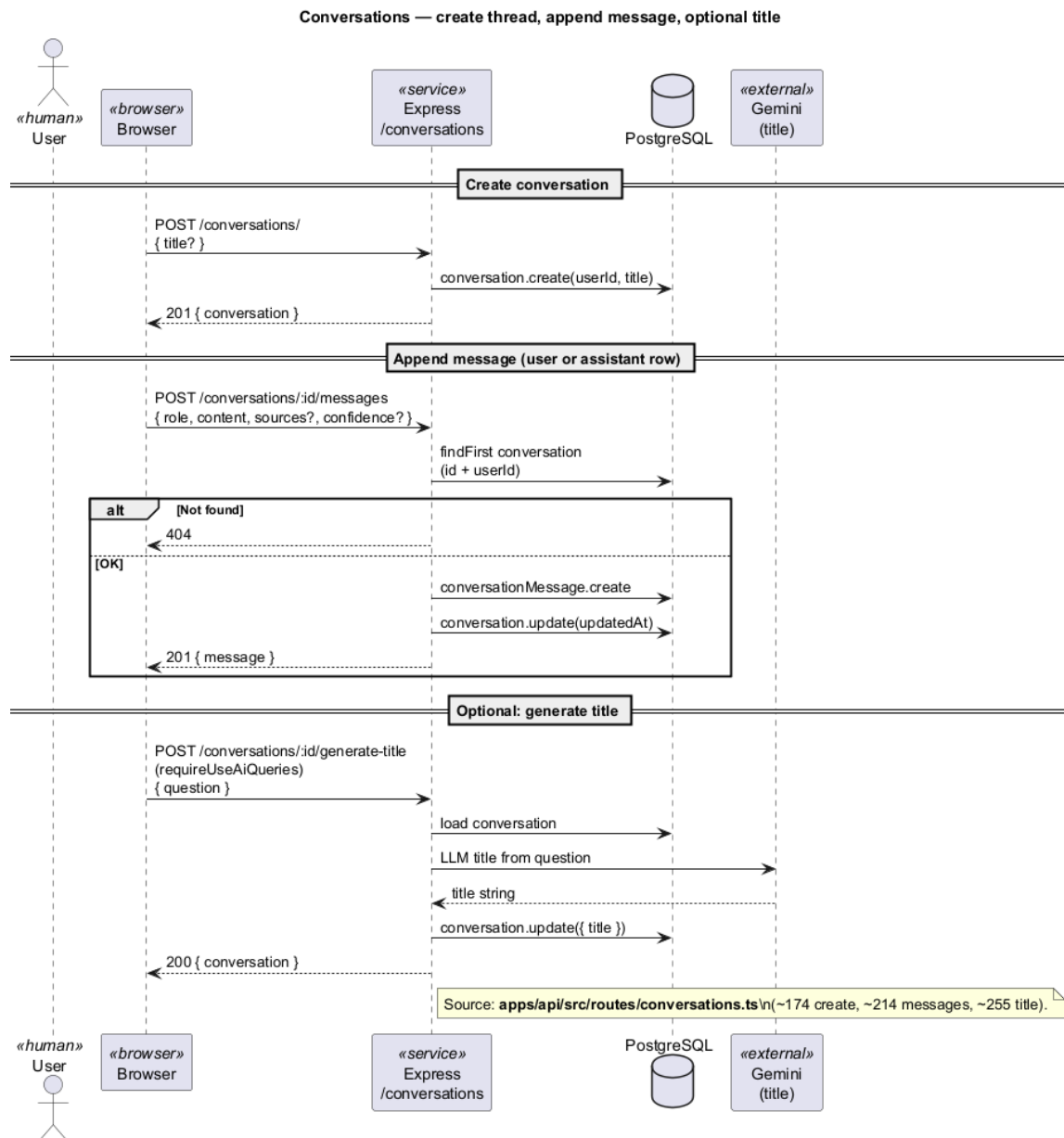


Figure 3.30 – Creating conversations and adding messages (conversation messaging flow)

Rating an answer.

After receiving an answer, users can submit positive or negative feedback with an optional comment, helping administrators monitor answer quality over time.

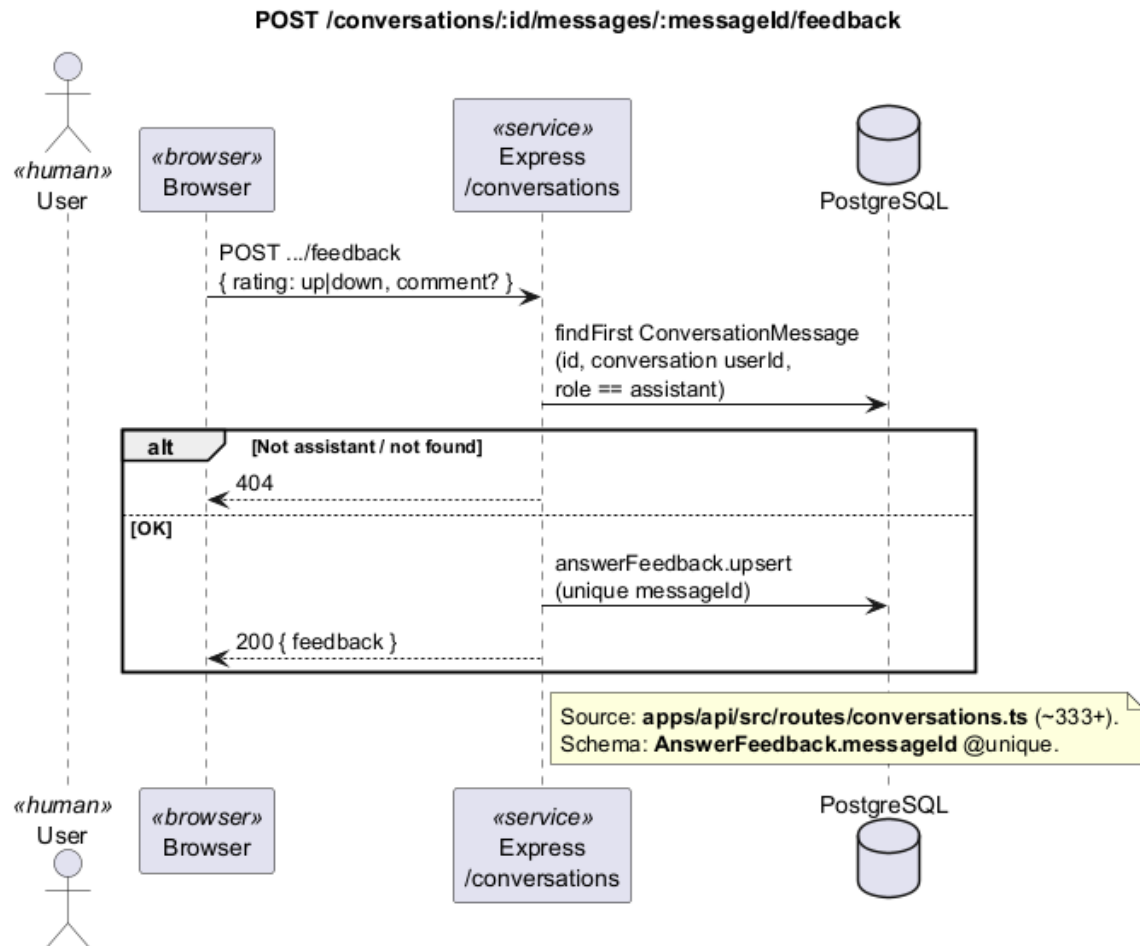


Figure 3.31 – Rating an answer (answer feedback flow)

Managing conversation threads.

Users list past conversations, open a thread with its message history, rename a thread, or delete it when it is no longer needed.

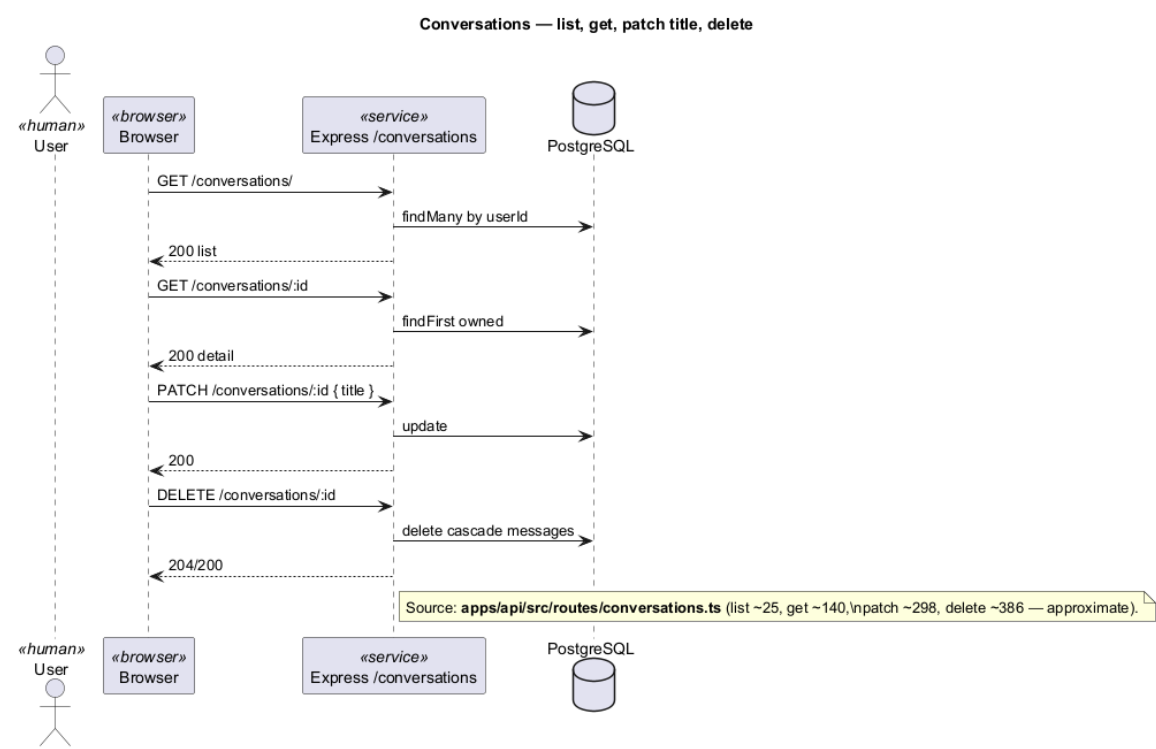


Figure 3.32 – Managing conversation threads (conversation management flow)

Administrator feedback overview.

Administrators consult aggregated feedback metrics—volume, satisfaction ratio, and recent comments—to track how well AI answers are received across the organisation.

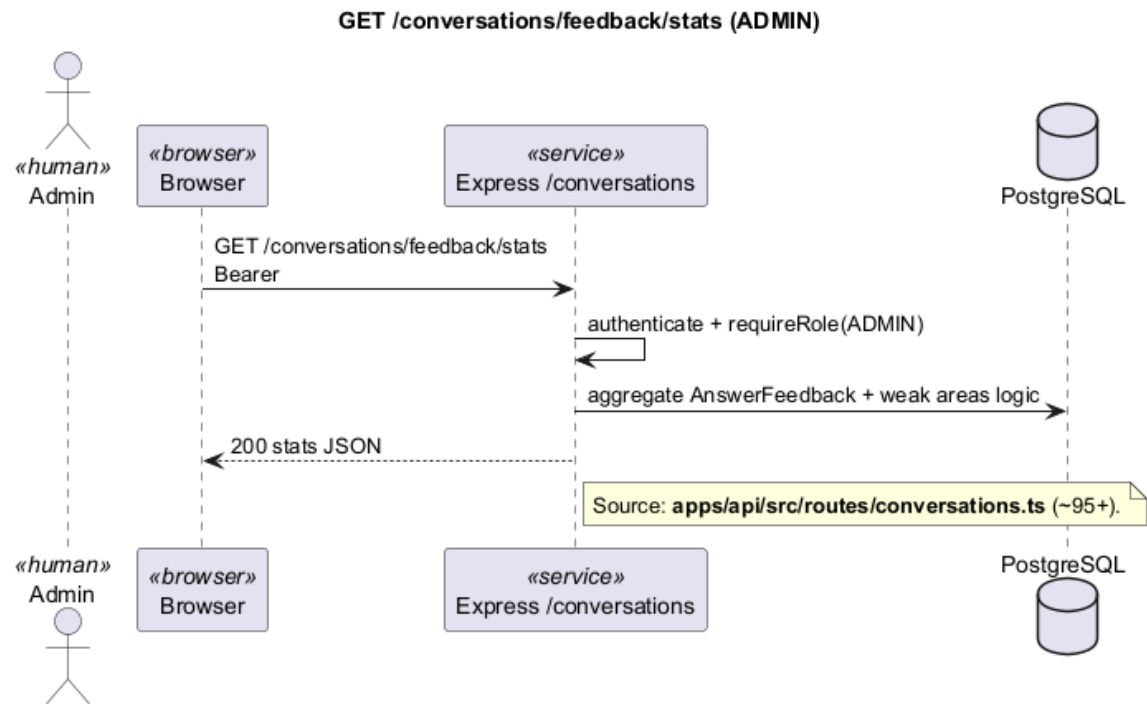


Figure 3.33 – Administrator feedback overview (feedback statistics flow)

3.5.4 Sprint 4: Governance and notifications

Sprint 4 supports day-to-day governance: keeping users informed through notifications, giving managers visibility over their teams, and providing administrators with tools to manage users, departments, and documents at scale.

3.5.4.1 Scoped structural views

Use-case model.

Sprint 4 use-case diagram — Governance and notifications

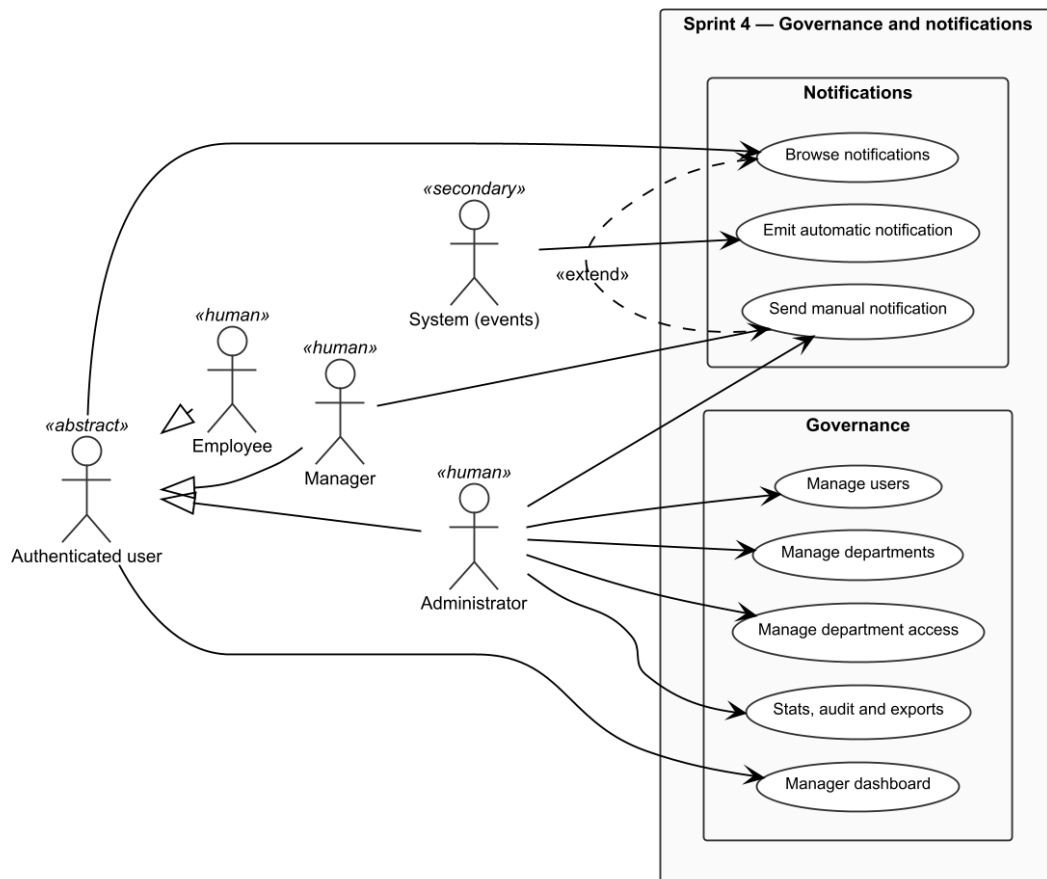


Figure 3.34 – Sprint 4 use-case diagram — Governance and notifications

Domain class model.

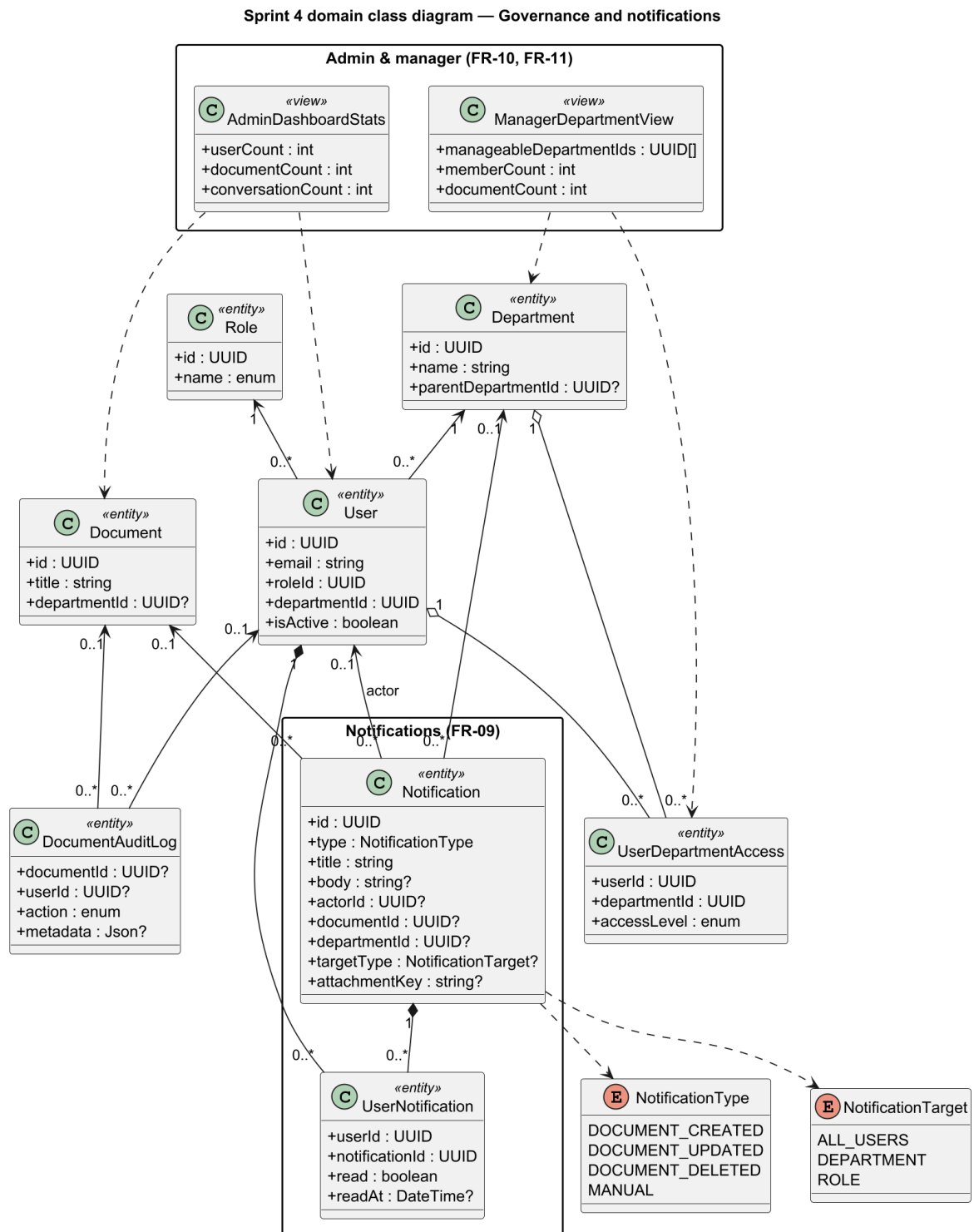


Figure 3.35 – Sprint 4 domain class diagram — Governance and notifications

3.5.4.2 Interaction sequence diagrams

Notifications

Automatic notification when a document is added.

When a new document is published, subscribed department members receive an inbox notification without delaying the upload experience.

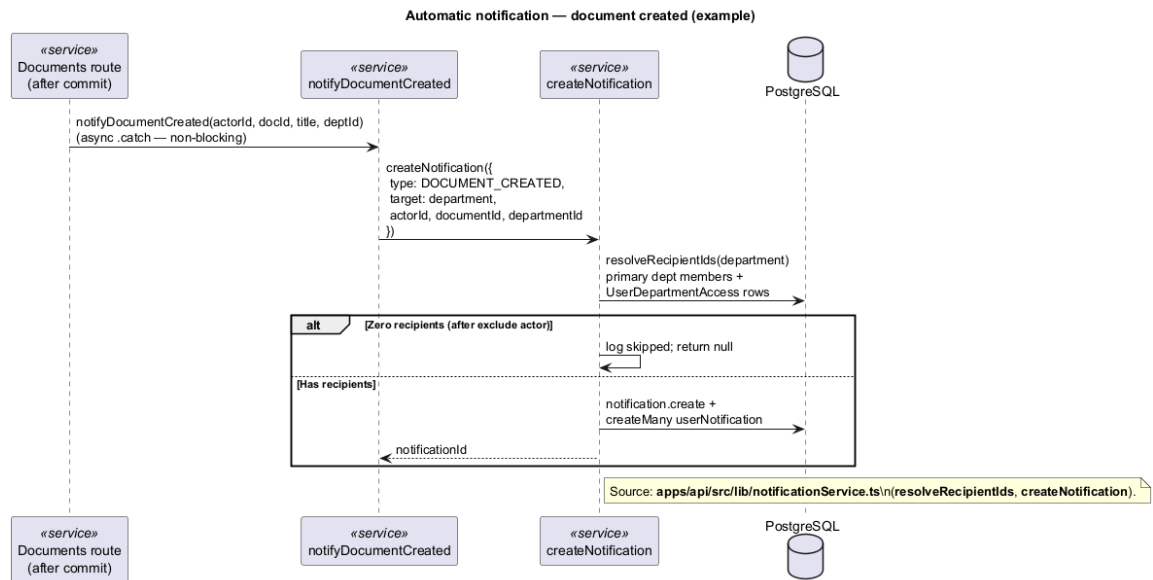


Figure 3.36 — Automatic notification when a document is added (document-created notification)

Sending a manual announcement.

Administrators and managers can send targeted announcements, optionally with attachments, to selected users or departments.

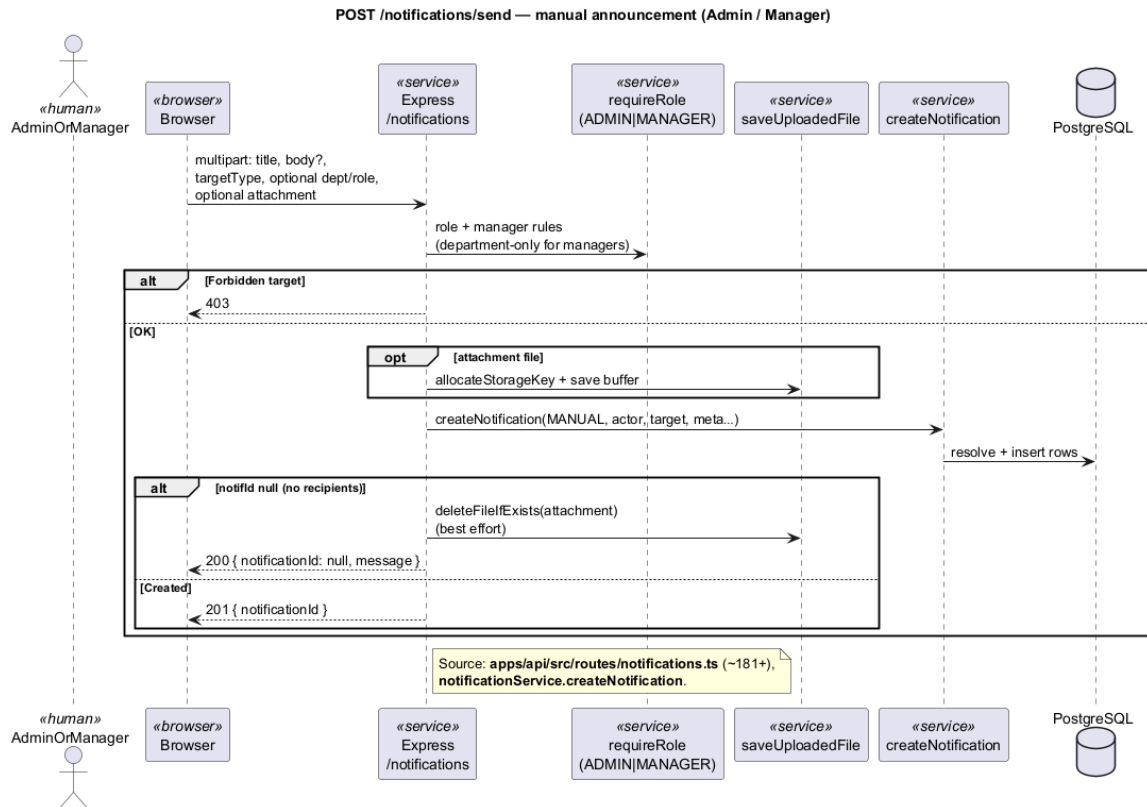


Figure 3.37 – Sending a manual announcement (manual announcement flow)

Reading and managing notifications.

Users open their notification inbox, see unread items, mark messages as read, and access any attached content.

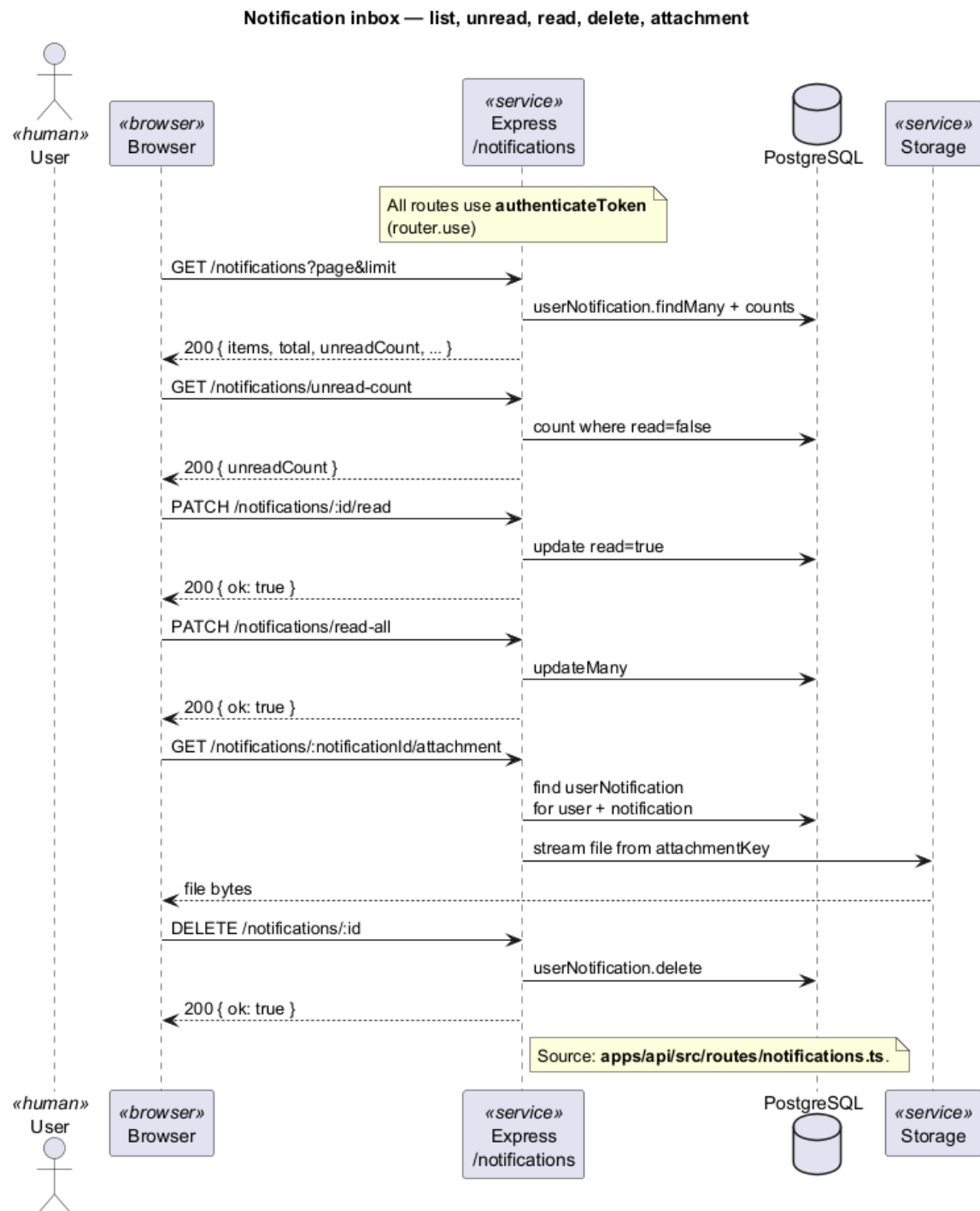


Figure 3.38 – Reading and managing notifications (notification inbox flow)

Manager workspace

Manager department overview.

Managers view the departments they oversee and inspect each department's member list to understand team composition.

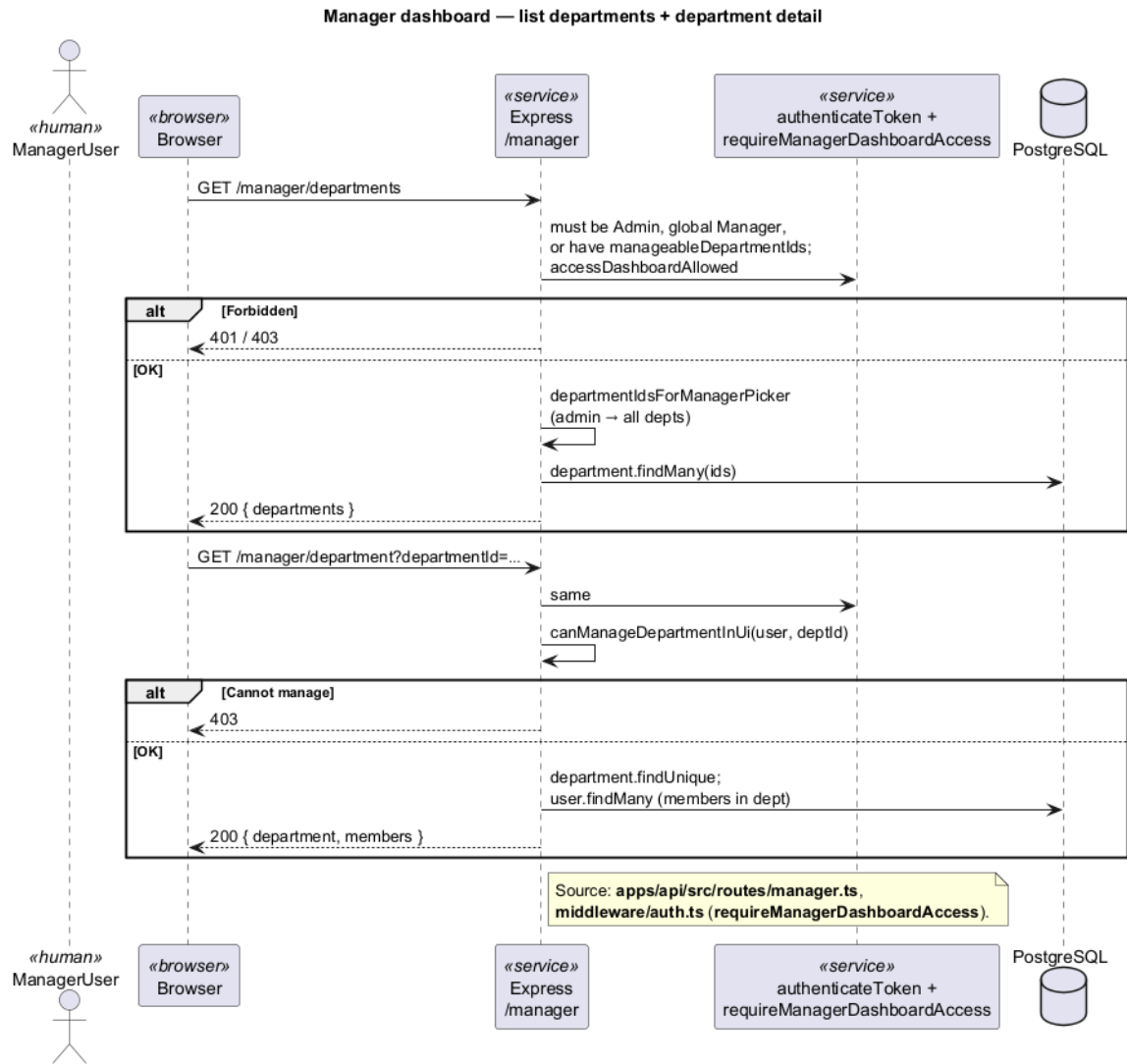


Figure 3.39 – Manager department overview (manager department view)

Administrator governance

Creating a user account.

Administrators create accounts with an initial role, department access, and optional feature restrictions. Credentials are shared through organisational channels.

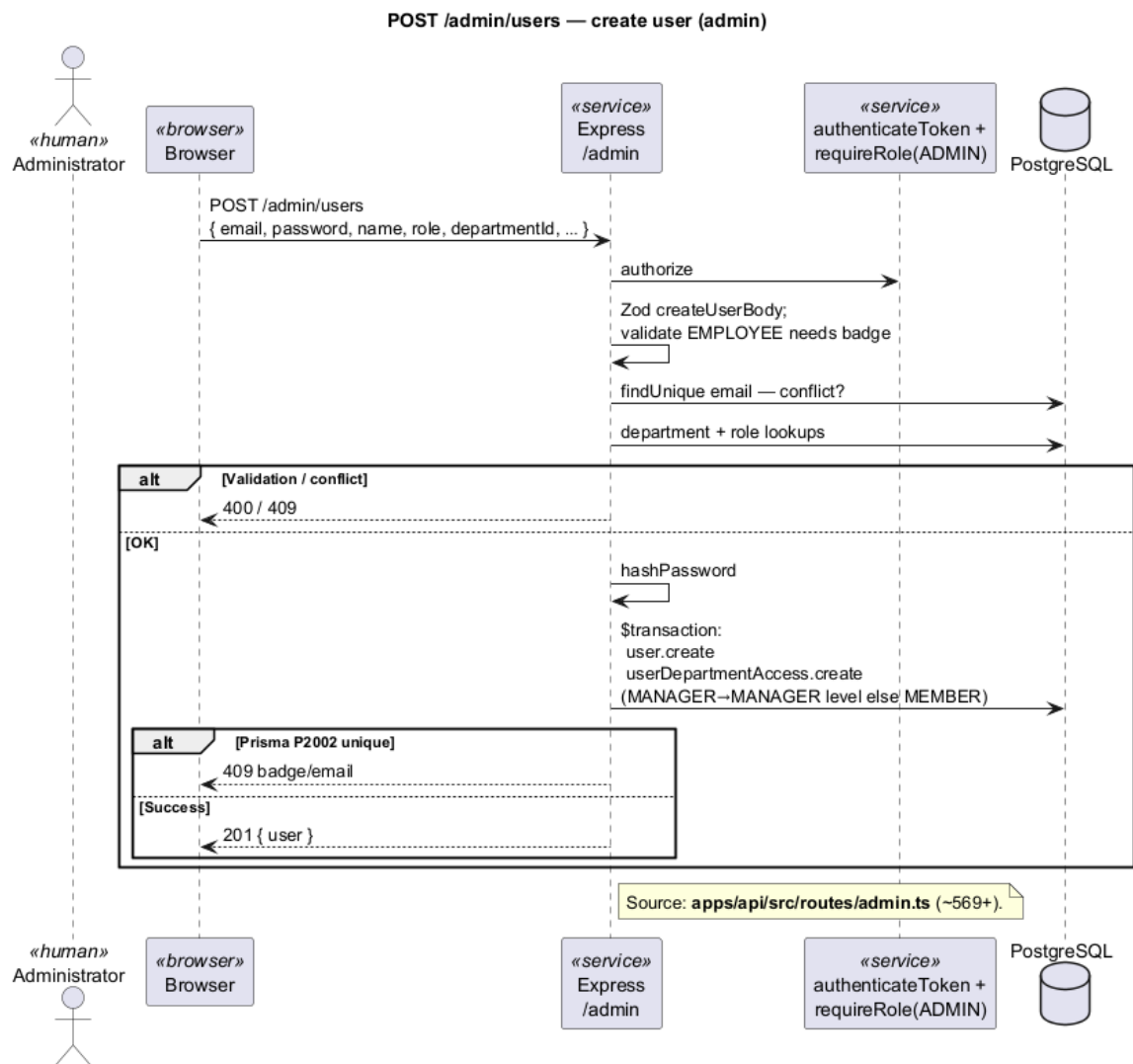


Figure 3.40 – Creating a user account (user creation flow)

Merging two departments.

Department merge moves documents, access rights, and memberships from one department into another while preserving data consistency.

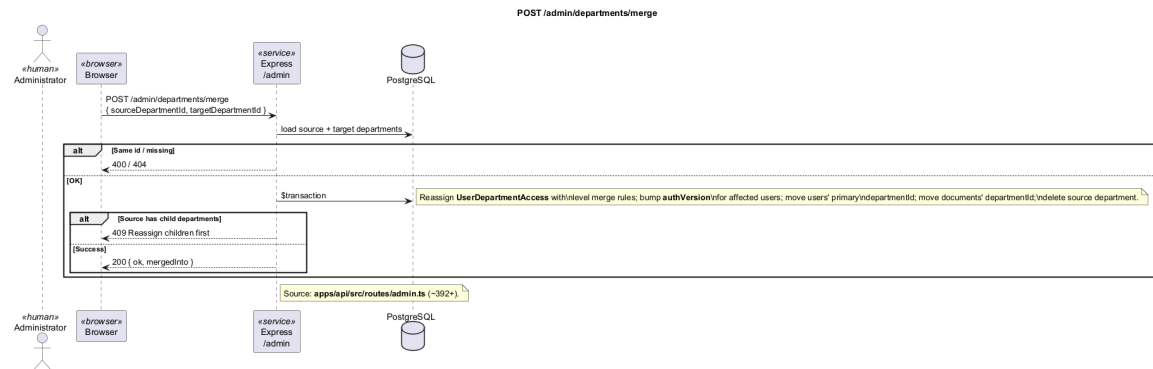


Figure 3.41 – Merging two departments (department merge flow)

Replacing a user's department access in bulk.

Administrators can replace all department-access entries for one user in a single operation, which simplifies onboarding and access reviews.

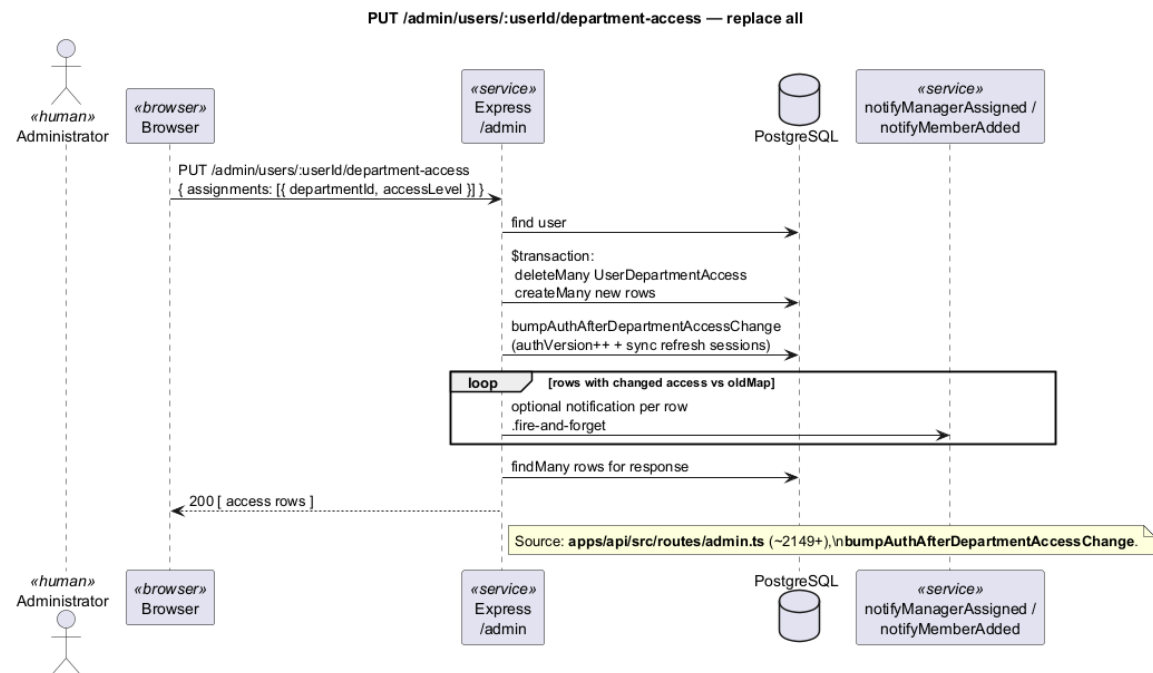


Figure 3.42 – Replacing a user's department access in bulk (bulk access update flow)

Creating a department.

Administrators add a new organisational unit with a unique identifier. The operation is logged for audit purposes.

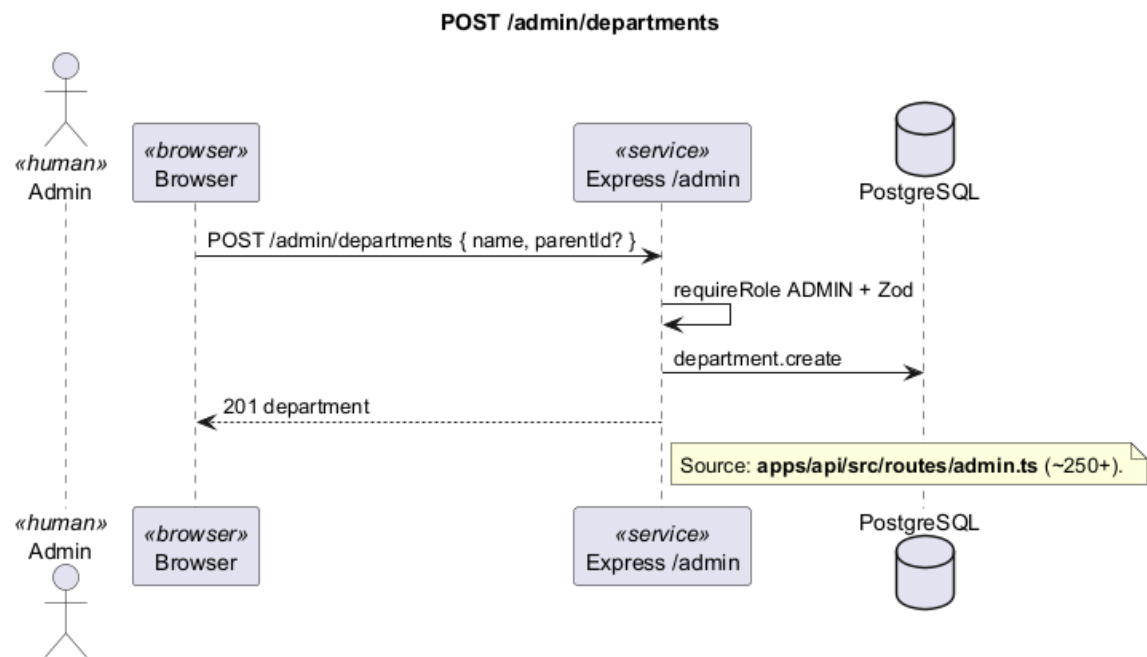


Figure 3.43 – Creating a department (department creation flow)

Granting or revoking single department access.

Fine-grained actions add or remove access to one department for one user without rewriting the entire access matrix.

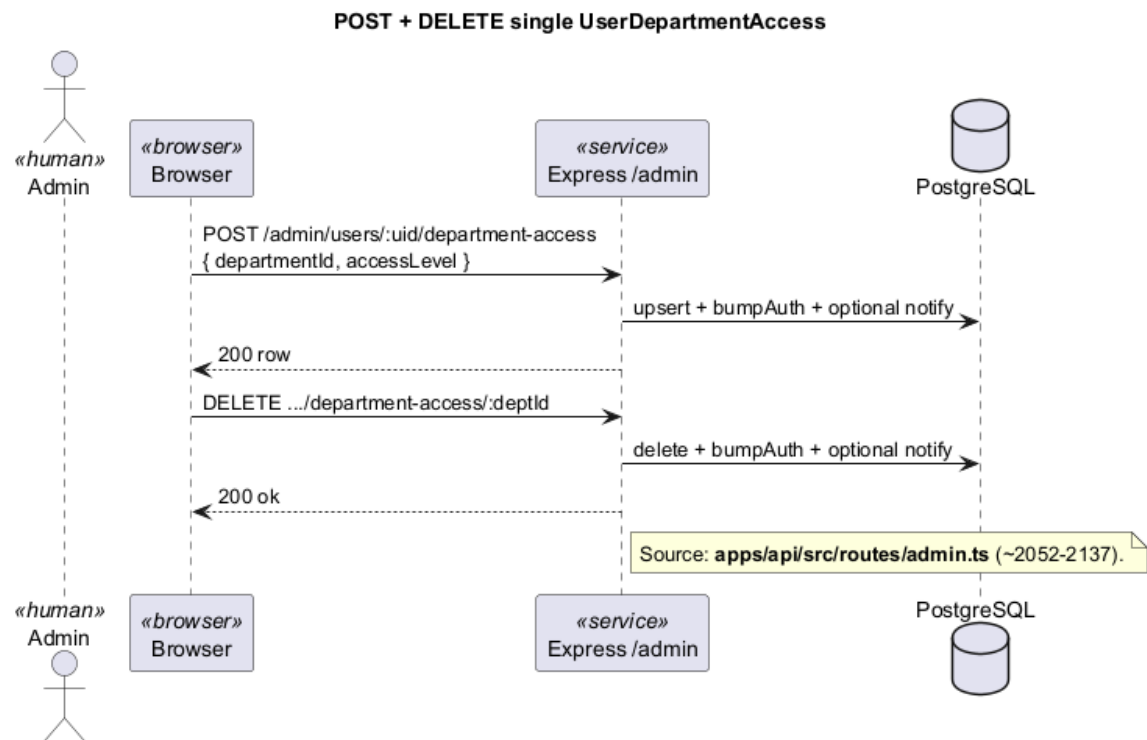


Figure 3.44 – Granting or revoking single department access (single access grant or revoke)

Platform-wide statistics.

Administrators view high-level indicators such as user count, document volume, conversation activity, and indexing backlog.

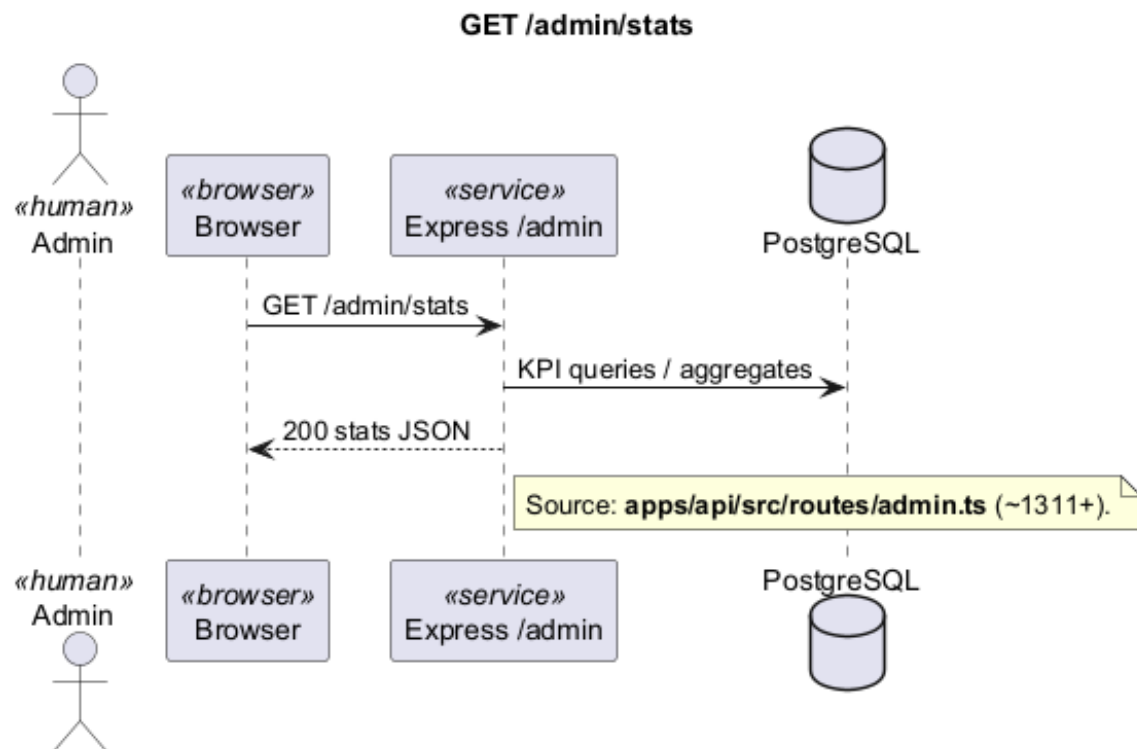


Figure 3.45 – Platform-wide statistics (admin statistics flow)

Activity log and export.

Administrators browse recent platform activity and export filtered events to spreadsheet format for offline review.

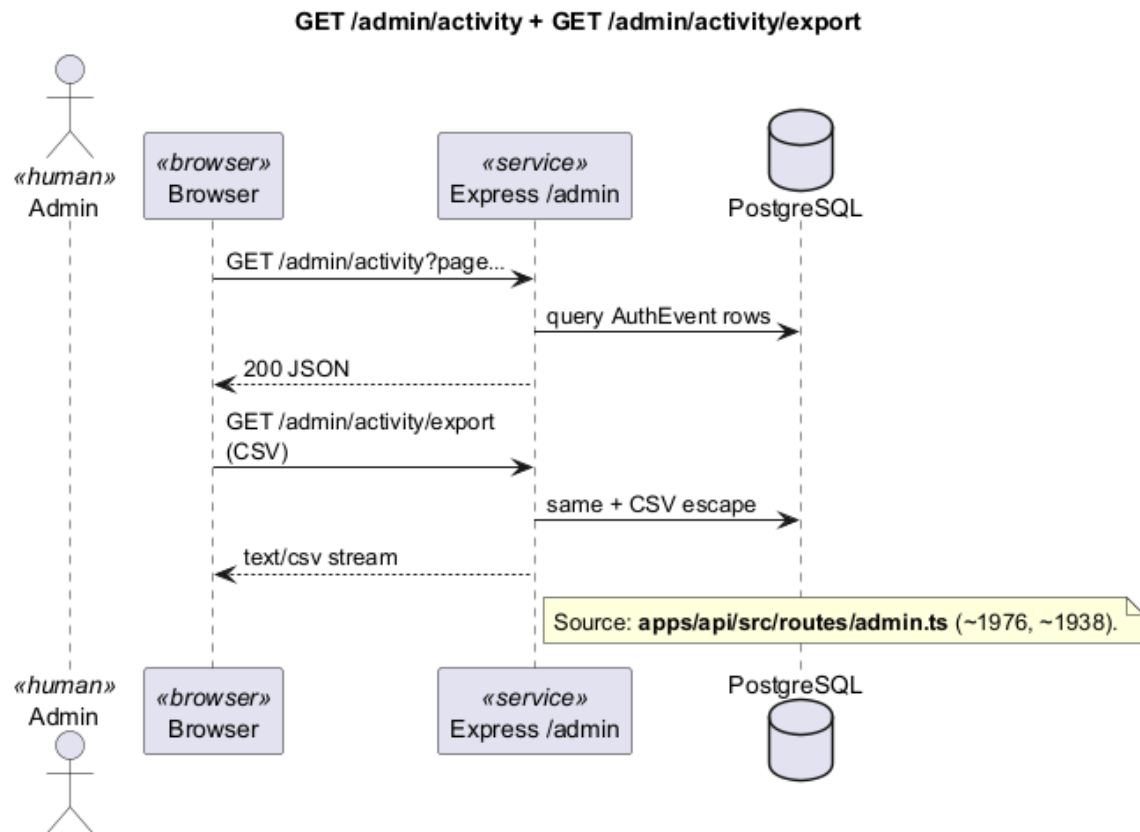


Figure 3.46 – Activity log and export (activity log export flow)

Document change audit log.

Administrators review and export a platform-wide log of document changes— metadata, visibility, and version updates—to support compliance and access reviews.

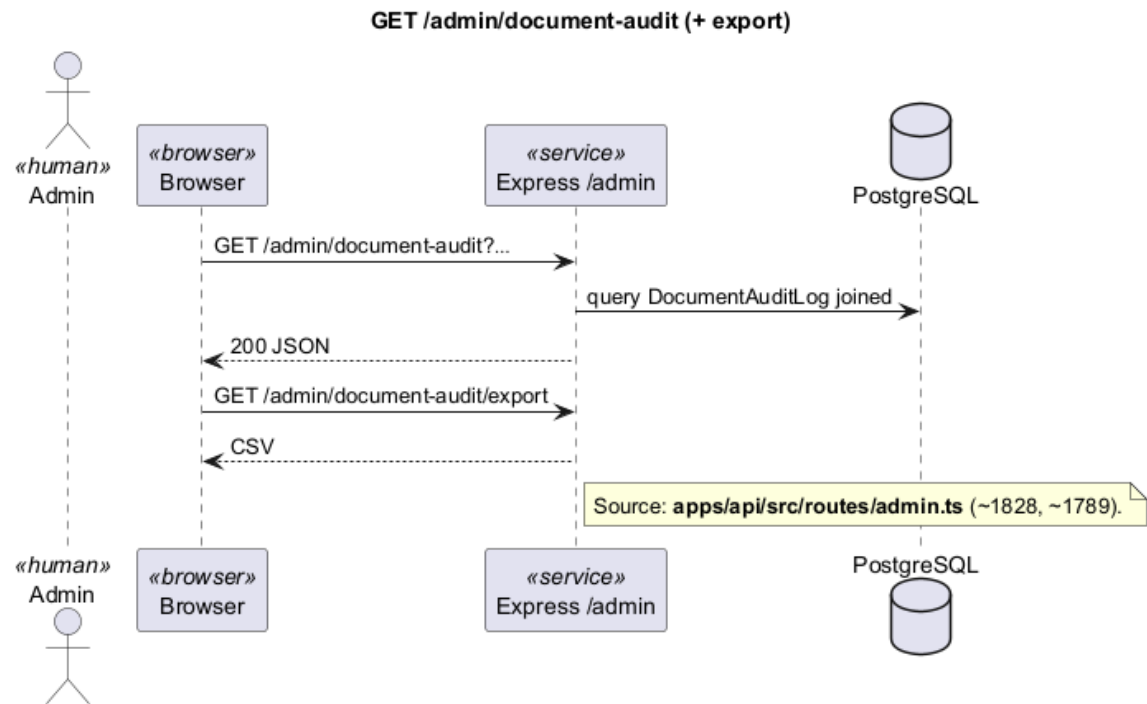


Figure 3.47 – Document change audit log (document audit log flow)

Exporting the document catalogue.

Administrators export the document inventory—titles, departments, tags, and processing status—for reconciliation or reporting.

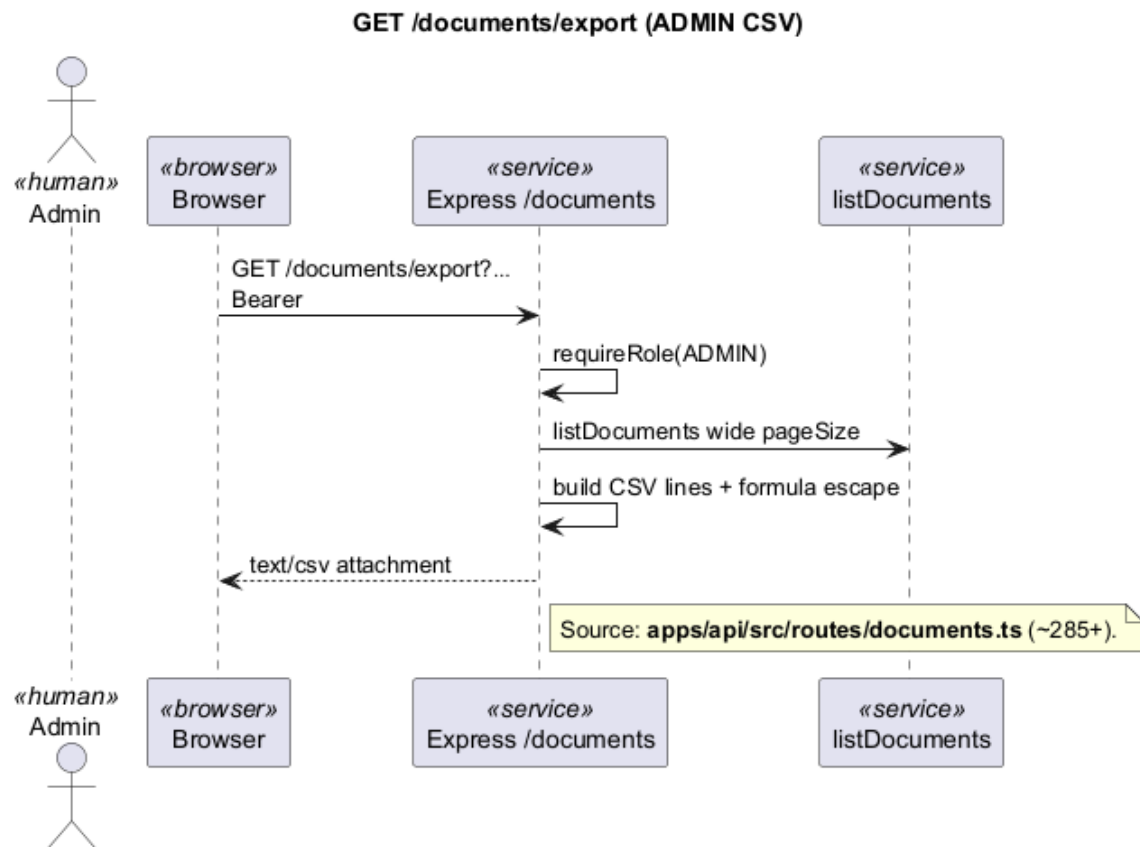


Figure 3.48 – Exporting the document catalogue (document catalogue export)

Deleting multiple documents.

Administrators remove a selected set of documents in one guarded operation, including stored files and related records.

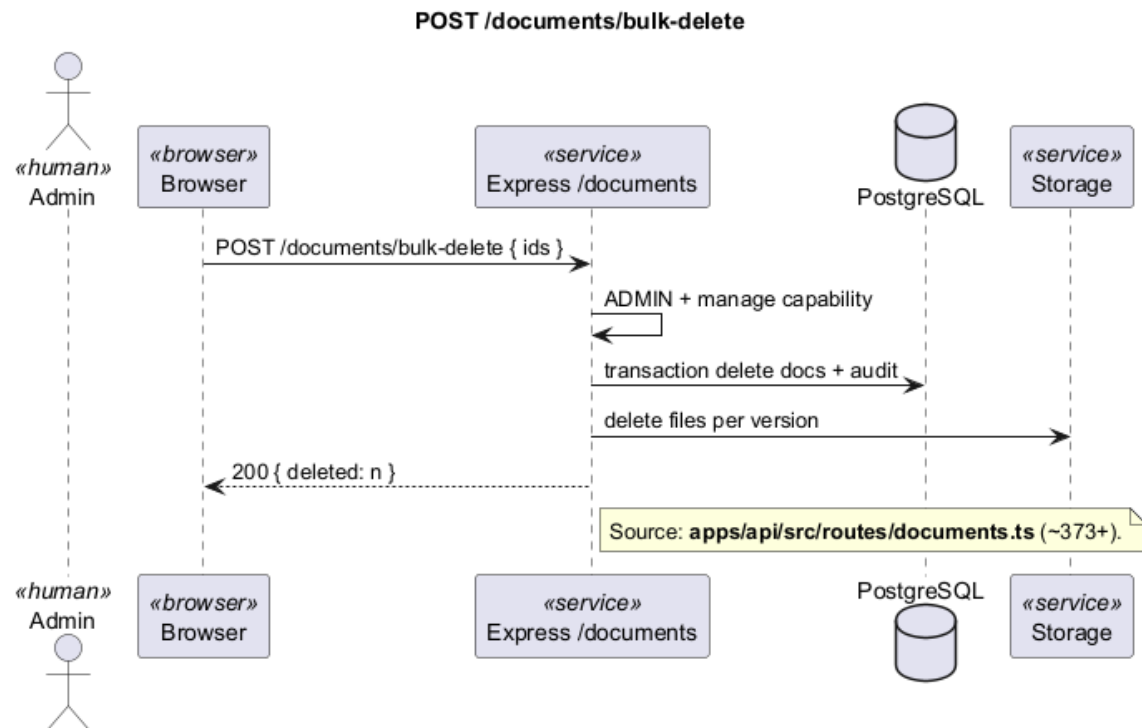


Figure 3.49 – Deleting multiple documents (bulk delete flow)

Updating or locking a user account.

Administrators adjust roles, restrictions, or active status. A sign-in lock prevents further authentication attempts for a defined period without affecting unrelated accounts.

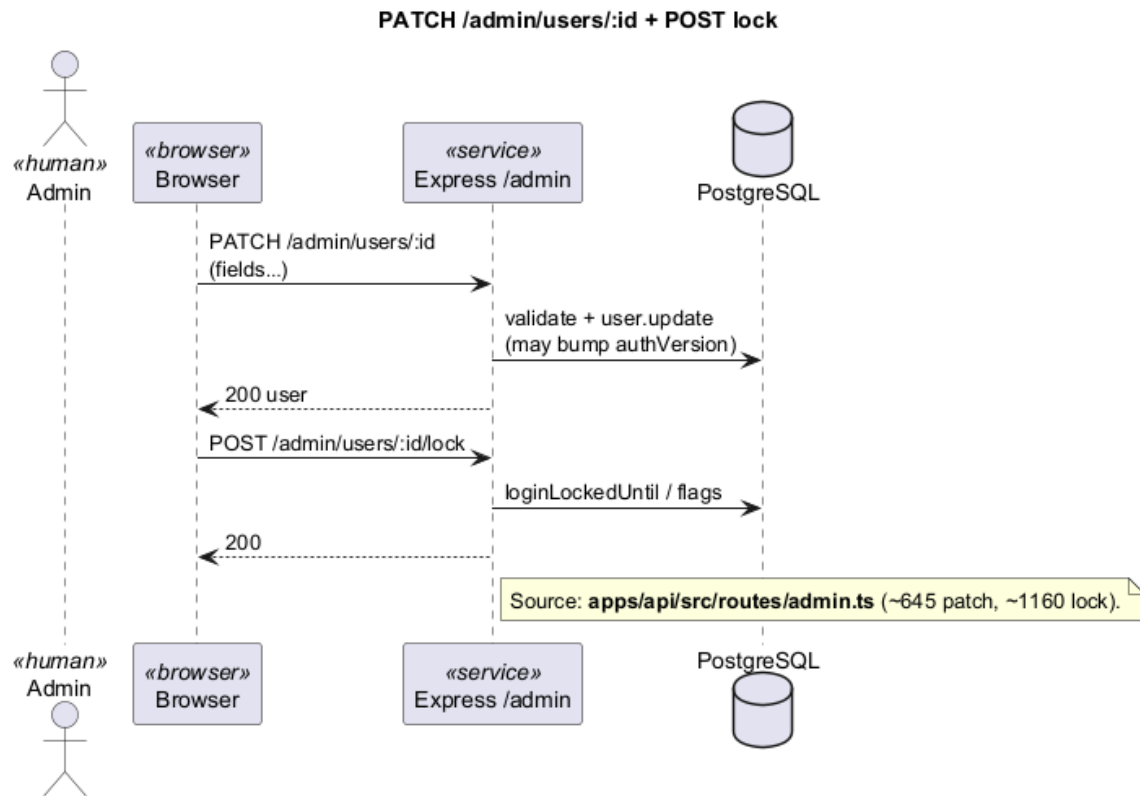


Figure 3.50 – Updating or locking a user account (user update and lock flow)

3.5.5 Sprint 5: Operations, testing and deployment

Sprint 5 focuses on operability: confirming that the platform is healthy, packaging it for reproducible deployment, and validating behaviour through automated tests and AI quality checks.

3.5.5.1 Scoped structural views

Use-case model.

Sprint 5 use-case diagram — Operations, testing and deployment

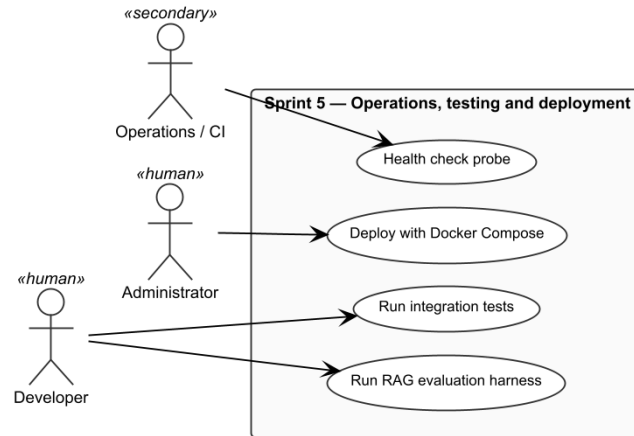


Figure 3.51 – Sprint 5 use-case diagram — Operations, testing and deployment

Domain class model.

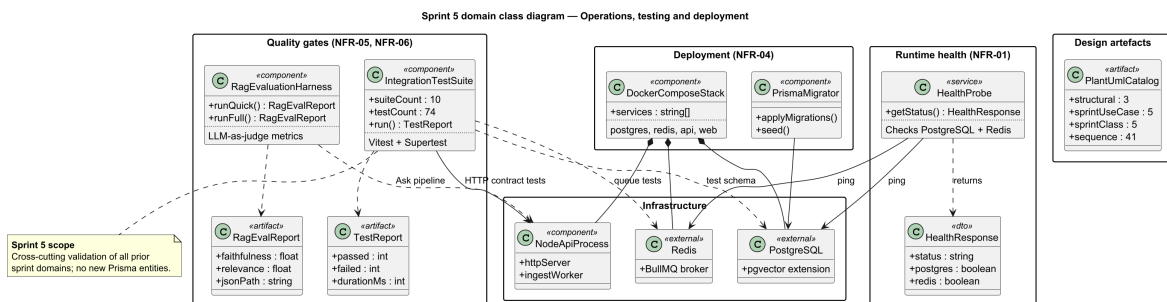


Figure 3.52 – Sprint 5 domain class diagram — Operations, testing and deployment

3.5.5.2 Interaction sequence diagrams

Service health check.

Monitoring tools call a public health endpoint to verify that the application is running and can reach its database and cache services.

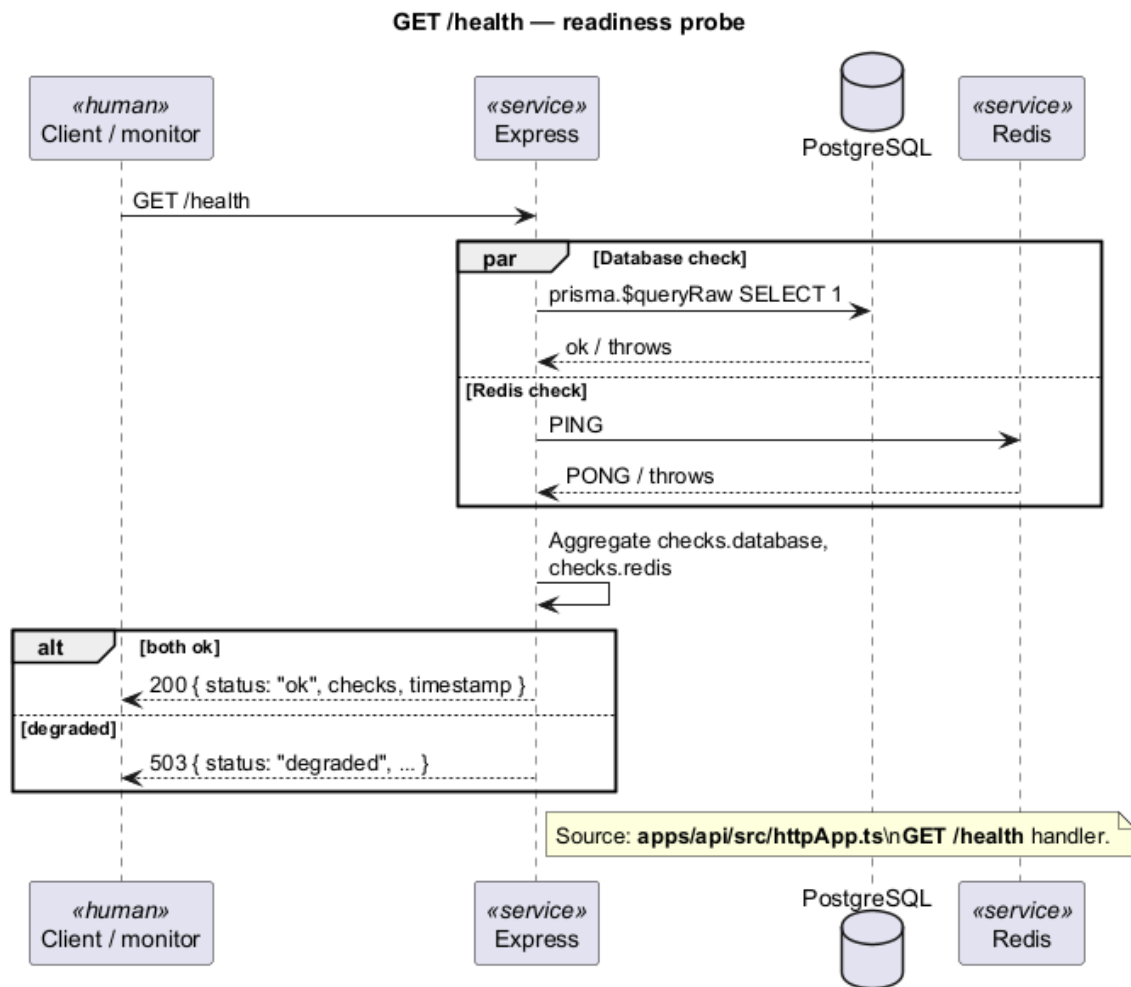


Figure 3.53 – Service health check (health check flow)

3.5.5.3 Operational deliverables

Integration test suite.

An automated test suite exercises the main user journeys—authentication, documents, search, conversations, administration, and manager views—so regressions are detected before release.

Containerised deployment.

The platform is packaged with its database, cache, API, and web application so the full environment can be started consistently on any workstation or in continuous integration.

RAG quality evaluation.

A repeatable evaluation workflow runs predefined questions through the Ask pipeline and scores answer quality, producing reports that can be compared across development iterations.

3.6 Chapter Conclusion

This chapter established how the Knowledge Platform is structured and how its parts interact at design time. The three-tier architecture separates user-facing workflows from business coordination and from persistence and AI processing, while subsystem boundaries clarify ownership of authentication, documents, search, conversations, and governance. Technology choices favour a pragmatic TypeScript stack that supports streaming answers, hybrid retrieval, and manageable deployment for a prototype-scale enterprise system. Sprint-scoped interaction diagrams complete the picture by showing how real user journeys unfold across the web application, API services, and background workers. Together, these design artefacts provide an implementable blueprint whose adequacy is verified in Chapters ?? and ??.

CHAPTER 4

Realization, implementation or experimentation

This chapter explains how the sprint-aligned design of Section ?? (Chapter ??) was turned into working software. The goal was to deliver the five increments of Table ?? in order, each building on the previous one. The approach was to implement each sprint against its design subsection, validate the principal flows, and embed representative UI screenshots where they add clarity. The outcome is a monorepo with a secured API, a document library with asynchronous ingest, hybrid RAG, governance consoles, and operational tooling described in the sections below.

4.1 Hardware and Software Environment

Development was carried out on a Windows 10/11 workstation with Node.js 20+ as the primary runtime. The application is organised as a monorepo with separate API and web packages, orchestrated from the repository root for local development, database migration, seeding, linting, and RAG evaluation. Table ?? summarises the languages, frameworks, data services, and tooling used in implementation.

Table 4.1 – Development and deployment stack (detailed)

Category	Technology and role	Version
Workstation	Windows 10/11; Git; Docker Desktop; VS Code / Cursor IDE	—
Languages	TypeScript (strict typing across API and web); JavaScript (ES modules, Node runtime); SQL (PostgreSQL migrations and analytics queries)	5.9
Web frame-works	Next.js 15 (App Router, SSR/CSR hybrid, route handlers); React 19 (client components, hooks); CSS Modules + design tokens (light/dark themes)	15 / 19
API layer	Express 4 REST API; tsx dev runner; JWT + HttpOnly refresh cookies; Multer uploads; Helmet / CORS /rate limiting	4.21
Data access	Prisma 5 ORM (29 migrations, seed script); Zod request validation; bcryptjs password hashing	5.22
Persistence	PostgreSQL 16 with pgvector (768-d embeddings); full-text search indexes for hybrid retrieval	16
Async pro-cessing	Redis 7 job broker; BullMQ 5 ingest worker queue (text extraction, chunking, embedding)	7 / 5
AI services	Google Gemini — <code>gemini-2.5-flash</code> (generation/rerank), <code>gemini-embedding-001</code> (vectorisation); <code>@google/generative-ai</code> SDK	API
Deployment	Docker Compose (dev + production compose files); multi-stage Node 22 Alpine images for API and web	Compose
Quality assur-ance	Vitest 3 + Supertest integration tests (74 cases); ESLint + Prettier ; RAG evaluation CLI	3.2

4.2 Implementation Steps by Sprint

4.2.1 Sprint 1: Authentication and access control

The goal of Sprint 1 was to establish a secure perimeter before any document or AI feature could be exposed. The approach followed Subsection ??: short-lived access tokens kept in memory on the web client (never in `localStorage`), refresh tokens in `HttpOnly` cookies, a single in-flight refresh to avoid parallel rotations, and middleware that enforces roles plus per-user restriction flags for document library and Ask access. Profile avatars are served from a CORP-aware public route so browsers can embed them across origins. The outcome is a complete sign-in, recovery, profile, and role-based navigation shell, with integration tests confirming flows ??–??, ??, ??, and ??.

4.2.1.1 Implemented web interfaces

The Sprint 1 web client delivers credential-based sign-in, password recovery, profile management, and role-based home hubs. Error and restriction states are validated manually in Section ??.

Authentication pages.

The web client implements credential-based sign-in and password recovery as static Next.js routes. Figure ?? shows the production sign-in layout; Figure ?? shows the recovery workflow.

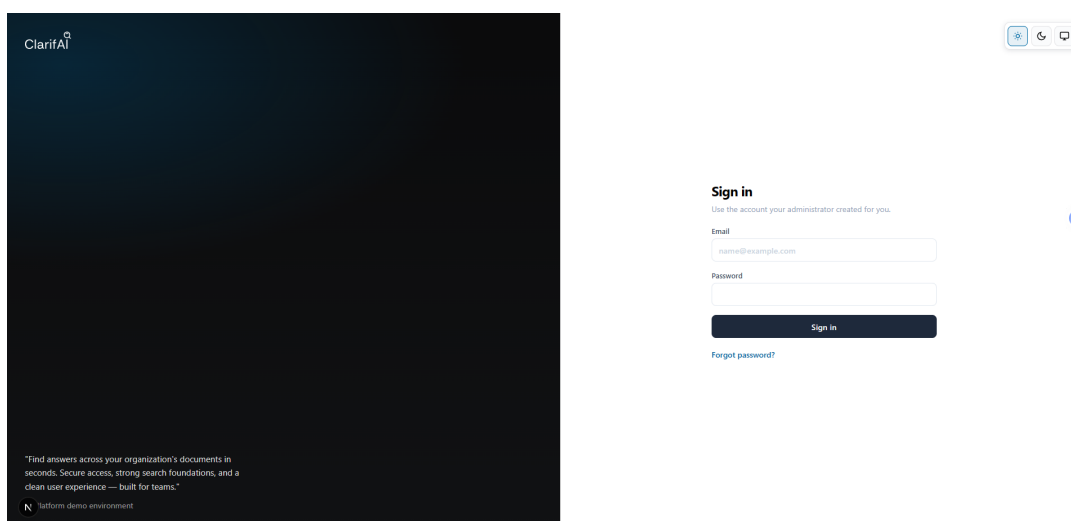


Figure 4.1 – ClarifAI sign-in page with email, password, and theme toggle

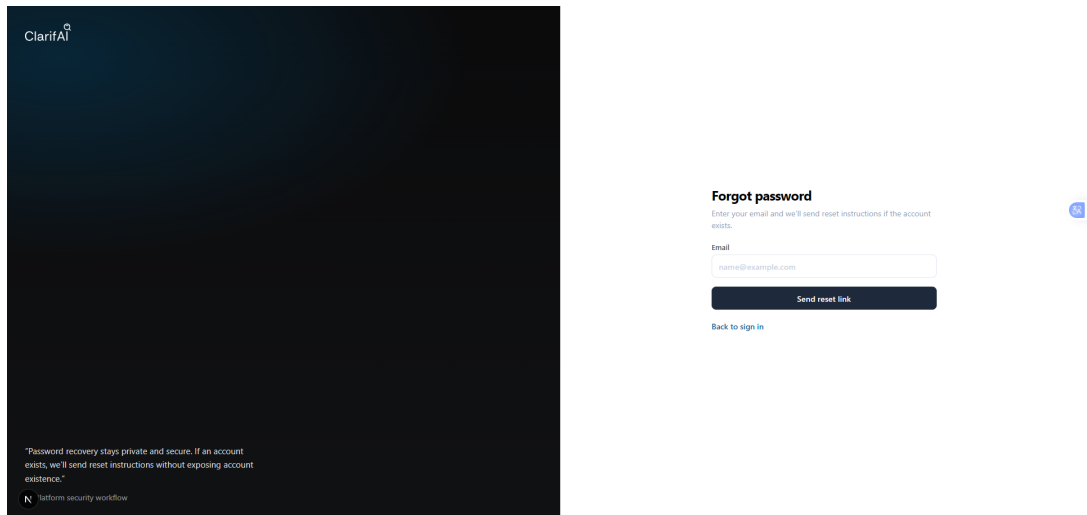


Figure 4.2 – Password recovery request form with email field and submit action

Profile management.

Figure ?? shows the profile settings page: avatar upload, editable contact fields, read-only role and department (set by administrators), and inline change-password controls.

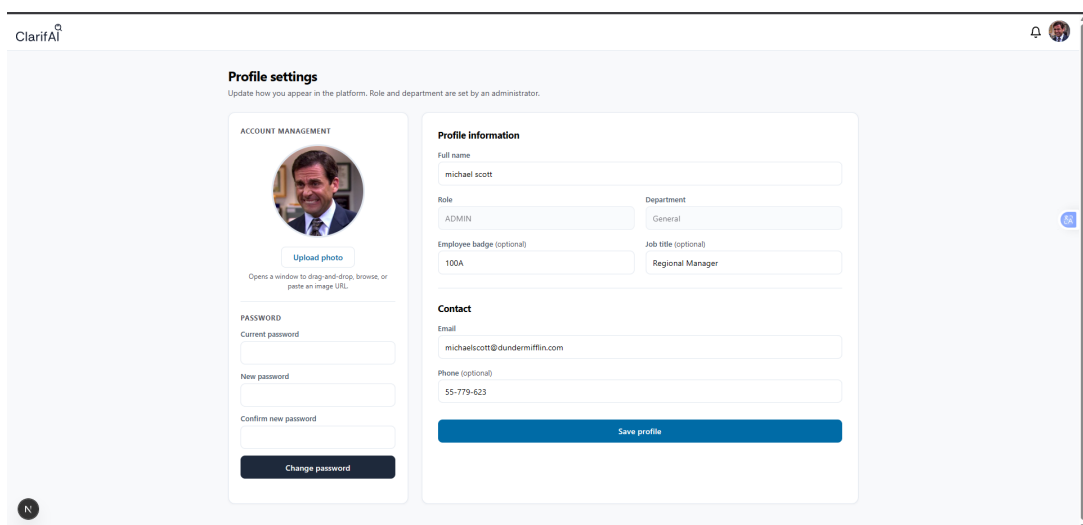


Figure 4.3 – Profile settings with avatar upload, read-only role and department, and change-password form

Role-based home hubs.

After authentication, users land on a tile-based hub whose options depend on platform role. Employees see Documents and Ask (Figure ??); managers additionally see Department overview (Figure ??); administrators see the full four-tile hub including

Administration (Figure ??).

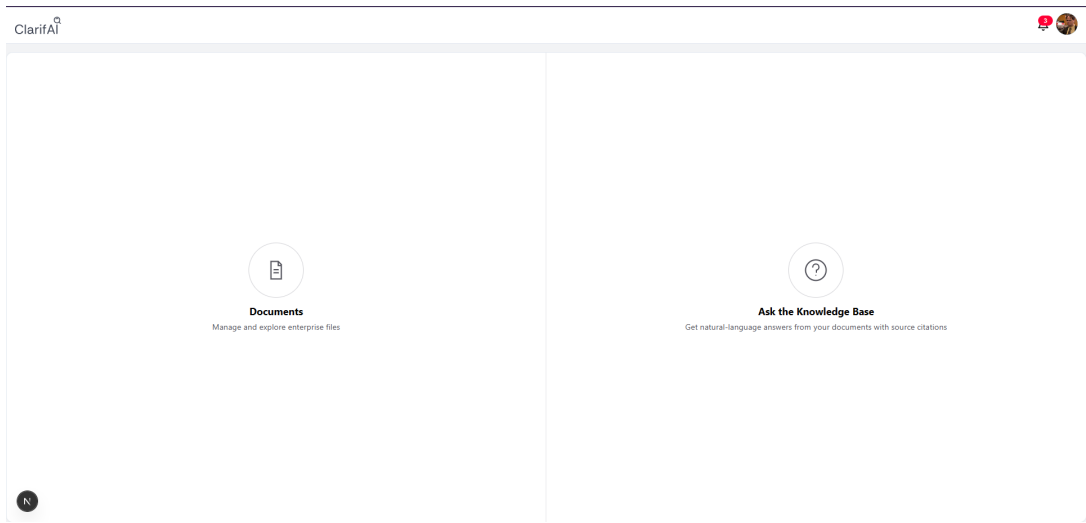


Figure 4.4 – Employee landing page with Documents and Ask tiles

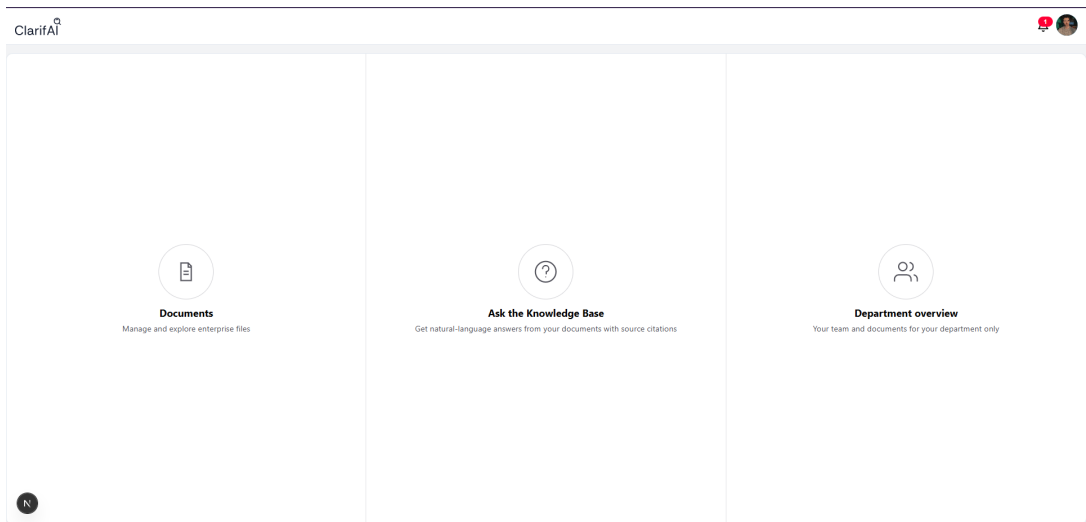


Figure 4.5 – Manager landing page with Department overview tile

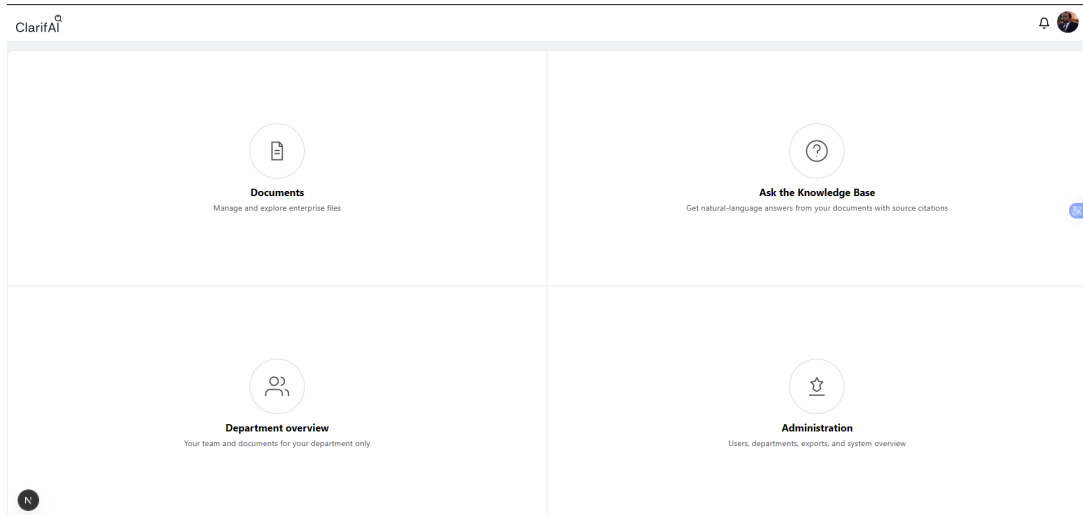


Figure 4.6 – Administrator landing page with Administration tile

Theme switching.

Figure ?? shows the same administrator hub rendered in dark mode, confirming that the CSS theme tokens apply consistently across navigation tiles.

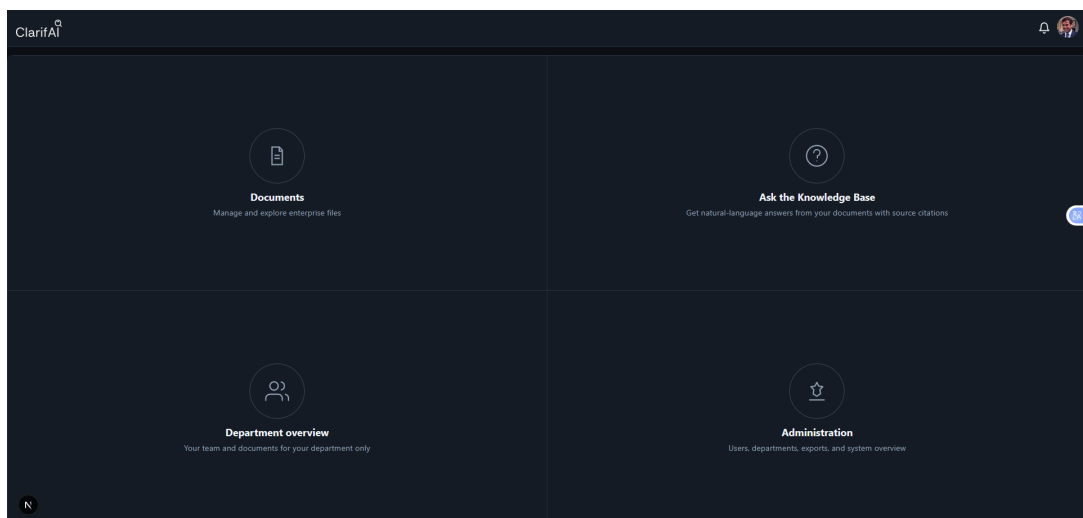


Figure 4.7 – Administrator home hub in dark mode showing consistent theme tokens

4.2.2 Sprint 2: Document library and ingest pipeline

Sprint 2 aimed to make organisational knowledge uploadable, searchable, and governed by visibility rules. Uploads pass MIME and size checks, are stored on disk, and enqueued for asynchronous processing. The worker extracts text per format, chunks content with sentence-aware splitting and table preservation, computes 768-dimensional embeddings,

and persists indexed chunks. Visibility (ALL, DEPARTMENT, PRIVATE) is enforced so unauthorised callers receive a neutral 404 rather than revealing document existence. Tags, favorites, recents, and versioning complete the library experience. Integration tests cover document flows ??–?? and ??–??.

4.2.2.1 Implemented web interfaces

The document library lets users browse department-scoped collections, inspect file details, preview documents in the browser, and manage versions.

Library browse and file details.

Figure ?? shows the department card view with sidebar navigation (Recent, Favorites, Archived) and filter controls. Figure ?? shows the file detail side panel with ingest status, visibility, and action buttons (Open, Download, Favorite, Versions).

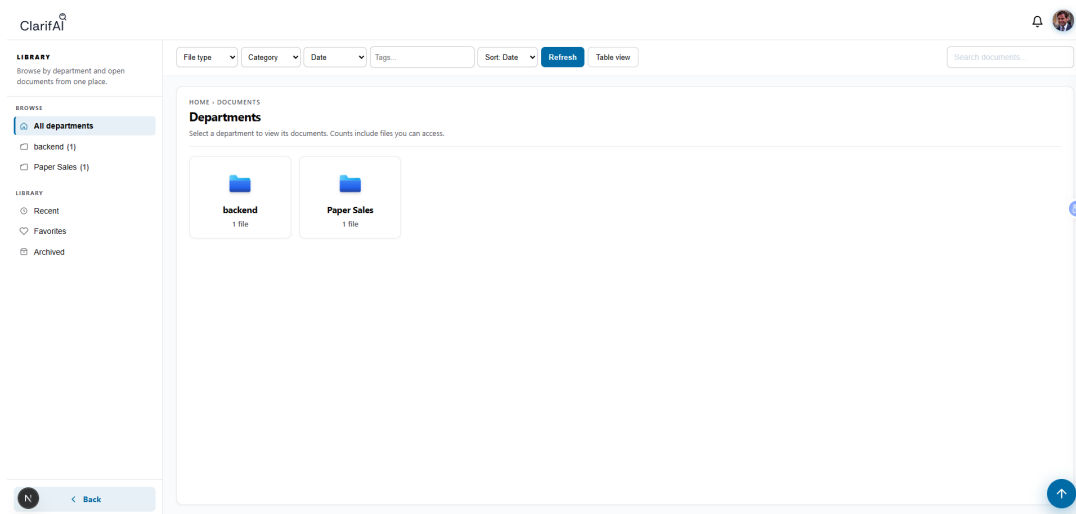


Figure 4.8 – Department library browse with sidebar navigation and filter controls

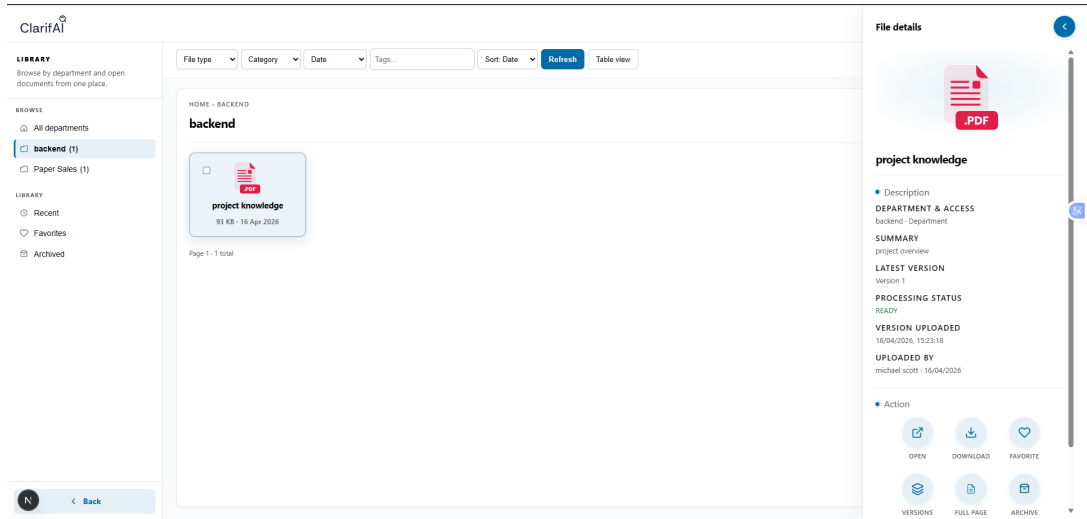


Figure 4.9 – Document detail side panel with metadata and action buttons

Upload workflow.

Managers and administrators open a multi-step upload modal (file, metadata, tags) from the library FAB (Figure ??), matching the upload sequence of Figure ??.

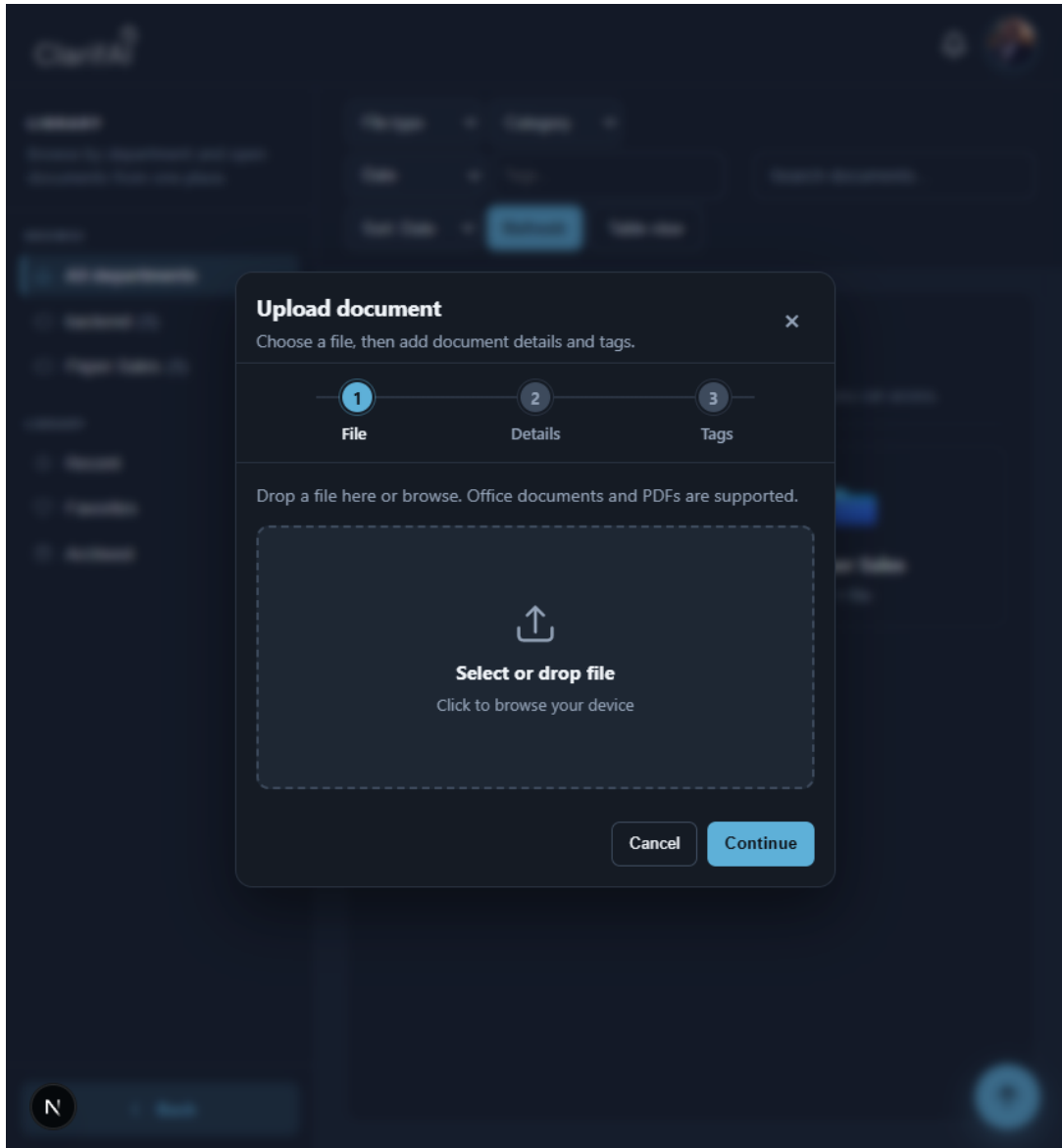


Figure 4.10 – Multi-step upload modal with file drop zone and metadata fields

Recent, favorites, and table view.

Sidebar scopes filter the library to recently opened files (Figure ??) or starred favorites (Figure ??). Users can switch between card and tabular layouts (Figure ??).

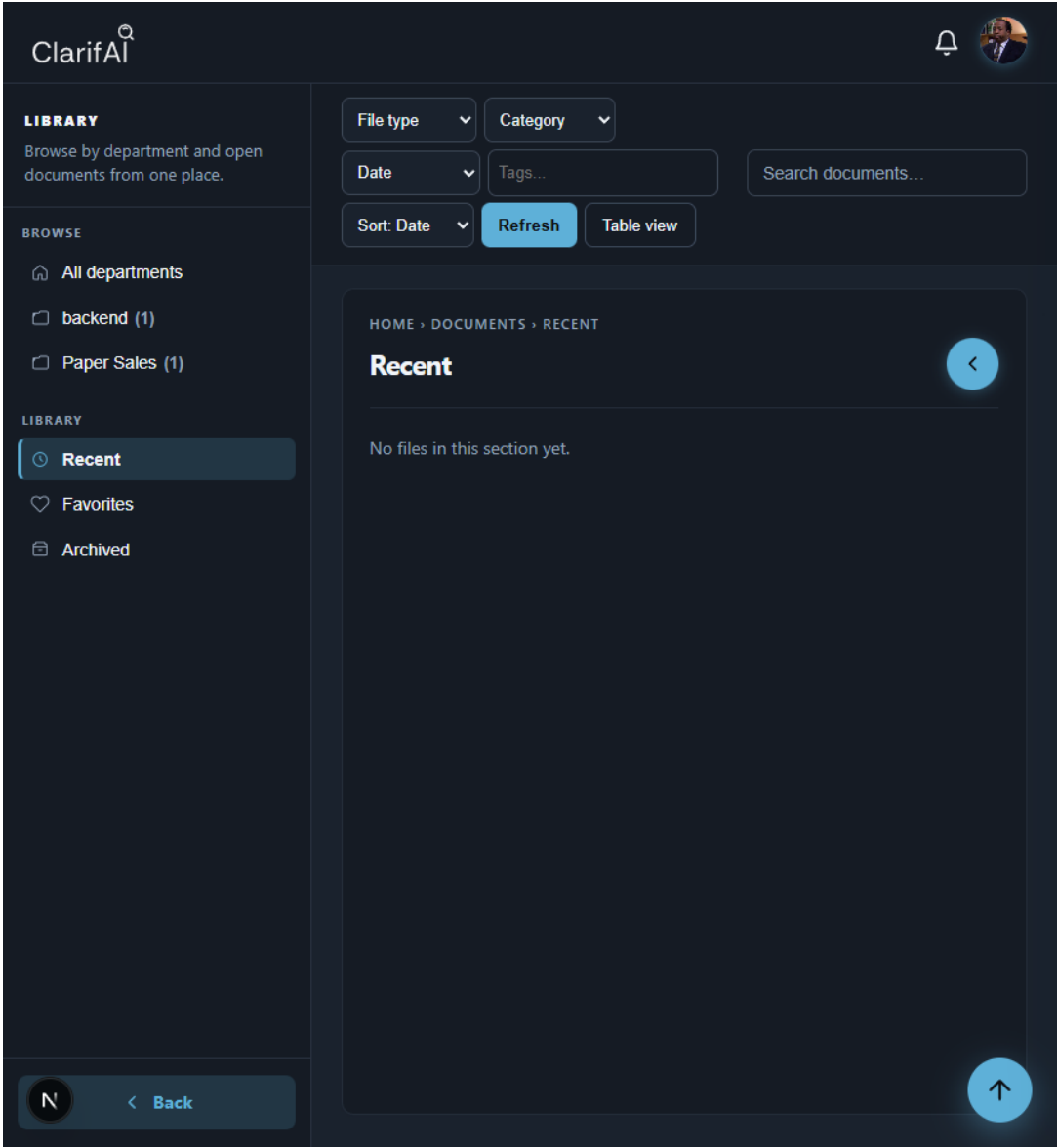


Figure 4.11 – Recent documents scope with sidebar highlight

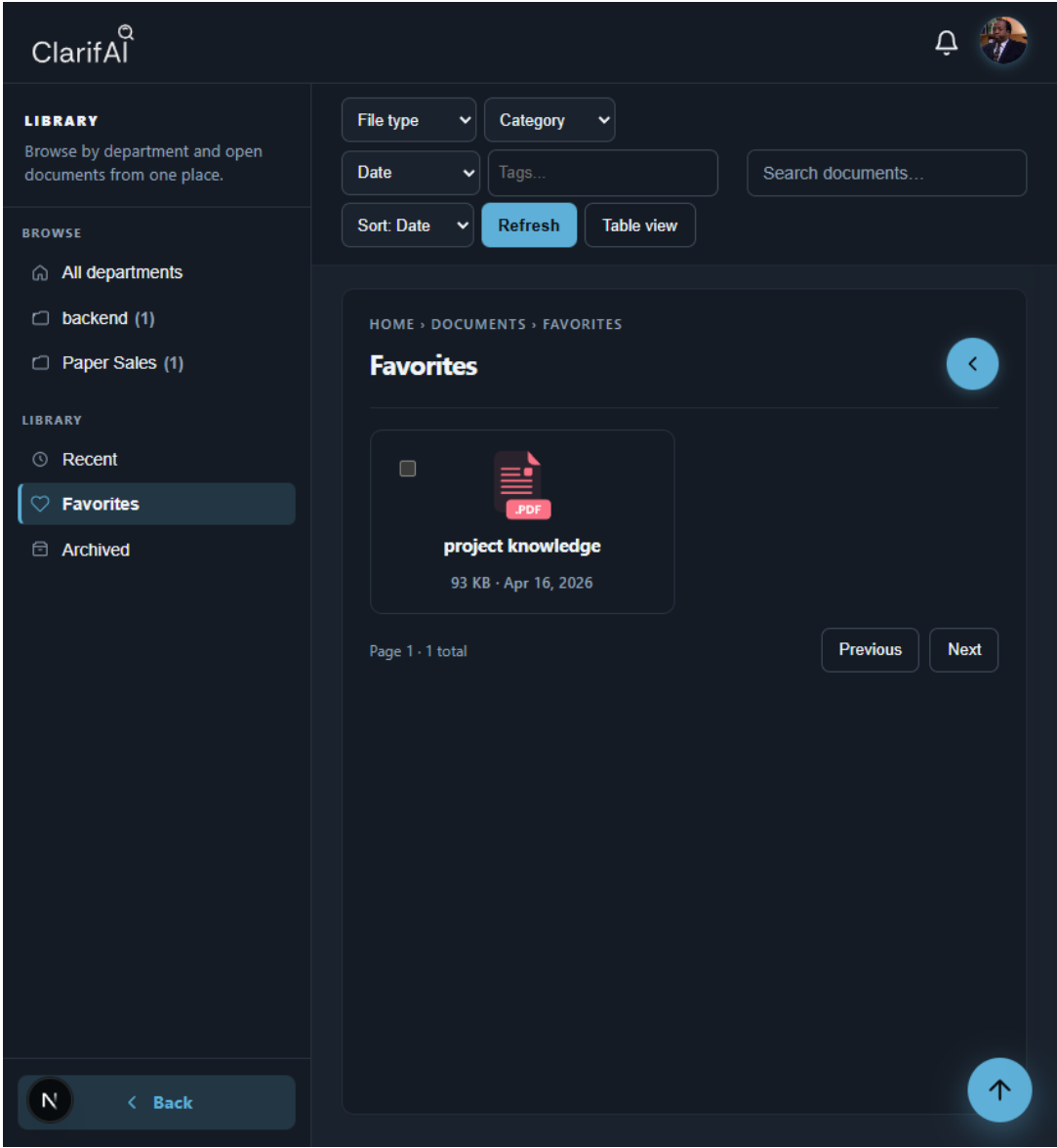


Figure 4.12 – Favorites scope listing starred documents

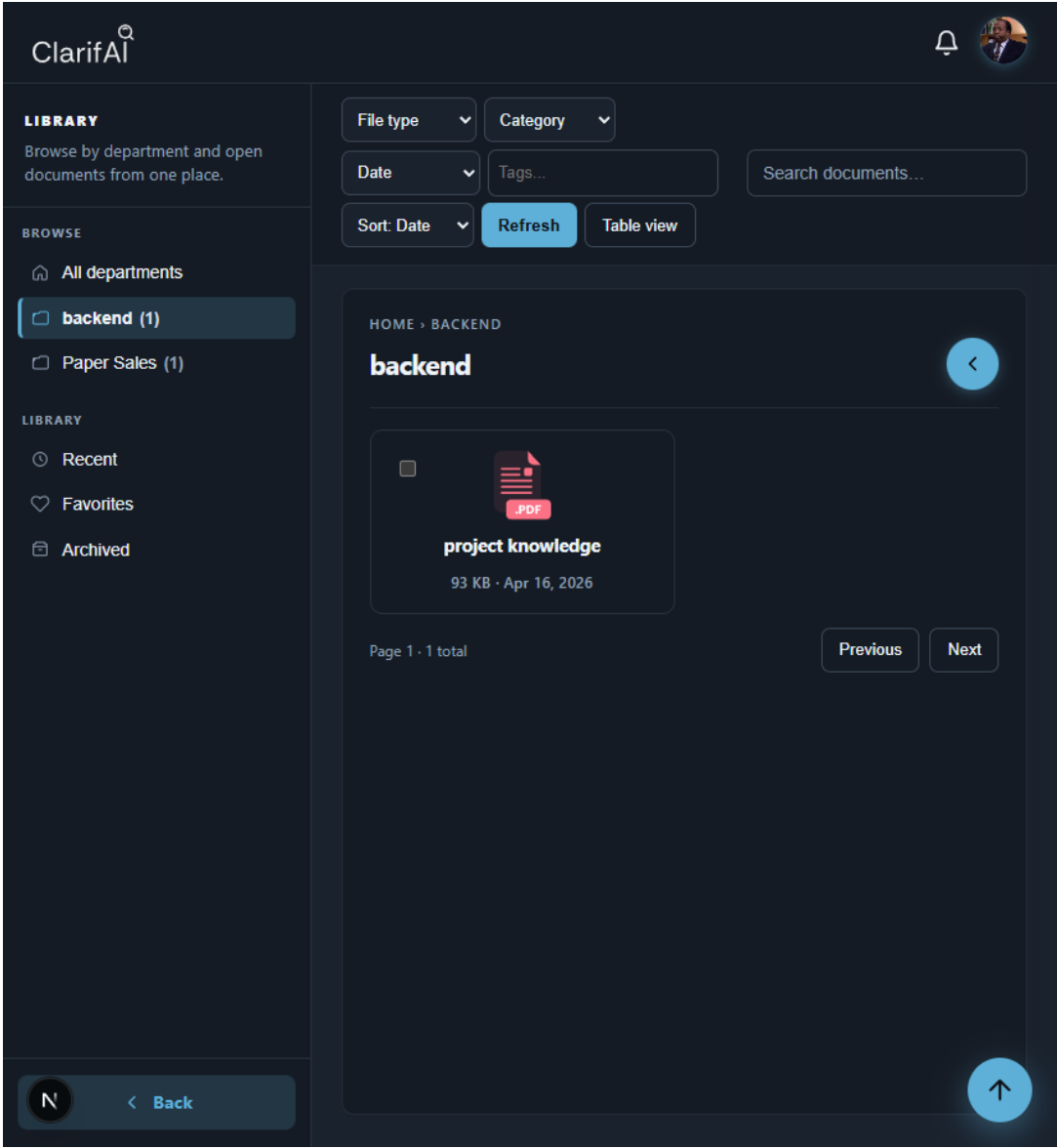


Figure 4.13 – Department library in table layout with column filters

Ingest processing status.

The detail panel surfaces asynchronous worker progress via a **Processing** badge and progress bar before the document reaches **READY** (validated end state in Chapter ??).

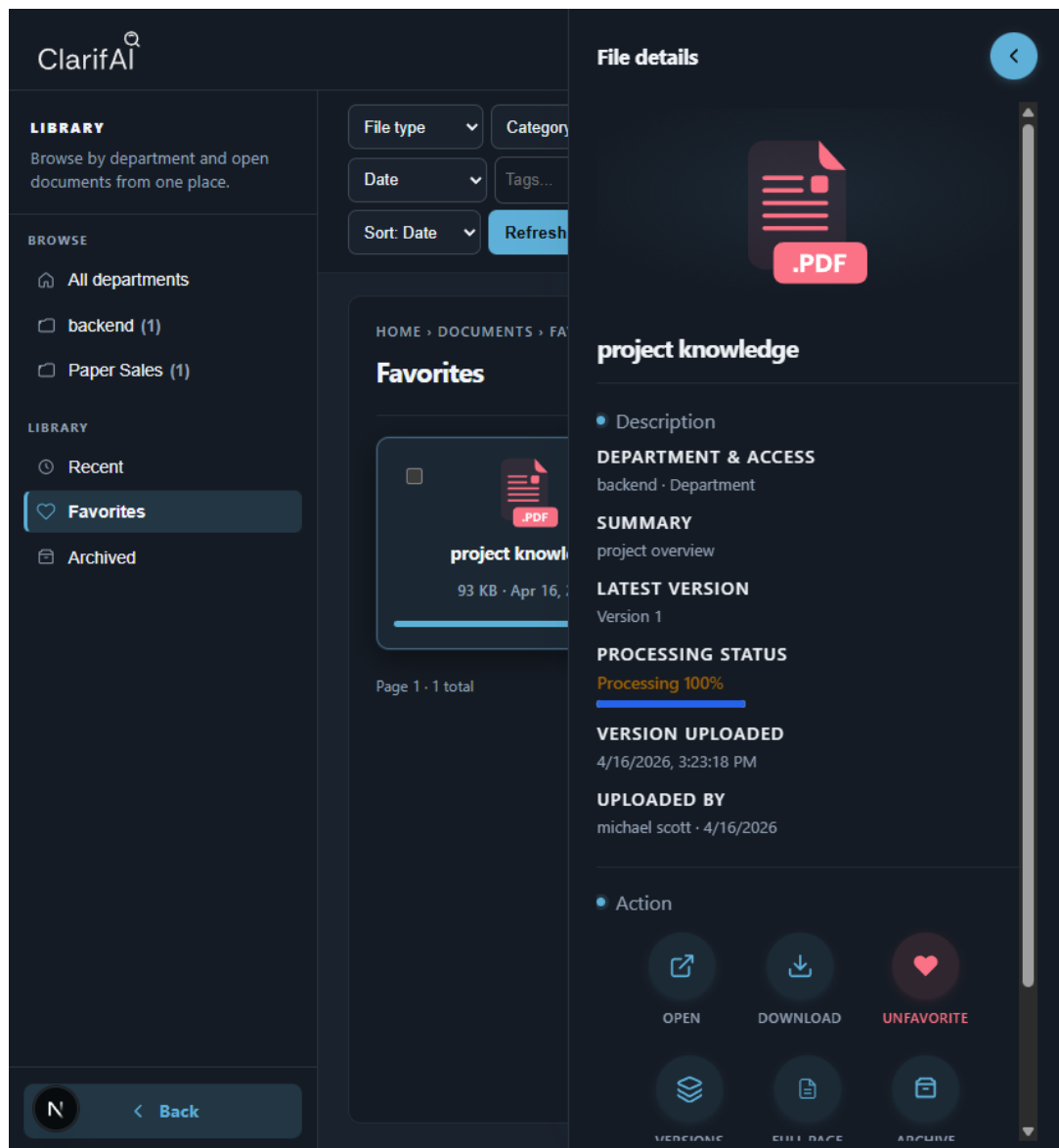


Figure 4.14 – File detail panel during ingest with processing badge and progress bar

Preview and versioning.

Authorized users open PDFs in an in-browser preview modal (Figure ??). Managers and administrators can upload new versions and inspect the version archive from the Versions action (Figure ??).

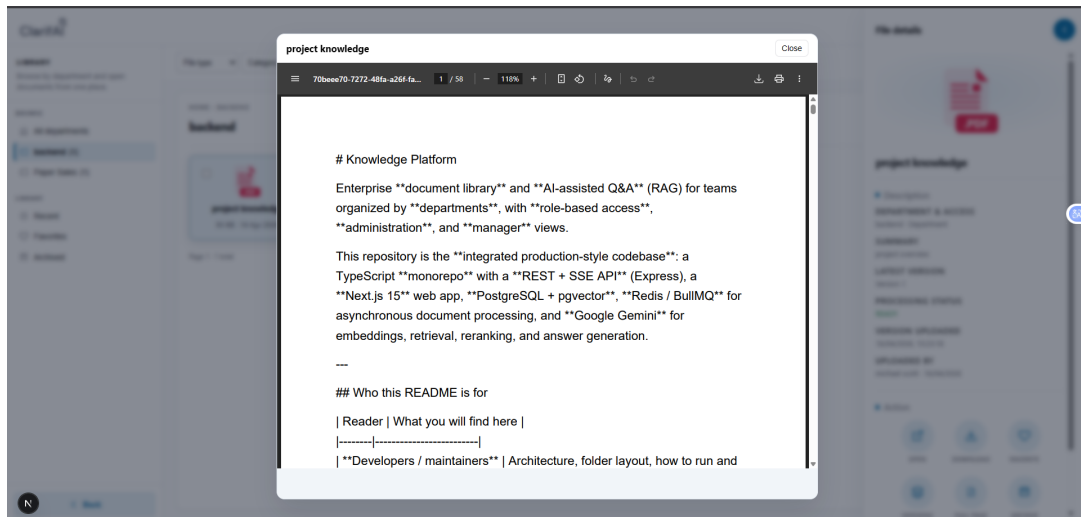


Figure 4.15 – In-app PDF preview modal with navigation controls

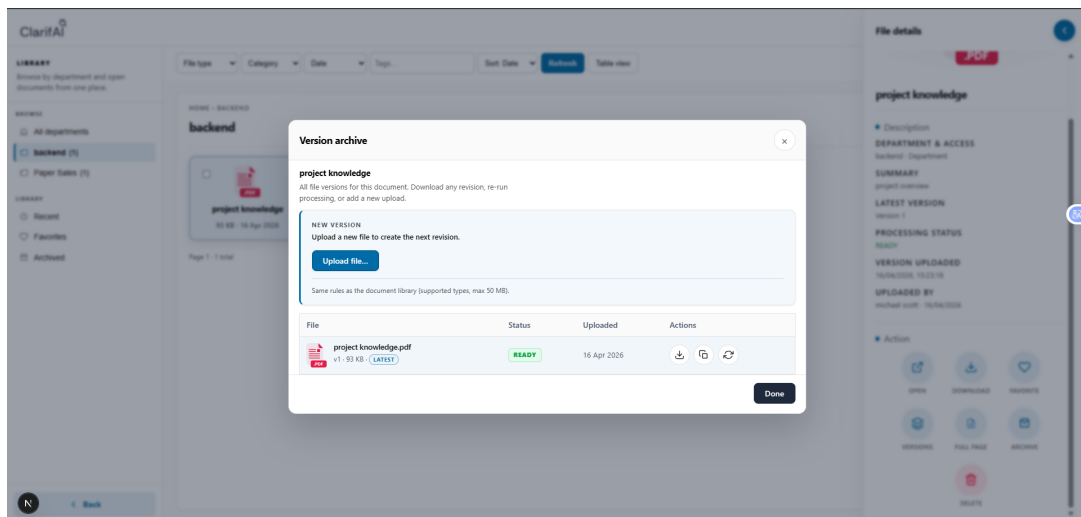


Figure 4.16 – Version archive dialog with upload and reprocess actions

4.2.3 Sprint 3: Search, RAG and conversations

The goal of Sprint 3 was to let authorised users find information semantically and receive cited, streaming answers grounded in their accessible corpus. Hybrid retrieval combines dense vectors and full-text search, fuses rankings, reranks with Gemini, and streams results over SSE. Algorithm ?? summarizes the Ask endpoint logic.

Algorithm 1 Hybrid RAG Ask pipeline (simplified)

- 1: Authenticate user and resolve `readableDepartmentIds`
- 2: Optimize query (type, topic, rewrite, optional multi-hop)
- 3: Retrieve dense vectors (pgvector) and sparse FTS (BM25-style)
- 4: Fuse rankings with weighted reciprocal rank fusion (RRF)
- 5: Rerank top chunks with Gemini; optionally expand neighbor chunks
- 6: Stream SSE events: `sources`, `token`, optional `correction`
- 7: Persist conversation message with citations and confidence
- 8: Optionally cache answer keyed by prompt version and filters

The Ask UI consumes SSE streams and renders markdown with sanitized links. Conversation threads, message persistence, and thumbs-up/down feedback round out the assistant experience. Search, Ask, and conversation flows (??-??, ??, ??) were validated by automated integration tests.

4.2.3.1 Implemented web interfaces

Ask interface.

The Ask workspace streams structured answers with confidence indicators and clickable source citations grounded in retrieved chunks (Figure ??, Figure ??).

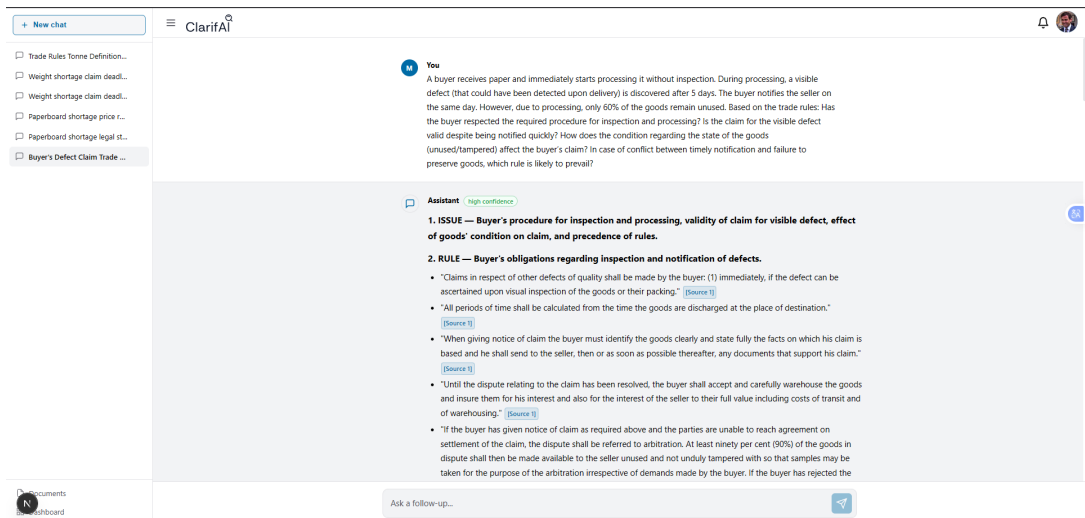


Figure 4.17 – Ask chat interface with structured answer, confidence indicator, and source citations

Answer feedback.

Each completed assistant message exposes copy, thumbs-up, and thumbs-down controls; negative feedback optionally captures a free-text comment for export and RAG evaluation (Figure ??, Figure ??).

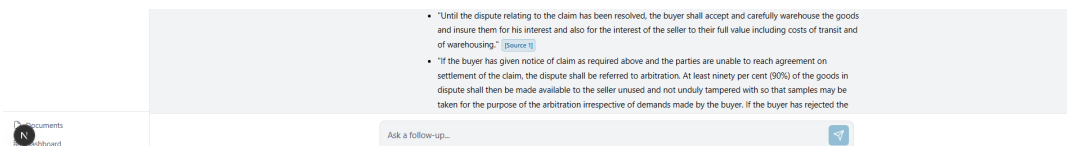


Figure 4.18 – Per-answer feedback controls with copy, thumbs up/down, and grounding badge

4.2.4 Sprint 4: Governance and notifications

Sprint 4 addressed organisational governance: administrators manage users, departments, and audit trails; managers oversee department-scoped activity; and all roles receive timely notifications. Department access logic computes readable and manageable scopes including inherited manager and viewer rights. Administrators can bulk-restrict users, import CSV rosters with formula-injection escaping, merge departments, export KPIs, and review document audit logs. Managers access department dashboards; the notification service emits automatic events on document lifecycle changes and supports manual announcements with attachments. Document previews use blob URLs for PDFs and images; the inbox polls for unread badges. Admin, manager, and notification flows (??–??, ??–??) were confirmed by integration tests.

4.2.4.1 Implemented web interfaces

Governance consoles cover notifications, manager department overviews, administration modules, and audit dashboards. The screenshots below illustrate each area.

Notifications.

Users receive announcements in a slide-out inbox (Figure ??). Administrators and managers compose manual notifications through a modal form with recipient scope and optional attachments (Figure ??).



Figure 4.19 – Notifications inbox drawer listing unread announcements

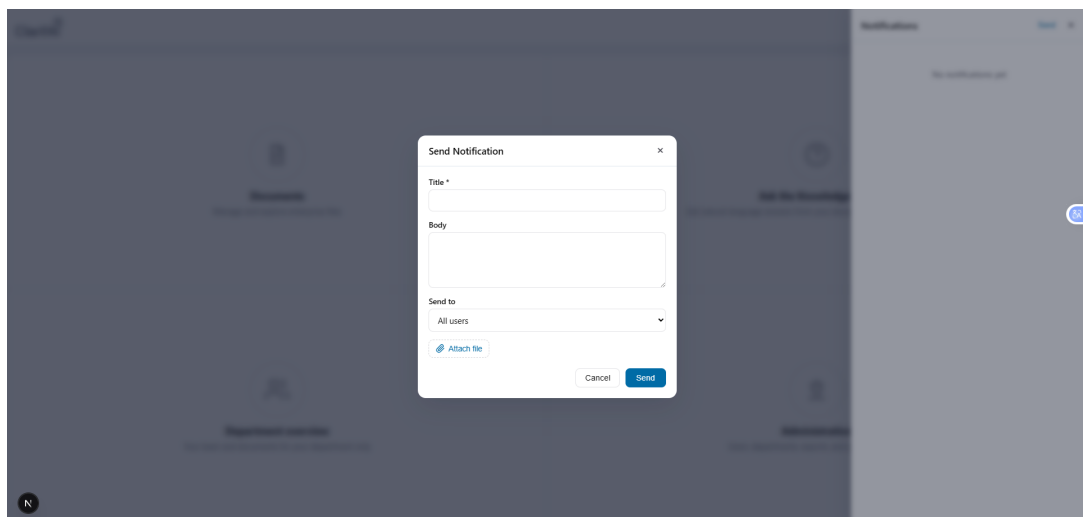


Figure 4.20 – Send notification modal with title, body, and recipient scope

Manager dashboard.

Figure ?? shows the department overview: team directory, department-scoped document list, and ingest status for manageable departments.

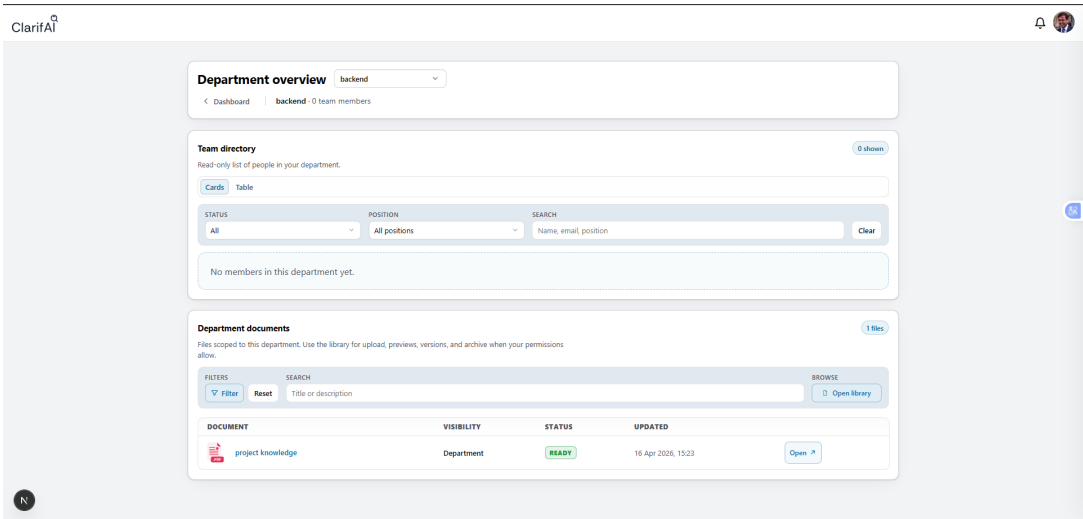


Figure 4.21 – Manager department overview with team directory and document list

Administration hub and directories.

The administration landing page (Figure ??) routes to six modules. Figure ?? shows the user directory with role, department, lock state, and bulk import. Figure ?? shows department cards with hierarchy and merge tool. Figure ?? shows the admin document library with processing filters and bulk delete.

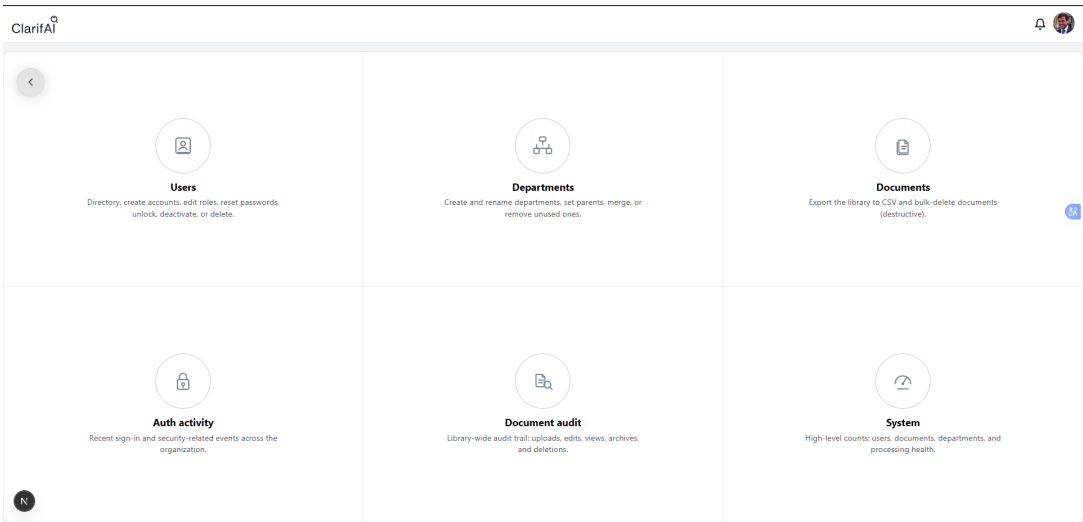


Figure 4.22 – Administration hub with Users, Departments, Documents, Auth activity, Document audit, and System modules

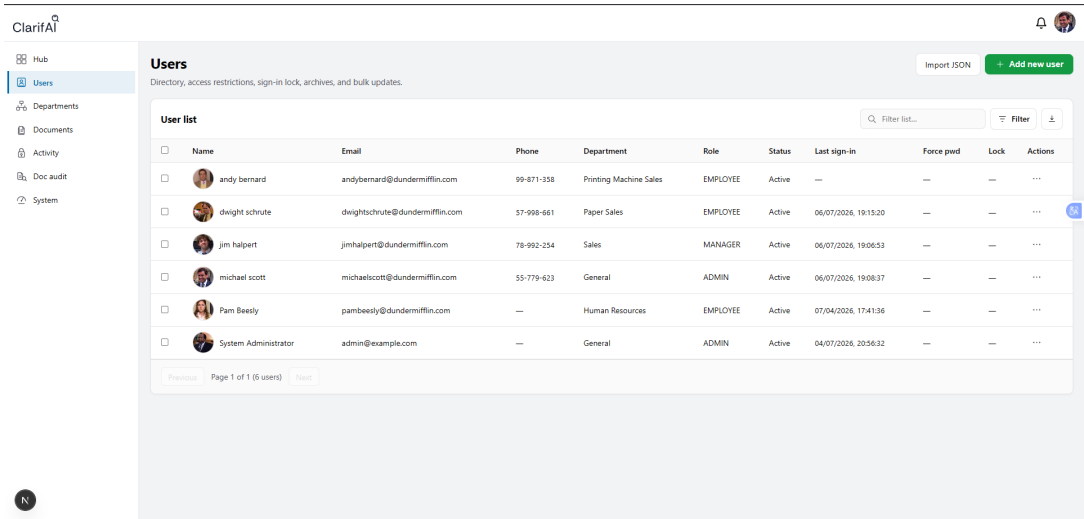


Figure 4.23 – Administrator user directory with roles, restrictions, and lock controls

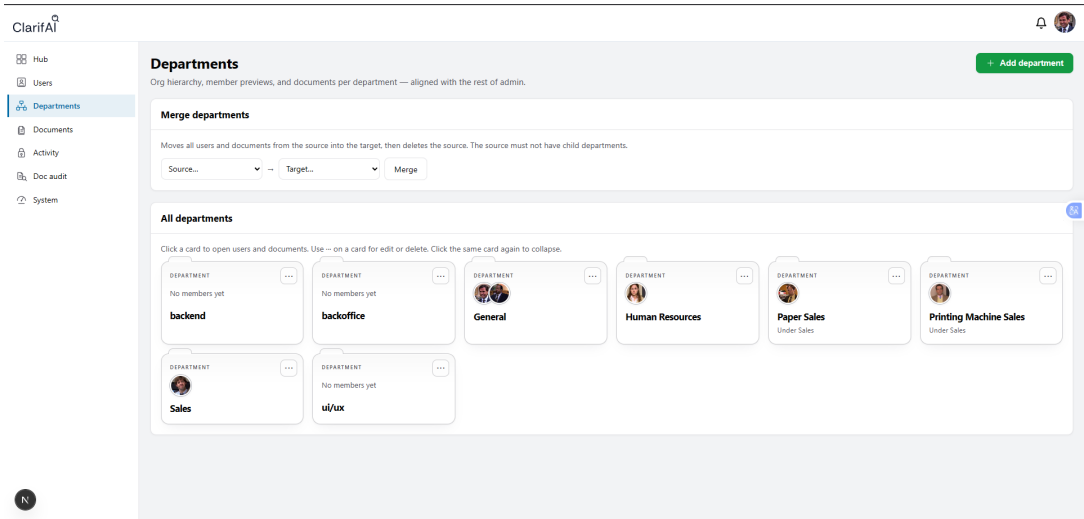


Figure 4.24 – Department management with hierarchy tree and merge tool

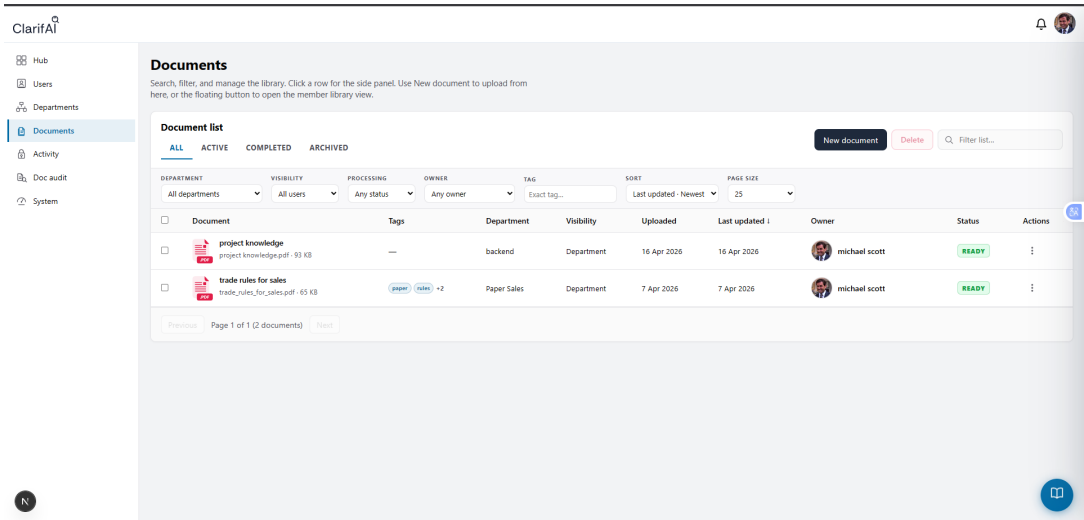


Figure 4.25 – Administrator document library with status filters and bulk operations

Audit trails and system KPIs.

Figure ?? shows the library-wide document audit log. Figure ?? shows authentication and security events with IP and client metadata. Figure ?? shows the system overview dashboard with period-over-period user, document, and department counts.

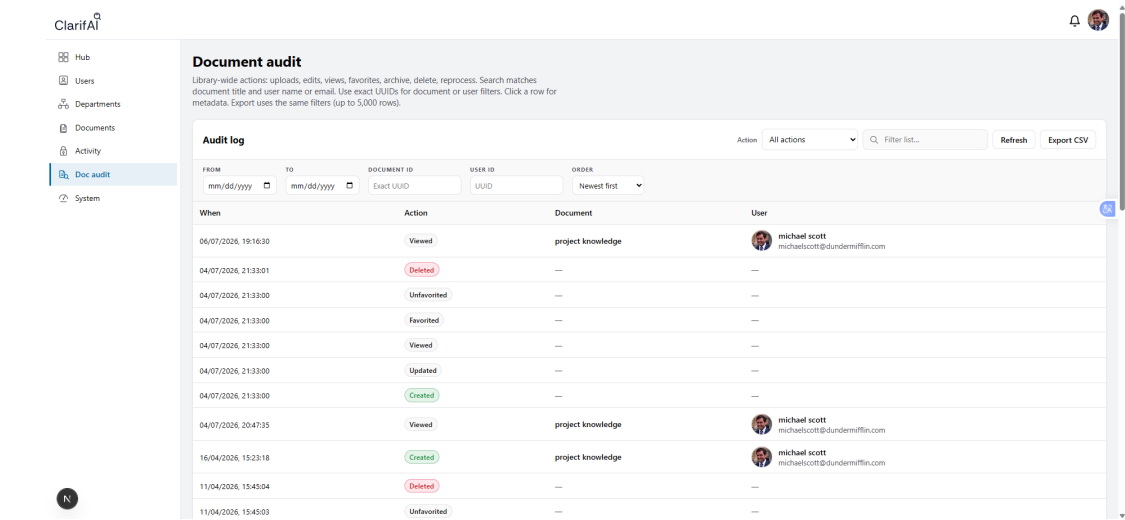


Figure 4.26 – Library-wide document audit log with CSV export action

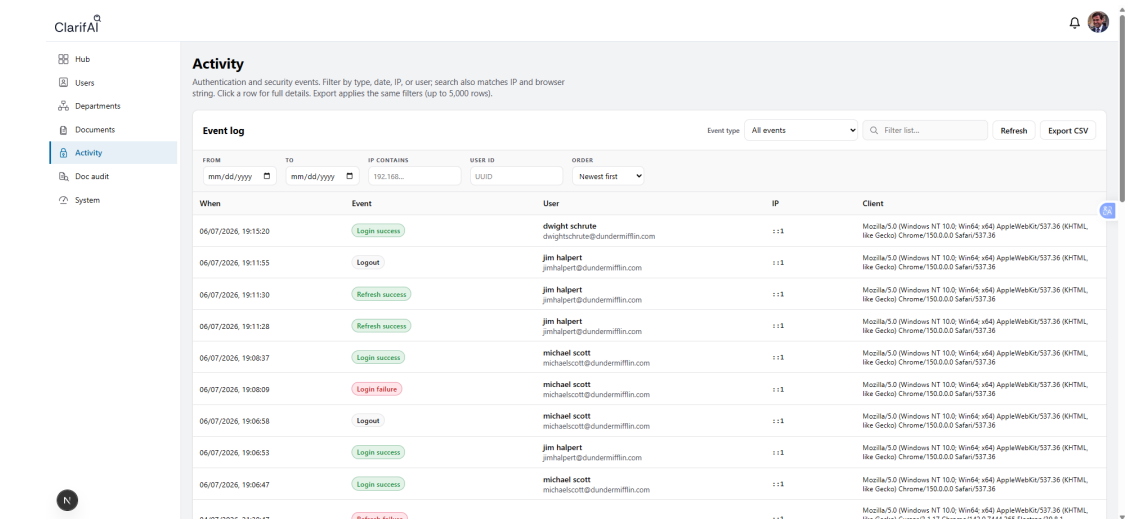


Figure 4.27 – Authentication and security event log with IP and client metadata

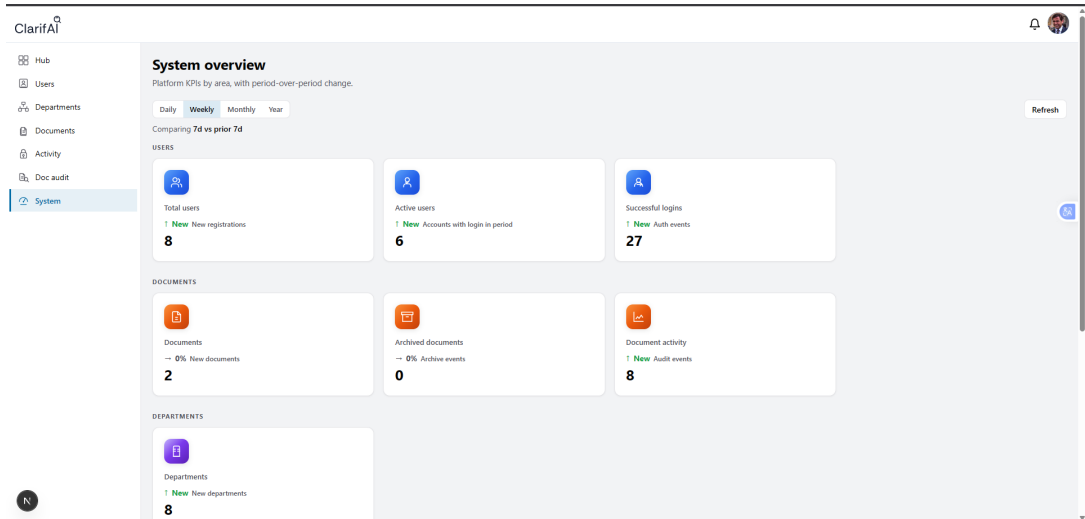


Figure 4.28 – System overview KPI dashboard with period-over-period comparison

4.2.5 Sprint 5: Operations, testing and deployment

Sprint 5 consolidated operational readiness: health probes report PostgreSQL and Redis status, a 74-test integration suite exercises the API against real services, Docker Compose provides reproducible dev and production stacks with automatic migrations, and a RAG evaluation CLI writes JSON reports for regression tracking. Operational readiness has no dedicated web UI surface; validation evidence is tabular in Chapter ??.

Table ?? summarises all sprint sequence flows presented in Section ??.

Table 4.2 – Index of implementation sequence diagrams (Chapter ??, §??)

Sprint	Area	Fig.	Flow
S1	Auth	??– ??, ??, ??, ??	Login, refresh, logout, password reset/change, profile, restrictions, /me, register info, restricted UX
S2	Documents	??– ??, ??– ??	Upload, ingest worker, download, list, detail, favorites, metadata, versions, reprocess
S3	Search / RAG	??, ??	Semantic search, hybrid Ask SSE
S3	Conversations	??, ??, ??, ??	Threads, messages, feedback, CRUD supplement, admin stats
S4	Notifications	??– ??	Automatic events, manual send, inbox
S4	Manager	??	Department dashboard
S4	Admin	??– ??, ??– ??	Users, departments, access, stats, audit, exports, bulk delete, lock
S5	Operations	??	Health check (tests, deploy, RAG eval: see §??)

4.3 Difficulties Encountered and Solutions

Table 4.3 – Implementation challenges and resolutions

Difficulty	What went wrong	How it was resolved
Avatar not displayed	Browsers blocked cross-origin profile images because the avatar route lacked a permissive CORP header.	The API now allows cross-origin embedding for public avatar responses.
Random logout on navigation	Parallel refresh requests rotated the refresh token twice, invalidating the session.	The web client serialises refresh into a single in-flight promise so only one rotation occurs.
Next.js dev webpack errors with avatars	The optimised image component correlated with RSC bundling issues in development.	Profile avatars use a standard HTML image element instead of the framework image wrapper.
Gemini rate limits	External API throttling caused intermittent failures under load.	Answers are cached with TTL; retries use exponential backoff.
Spreadsheet chunking	Tables split mid-row lost row semantics in retrieved chunks.	Chunking preserves table structure as markdown before embedding.

4.4 Produced Technical Deliverables

The project deliverables include:

- **Source code** — full-stack monorepo with API and web applications;
- **Database** — relational schema with migrations and seeded demonstration data;
- **Documentation** — README, architecture guide, and **54 PlantUML diagrams**

(sequence, use-case, and sprint class flows in Chapter ??, Section ??);

- **Tests** — ten integration test suites covering 74 scenarios;
- **Operations** — containerised deployment, RAG evaluation tooling, and maintenance scripts;
- **User interface** — dashboard, document library, Ask workspace, and administrative consoles (Figures ??–??, ??–?? in §??–??).

Representative running-application screenshots are embedded per sprint above; authentication error and access-restriction validation evidence appears in Section ??.

4.5 Chapter Conclusion

This chapter described the concrete realization of the platform across five sprint increments, from authentication and document ingest through hybrid RAG, governance, and operational readiness. It documented implementation constraints and technical resolutions, showing how architectural choices were translated into operational code, testable workflows, and reproducible deliverables. Chapter ?? presents the validation results against the KPIs defined in Table ??.

CHAPTER 5

Testing, validation and discussion

This chapter evaluates whether the implementation described in Chapter ?? satisfies the functional, non-functional, and KPI targets defined in Chapter ?? (Table ??). Validation combines automated API tests, RAG evaluation harness runs, and manual browser scenarios. The level of confidence attached to each result class is discussed in Section ??.

5.1 Validation plan

Validation was structured at four complementary levels. Pure helper functions were checked with unit tests. The API was exercised end-to-end through integration tests against a live PostgreSQL and Redis stack. Browser scenarios on the Next.js client covered flows that are difficult to assert purely at the HTTP layer. Finally, a dedicated RAG evaluation harness measured retrieval and answer quality with automated judge metrics where an API key was available.

Relative to the solutions surveyed in Chapter ??, this project validates capabilities that generic document stores or chatbots typically lack: **department-scoped hybrid retrieval**, **cited streaming answers**, **per-user feature restrictions**, and **administrative audit trails**. Unit coverage is intentionally narrow; the primary automated quality gate is the **74-test integration suite**, supplemented by manual UX checks for security and restriction flows.

5.2 Protocols and experimental conditions

Table ?? summarises the experimental environment. Integration tests run against a seeded database with Docker-hosted PostgreSQL and Redis. RAG evaluation uses a quick mode (optimizer-only, 34 cases) and a full judge pipeline when a Gemini API key is configured. Manual scenarios were executed in the browser on the local development server and are listed in Table ?? with screenshot evidence in Section ??.

Table 5.1 – Experimental setup for validation

Parameter	Value
Workstation	Windows 10/11; Node.js 20+
Test framework	Vitest 3.2 + Supertest (<code>npm run test:api</code>)
Database / queue	PostgreSQL 16 + pgvector; Redis 7 (Docker Compose)
Test corpus	Seeded users, departments, and documents (<code>npm run db:seed</code>)
RAG evaluation	<code>eval:rag:quick</code> (34 topic cases); full judge if API key configured
Automated run date	4 July 2026 (74/74 passed, duration ≈ 39 s)
Manual UX	Browser on <code>localhost:3000</code> ; scenarios T1–T5 (Table ??)

5.3 Presentation of Results

5.3.1 KPI validation results

Table ?? maps each KPI from Chapter ?? to its measured outcome and validation conclusion.

Table 5.2 – KPI validation results

KPI	Measurement method	Target	Obtained	Status
Ingest reliability	Uploads reaching ready state without failure	$\geq 95\%$	100% (14 doc. tests)	Passed
Integration pass rate	Full API integration suite	74 / 74	74 / 74	Passed
RAG topic (no API key)	Quick eval optimizer pass rate	$\geq 15/34$	15 / 34	Passed
RAG judge (with API key)	Automated faithfulness / relevance judge	Documented	Pending full run	Partial
Security: re-fresh replay	Rotated token revokes family	Auth suite	Pass (11 tests)	Passed
Security: avatar CORP	Cross-origin avatar embedding permitted	Avatars test	Pass (1 test)	Passed

Analysis. Five of six KPI rows passed fully. Ingest reliability exceeds the 95% target on the seeded corpus. The integration suite provides strong evidence that API contracts match the design sequences of Chapter ???. RAG topic classification meets the degraded-mode threshold (15/34) without an external API key; full judge scores remain **partial** pending a complete evaluation run with credentials.

5.3.2 Integration test results by sprint domain

On 4 July 2026, the automated API test suite reported **74 tests passed** across 10 files with zero failures. Table ?? groups results by sprint domain, aligned with Table ??.

Table 5.3 – Validation test summary by sprint domain

Spr.	Objective	Expected	Obtained	Status
S1	Secure login and token refresh	200 + new token	Pass (11 tests)	Passed
S2	Document access control	403/404 when re-stricted	Pass (14 tests)	Passed
S3	Search and Ask restrictions	AI flag enforced	Pass (7 tests)	Passed
S4	Admin RBAC	Non-admin rejected	Pass (18 tests)	Passed
S4	Notifications lifecycle	CRUD + unread count	Pass (8 tests)	Passed
S1/S5	Avatar CORP + health probe	Headers + DB/Redis ok	Pass (4 tests)	Passed

Analysis. Every sprint increment from Table ?? has at least one passing integration-test group, confirming end-to-end traceability from requirements through design to executable verification. The largest groups are S4 administration (18 tests) and S2 document access (14 tests), reflecting the complexity of RBAC and visibility rules. Figure ?? shows the Vitest summary from the 7 July 2026 re-run (74/74, duration ≈ 41 s).

```
● ● ● npm run test:api – Vitest integration suite

> test:api
> npm run test -w @knowledge-platform/api

> @knowledge-platform/api@0.0.1 test
> npx vitest run

RUN v3.2.4 C:/dev/finalproject/apps/api

  src/auth.integration.test.ts (11 tests) 7424ms
  src/admin.integration.test.ts (18 tests) 2449ms
  src/documents.integration.test.ts (14 tests) 1850ms
  src/search.integration.test.ts (7 tests) 1485ms
  src/manager.integration.test.ts (4 tests) 1453ms
  src/conversations.integration.test.ts (7 tests) 847ms
  src/notifications.integration.test.ts (8 tests) 904ms
  src/avatars.integration.test.ts (1 test) 433ms
  src/health.integration.test.ts (3 tests) 63ms
  src/lib/roleNameContract.test.ts (1 test) 2ms

Test Files  10 passed (10)
Tests       74 passed (74)
Start at    16:04:15
Duration    41.20s
```

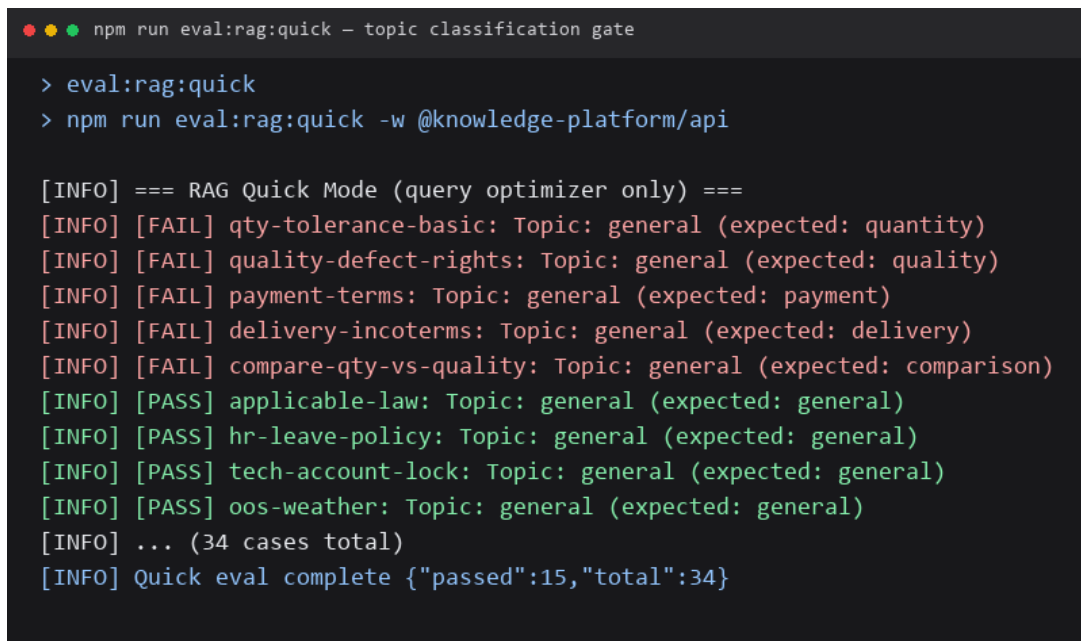
Figure 5.1 – Vitest integration suite output showing 74 tests passed across 10 files

5.3.3 RAG evaluation results

Quick evaluation without a configured Gemini API key classified all optimizer topics as **general**, yielding **15/34 PASS** on topic-label cases—an expected degradation when the LLM optimizer cannot run. With API credentials, the full harness produces JSON reports under `apps/api/eval-reports/` for faithfulness and citation metrics.

Interpretation. Retrieval and serving paths are testable without the external API, but semantic query optimisation and judged answer quality require it. The 15/34 result should be read as a **minimum regression gate**, not as a measure of answer quality in

production. Figure ?? captures the quick-mode harness output on 7 July 2026.

A terminal window with a dark background and light-colored text. The title bar shows three colored dots (red, yellow, green) and the text 'npm run eval:rag:quick - topic classification gate'. The terminal content shows a command prompt followed by 'eval:rag:quick' and 'npm run eval:rag:quick -w @knowledge-platform/api'. The output consists of several lines of log messages. The first line is '[INFO] === RAG Quick Mode (query optimizer only) ==='. The next seven lines are '[INFO] [FAIL] : Topic: general (expected:)' for topics: 'qty-tolerance-basic' (expected: quantity), 'quality-defect-rights' (expected: quality), 'payment-terms' (expected: payment), 'delivery-incoterms' (expected: delivery), 'compare-qty-vs-quality' (expected: comparison), 'applicable-law' (expected: general), and 'hr-leave-policy' (expected: general). The next three lines are '[INFO] [PASS] : Topic: general (expected: general)' for topics: 'tech-account-lock' (expected: general), 'oos-weather' (expected: general), and an ellipsis representing 34 cases total. The final line is '[INFO] Quick eval complete {"passed":15,"total":34}'.

```
● ● ● npm run eval:rag:quick - topic classification gate

> eval:rag:quick
> npm run eval:rag:quick -w @knowledge-platform/api

[INFO] === RAG Quick Mode (query optimizer only) ===
[INFO] [FAIL] qty-tolerance-basic: Topic: general (expected: quantity)
[INFO] [FAIL] quality-defect-rights: Topic: general (expected: quality)
[INFO] [FAIL] payment-terms: Topic: general (expected: payment)
[INFO] [FAIL] delivery-incoterms: Topic: general (expected: delivery)
[INFO] [FAIL] compare-qty-vs-quality: Topic: general (expected: comparison)
[INFO] [PASS] applicable-law: Topic: general (expected: general)
[INFO] [PASS] hr-leave-policy: Topic: general (expected: general)
[INFO] [PASS] tech-account-lock: Topic: general (expected: general)
[INFO] [PASS] oos-weather: Topic: general (expected: general)
[INFO] ... (34 cases total)
[INFO] Quick eval complete {"passed":15,"total":34}
```

Figure 5.2 – RAG quick evaluation output showing 15 of 34 topic cases passed without Gemini API key

5.3.4 Manual UX validation

Table ?? summarises the five manual browser scenarios. Figures below provide visual evidence; implementation counterparts appear in Chapter ?? with complementary captions.

Table 5.4 – Manual UX validation summary

Test	Objective	Expected Outcome	Observed Outcome	Status
T1	Reject invalid credentials without revealing account existence	Generic error message, no enumeration	Fig. ??	Passed
T2	Explain when document library access is disabled	Dedicated restricted page	Fig. ??	Passed
T3	Confirm ingest completes after upload	Processing status shown as complete on detail panel	Fig. ??	Passed
T4	Verify hybrid Ask returns cited answers	Answer with source citations	Fig. ??	Passed
T5	Show manager department overview	Dept-scoped team and documents	Fig. ??	Passed

Authentication failure (T1).

Invalid credentials produce a clear client-side error without revealing whether the email exists, consistent with the auth integration suite (Table ??).

Sign in

Use the account your administrator created for you.

Email

michaelscott@dundermifflin.com

Password

.....

Invalid email or password

Sign in

[Forgot password?](#)

Figure 5.3 – Invalid email or password message after failed sign-in attempt

Feature restriction (T2).

When document library access is disabled for a user, the API returns a feature restriction response and the client redirects to a dedicated explanation page (Figure ??, Chapter ??).

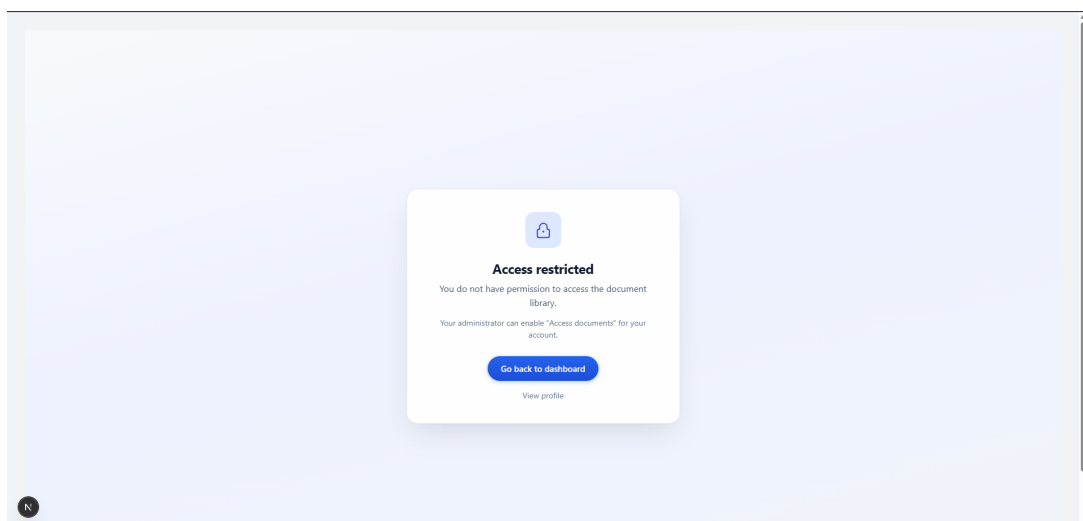


Figure 5.4 – Dedicated page explaining that document library access is disabled

Ingest completion (T3).

After upload and asynchronous processing, the file detail panel shows processing status **READY**, confirming the ingest worker completed embedding and chunk persistence.

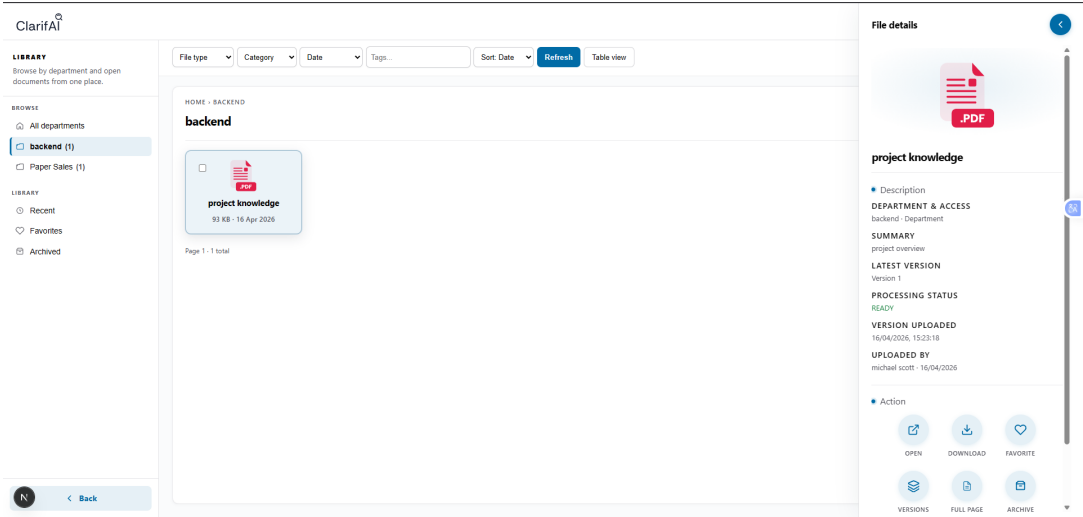


Figure 5.5 – Document detail panel showing READY ingest status

Hybrid Ask with citations (T4).

The Ask interface returns a structured answer with confidence indicator and clickable source citations grounded in retrieved chunks.

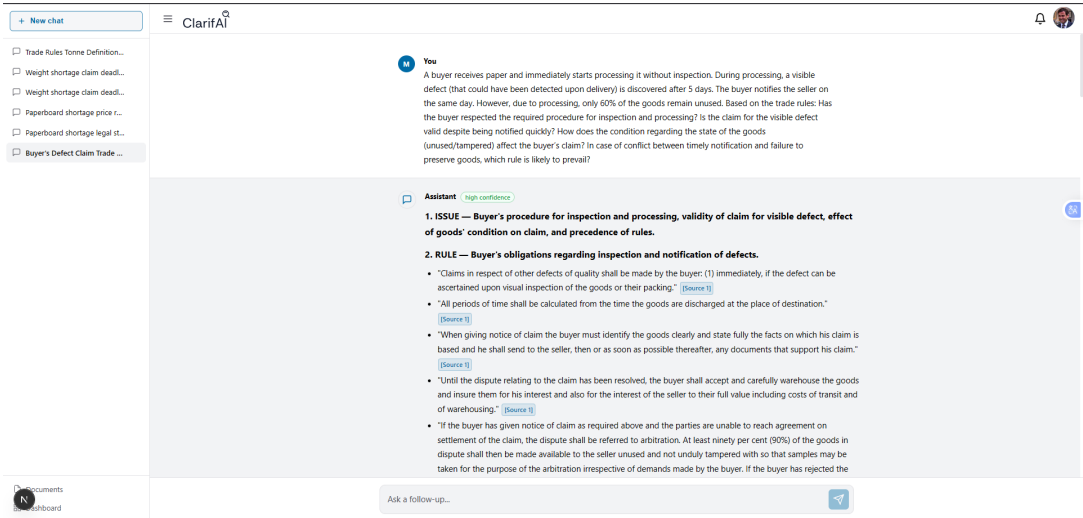


Figure 5.6 – Ask response with structured answer and source citations

Manager department overview (T5).

Managers view a read-only team directory and department-scoped document list for manageable departments.

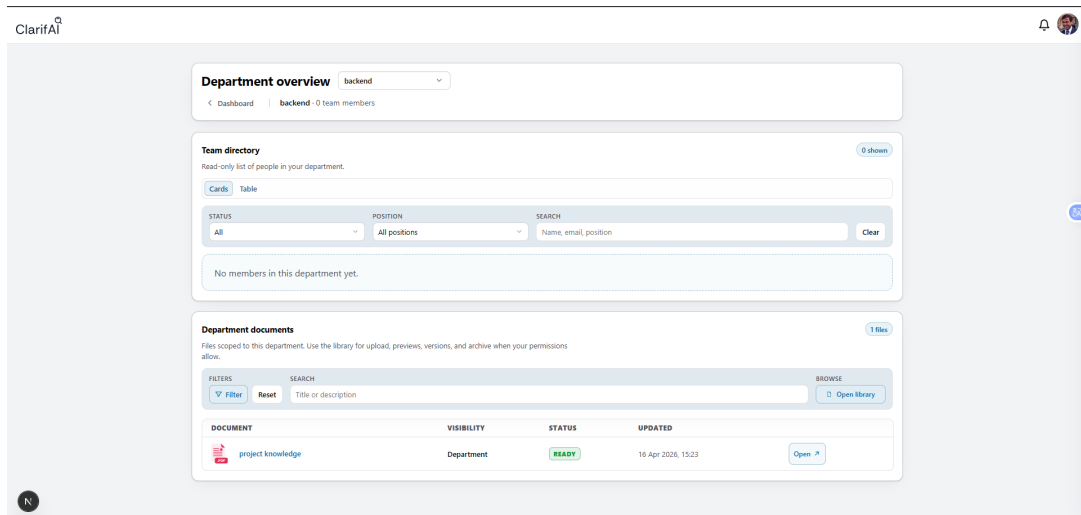


Figure 5.7 – Manager department overview with team directory and documents

5.4 Critical discussion

5.4.1 Strengths

The platform demonstrates end-to-end engineering coherence: traceable requirements, modular architecture, broad automated test coverage (74/74), and a documented RAG pipeline that goes beyond a minimal chat demo (hybrid retrieval, reranking, SSE, feedback, evaluation harness). Manual UX validation confirms that security and restriction flows are understandable to end users, not only correct at the API layer.

5.4.2 Deviations from targets

- **RAG judge KPI** — full faithfulness/relevance run pending; marked **Partial** in Table ??;
- **Scale testing** — no formal load test for concurrent Ask streams was executed; throughput remains unvalidated;

- **RAG topic eval** — 15/34 meets the degraded threshold but does not demonstrate production-grade query optimisation without API credentials.

5.4.3 Limitations

- **External LLM dependency** — latency, cost, and availability depend on Google Gemini;
- **Storage** — default local disk is not cloud-native object storage;
- **RAG metrics** — automated judge is itself an LLM; human evaluation remains valuable;
- **Corpus dependency** — answer quality rises with ingest completeness and chunking quality.

5.4.4 Threats to validity and confidence level

Integration tests use a seeded database smaller than production corpora; RAG eval cases are code-defined and may not cover all host-company domains; manual UX tests were run on a developer workstation.

Confidence assessment:

- **High** — API contracts, RBAC, document access, and notification lifecycles (74/74 integration tests);
- **Medium** — RAG answer quality and citation faithfulness (eval harness partial without full judge run);
- **Low** — concurrent load, production-scale corpus behaviour, and long-term operational monitoring.

5.4.5 Perspectives

Future work includes SSO/LDAP integration, S3-compatible storage, on-prem embedding models, horizontal worker scaling, multilingual UI, CI-integrated RAG regression when API keys are available, and field deployment with user acceptance testing on real organisational corpora.

5.5 Chapter Conclusion

This chapter validated the Knowledge Platform against the KPIs and sprint domains defined in Chapter ???. Automated integration tests passed without failure (74/74), ingest reliability exceeded the 95% target on the seeded corpus, and five manual UX scenarios passed with screenshot evidence. The RAG judge KPI remains partial pending a full evaluation run with API credentials, and scale testing was not performed. Critical discussion identified high confidence in API-level correctness, medium confidence in RAG quality metrics, and acknowledged limitations related to external LLM dependency and test corpus size. The general conclusion synthesises these outcomes relative to the objectives stated in the introduction.

General conclusion and perspectives

This final-year project addressed a practical problem faced by many organisations: internal knowledge exists, but it is hard to find, hard to govern, and hard to use confidently in daily work. Conducted at HyprCraft Solutions, the project produced the Knowledge Platform—a system that brings together document management, controlled access, semantic search, and cited AI-assisted answers within a single coherent experience.

The platform delivers a secure multi-role environment in which documents can be uploaded, indexed, searched, and discussed through persistent conversations. Retrieval combines semantic and keyword methods so that answers remain anchored in organisational sources. Administrative and managerial capabilities support user provisioning, department organisation, notifications, and activity review. From a validation standpoint, automated tests confirmed core behaviour across authentication, documents, search, conversations, and governance, while manual scenarios verified important user-facing flows such as sign-in, document readiness, hybrid Ask, and profile management.

Relative to the objectives stated in the introduction, the project achieved its main goals: a role-aware secure platform, improved retrieval over heterogeneous documents, intelligent assistance with source transparency, structured validation, and a reproducible deployment model. Remaining limitations include reliance on an external language-model provider, the absence of large-scale performance benchmarking, and evaluation sensitivity to corpus coverage and quality.

Beyond technical delivery, the project strengthened competencies in requirements engineering, system design, information retrieval, AI integration, security awareness, and critical evaluation of intelligent systems—skills directly relevant to enterprise software engineering and data-oriented practice.

Future work could extend the platform through cloud object storage, enterprise single

sign-on, on-premise model options, tighter continuous-evaluation pipelines for answer quality, mobile access, and field deployment with real corpus onboarding and formal user acceptance testing at the host organisation.

Bibliography

- [1] Y. Gao, Y. Xiong, X. Gao, K. Liu, J. Liu, H. Wang, and J. Zhang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024, accessed: July 2026.
- [2] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, “Dense passage retrieval for open-domain question answering,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781, 2020.
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.
- [5] O. Ram, Y. Levine, I. Dalmedigos, D. Jury, O. Kabeli, A. Shashua, K. Leyton-Brown, and Y. Shoham, “In-context retrieval-augmented language models,” *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023.
- [6] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Nogueira, and J. Lin, “Pyserini: A python toolkit for reproducible information retrieval research with sparse and dense representations,” in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 2356–2362.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural*

- Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [8] S. Es, J. James, F. Anton, L. Ott, M. Velizhev, and M. Buchwald, “RA-GAS: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2024, accessed: July 2026.
- [9] R. S. Sandhu, E. J. Coyne, H. J. Feinstein, and C. E. Youman, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [10] D. F. Ferraiolo, D. R. Kuhn, and R. Chandramouli, “Role-based access control,” *Artech House Computer Security Series*, 2003.
- [11] Microsoft Corporation, “Sharepoint documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://learn.microsoft.com/en-us/sharepoint/>
- [12] Pinecone Systems Inc., “Pinecone vector database documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://docs.pinecone.io/>
- [13] LangChain Inc., “Langchain documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://python.langchain.com/docs/>
- [14] LlamaIndex Inc., “Llamaindex documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://docs.llamaindex.ai/>
- [15] Microsoft Corporation, “Microsoft copilot for microsoft 365,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://www.microsoft.com/en-us/microsoft-365/copilot>
- [16] pgvector Contributors, “pgvector: Open-source vector similarity search for postgresql,” 2024, [Online]. Accessed: July 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [17] Google DeepMind, “Gemini api documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs>
- [18] Internet Engineering Task Force, “JSON Web Token Best Current Practices,” 2022, rFC 8725. [Online]. Accessed: July 2026.

CHAPTER A

Validation Summary

Table A.1 – Requirements Traceability Matrix

Req.	Design choice	Implemented element	Validation method	Status
FR-01	JWT with refresh rotation	Authentication service	Automated auth tests	Validated
FR-02	Profile and avatar handling	Profile management service	Automated profile tests	Validated
FR-03	Asynchronous document ingest	Ingestion pipeline	Automated document tests	Validated
FR-04	Department visibility rules	Access control layer	Automated document tests	Validated
FR-05	Library organisation features	Document library service	Automated document tests	Validated
FR-06	Semantic search	Search service	Automated search tests	Validated
FR-07	Hybrid cited answers	RAG completion service	Search and manual Ask tests	Validated
FR-08	Conversation history	Conversation service	Automated conversation tests	Validated
FR-09	Notification delivery	Notification service	Automated notification tests	Validated
FR-10	Administrative governance	Administration module	Automated admin tests	Validated
FR-11	Manager department view	Manager module	Automated manager tests	Validated
FR-12	Per-user feature restrictions	Authorization middleware	Auth and search tests	Validated

Table A.2 – Requirements Traceability Matrix (continued)

Req.	Design choice	Implemented element	Validation method	Status
NFR-01	Service health monitoring	Health endpoint	Automated health tests	Validated
NFR-02	HTTP security hardening	Application middle-ware	Integration tests and review	Validated
NFR-03	Safe session re-fresh	Client refresh strat-egy	Automated auth tests	Validated
NFR-04	Reproducible deployment	Container packag-ing	Manual deployment check	Validated
NFR-05	Automated regres-sion suite	Integration test suite	Local and CI execution	Validated
NFR-06	RAG quality eval-uation	Evaluation harness	Quick eval workflow	Partial

Table A.3 – Technology Architecture Comparison Matrix

Criterion	Mono-repo	Django	Micro-services	Rationale
Time to market	3	2	1	Shared TypeScript types; one repository
SSE stream-ing fit	3	2	2	Native Express streaming for RAG
Team skill alignment	3	2	1	Matches React + Node stack
Operational complexity	3	3	1	Single API process + worker

Table A.4 – Functional Validation Test Summary

T	Objective	Expected	Observed	Status
T1	Reject invalid credentials safely	Generic error, no enumeration	Clear sign-in error	Passed
T2	Explain disabled library access	Dedicated restricted page	Restricted page shown	Passed
T3	Confirm ingest completes	Status shown as complete	Ready on detail panel	Passed
T4	Verify cited Ask answers	Answer with citations	Answer with citations	Passed
T5	Show manager overview	Dept. team and documents	Team and files listed	Passed

CHAPTER B

Scientific Integrity and Reproducibility

B.1 Declaration of AI Tool Usage

Artificial intelligence tools were used as auxiliary resources during the development of this project to support productivity and improve writing clarity. Their use was limited to assistance with drafting, refining technical content, and providing general programming guidance. All technical decisions, system design, implementation, experimental results, and conclusions were independently developed, verified, and validated by the author, who assumes full academic responsibility for the final submitted work.

B.2 Reproducibility Details

The following parameters define the minimum environment needed to reproduce the reported results. Commands may be run from the repository root unless noted.

Table B.1 – Reproducibility parameters

Parameter	Value
Repository	Monorepo with API and web applications (<code>apps/api</code> , <code>apps/web</code>)
Database	PostgreSQL 16 + pgvector; migrations via <code>npm run db:migrate</code> ; seed via <code>npm run db:seed</code>
Services	<code>npm run docker:up</code> for PostgreSQL and Redis
Environment	API <code>.env</code> (JWT secret, Gemini API key, database URL); Web <code>.env.local</code> (public API URL)
Run	<code>npm run dev</code> from repository root (API port 3001, web port 3000)
Tests	<code>npm run test:api</code> ; RAG eval: <code>npm run eval:rag:quick</code> or <code>npm run eval:rag</code>
Prompt versioning	<code>RAG_PROMPT_VERSION</code> environment variable invalidates answer cache

B.3 Integrity Checklist

- ✓ Front matter complete (cover, dedication, acknowledgements, abstract, table of contents, lists, abbreviations).
- ✓ Introduction announces problem statement, objectives, and report plan.
- ✓ Each chapter follows need → design → implementation → validation logic.
- ✓ Conclusion explicitly answers the stated objectives.
- ✓ Design choices justified with comparison tables (Chapters ??–??).
- ✓ Validation protocol documented with 74 integration tests (Chapter ??).
- ✓ Limitations and threats to validity discussed.
- ✓ Every borrowed figure, table, or idea is cited in the bibliography.
- ✓ Results are interpreted critically (limitations in Chapter ??).
- ✓ Figures and tables numbered and referenced in text.
- ✓ Generative AI usage declared in Section ??.

- ● Similarity checking target: $< 15\%$ (institutional plagiarism tool).
- ● Run institutional similarity check before final submission.